VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF ELECTRONICS – TELECOMMUNICATIONS

BUI THANH DAT

Open-Source RISC-V SoC Implementation with DNN Acceleration: A Chipyard Framework Study with FPGA Prototyping

UNDERGRADUATE THESIS
MAJOR OF ELECTRONICS – TELECOMMUNICATIONS ENGINEERING

Ho Chi Minh City, 07/2025

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF ELECTRONICS – TELECOMMUNICATIONS

Bui Thanh Dat
Student ID: 21207001

Open-Source RISC-V SoC Implementation with DNN Acceleration: A Chipyard Framework Study with FPGA Prototyping

UNDERGRADUATE THESIS
MAJOR OF ELECTRONICS – TELECOMMUNICATIONS ENGINEERING

SUPERVISOR
Huynh Huu Thuan, Ph.D.

Ho Chi Minh City, 07/2025

XÁC NHẬN HOÀN TẤT CHỈNH SỬA BÁO CÁO TỐT NGHIỆP

TÊN ĐỀ TÀI: Nghiên cứu và Triển khai Hệ thống SoC RISC-V với Bộ tăng tốc xử lý DNN sử dụng Nền tảng mã nguồn mở Chipyard cho việc Xây dựng và Thử nghiệm trên FPGA.

Tên tiếng anh: Open-Source RISC-V SoC Implementation with DNN Acceleration: A Chipyard Framework Study with FPGA Prototyping.

Họ và tên sinh viên: Bùi Thành Đạt
Mã số sinh viên: 21207001

Hội đồng đánh giá họp ngày..... tháng ..... năm.....

1. Chủ tịch:
2. Thư ký:
3. Ủy viên:
4. Ủy viên:
5. Ủy viên:

Người hướng dẫn: TS. Huỳnh Hữu Thuận.
Xác nhận của người hướng dẫn:

ACKNOWLEDGEMENTS

First and foremost, I wish to express my deepest thanks to all the members of the Computer Embedded Systems Laboratory (CESLAB). Their extensive support, both throughout my academic years and during the intensive work on this thesis, created an environment of collaboration and learning that was essential for my success.

I owe a special debt of gratitude to my supervisor, Dr. Huynh Huu Thuan. His encouragement from my freshman year was pivotal in starting my research journey, and his diligent guidance, invaluable advice, and mentorship were instrumental in the completion of this work.

On a personal note, I extend a heartfelt thanks to my family for their unconditional trust and support, regardless of my choices. I am also deeply thankful for my friends, who stood by me even when I was at the lowest point of my undergraduate life.

I am also immensely grateful to the researchers and professors, both within the Faculty of Electronics - Telecommunications (FETEL) and from universities both within Vietnam and across the USA, Australia, and Denmark, who offered their time and wisdom. Their mentorship and insights into academia were profoundly inspiring and helped shape my decision to pursue a research career.

Furthermore, I wish to acknowledge the global open-source community. For a student from a developing country, access to high-quality tools and shared knowledge is a profound blessing that breaks down barriers to research and innovation. A special thank you is due to Eugene Tarassov, Principal Member of Technical Staff at AMD, whose `vivado-risc-v` open-source repository served as a foundational tool for this study.

Finally, I extend my appreciation to all the instructors and staff at the Faculty of Electronics - Telecommunications (FETEL) for their dedication to education.

TRANG CAM KẾT TÍNH TRUNG THỰC CỦA BÁO CÁO

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong khóa luận tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định.

Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

Bùi Thành Đạt

TÓM TẮT

Nhu cầu tính toán ngày càng tăng của các Mạng Nơ-ron Sâu (DNN) đã bộc lộ những hạn chế của bộ xử lý đa dụng truyền thống, từ đó đặt ra yêu cầu cấp thiết cho các bộ tăng tốc phần cứng chuyên dụng. Mặc dù FPGA đã được sử dụng như một nền tảng để xây dựng mô hình nguyên mẫu, việc từ nguyên mẫu đó chuyển đổi một cách có hiệu quả sang các giải pháp Mạch tích hợp chuyên dụng (ASIC) có khả năng mở rộng hơn đòi hỏi một phương pháp thiết kế mã nguồn mở và toàn diện. Luận văn này trình bày việc thiết kế, triển khai full-stack và đánh giá một Hệ thống trên Vi mạch (SoC) RISC-V có khả năng chạy hệ điều hành Linux, và được tích hợp một bộ tăng tốc DNN chuyên dụng. Mục tiêu chính của luận văn là ứng dụng và là minh chứng cho một quy trình làm việc trên một chuỗi công cụ mã nguồn mở hoàn chỉnh, từ cấu hình phần cứng mức cao bằng Chisel cho đến việc xây dựng nguyên mẫu trên FPGA. Hệ thống được thiết kế trên nền tảng Chipyard, tích hợp lõi xử lý 64-bit RISC-V Rocket Core với bộ tăng tốc Gemmini có kiến trúc mảng xử lý song song theo nhịp (systolic array) thông qua giao thức kết nối RoCC. Hệ thống SoC đã được triển khai thành công trên bo mạch FPGA Xilinx (AMD) Virtex 7 VC707, khởi chạy thành công hệ điều hành Debian Linux. Kết quả đánh giá hiệu năng cho thấy bộ tăng tốc Gemmini đạt được mức tăng tốc xấp xỉ 958 lần trên một tác vụ tích chập tiêu biểu so với giải pháp phần mềm thuần túy trên Rocket Core. Công trình này đã xác thực thành công nền tảng Chipyard là một hướng đi khả thi cho việc thiết kế SoC tiên tiến, phù hợp cho việc phát triển thiết kế full-stack, có thể tái tạo cho các nghiên cứu trong tương lai về lĩnh vực tăng tốc phần cứng chuyên dụng, tạo tiền đề cho việc phát triển nguyên mẫu trên cả hai nền tảng FPGA và ASIC.

ABSTRACT

The increasing computational demand of Deep Neural Networks (DNNs) has exposed the limitations of traditional general-purpose processors, creating a need for specialized hardware accelerators. While FPGAs have served as a prototyping platform, transitioning to more efficient and scalable Application-Specific Integrated Circuit (ASIC) solutions requires a comprehensive, open-source design methodology. This thesis presents the design, full-stack implementation, and evaluation of a Linux-capable RISC-V System-on-Chip (SoC) featuring a dedicated DNN accelerator. The primary objective is to demonstrate a complete open-source workflow, from high-level hardware configuration in Chisel to FPGA prototyping. The system was designed using the Chipyard framework, integrating a 64-bit RISC-V Rocket Core with the Gemmini systolic array accelerator via the RoCC interface. The implemented SoC was successfully deployed on a Xilinx VC707 FPGA board, booting a full Debian Linux operating system. Performance evaluation revealed that the Gemmini accelerator achieved an approximate **958x speedup** on a representative convolution task compared to a software-only implementation on the Rocket Core. This work successfully validates the Chipyard framework as a viable path for advanced SoC design and establishes a reproducible, full-stack foundation for future research in specialized hardware acceleration for prototyping on both FPGA and ASIC platforms.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS iii

TÓM TẮT vii

ABSTRACT ix

LIST OF FIGURES xvii

LIST OF TABLES xxi

1 Introduction 1

1.1 Background and Motivation 1
1.2 A Survey of Modern Open-Source SoC Frameworks 2
1.2.1 LiteX: The Flexible SoC Builder 2
1.2.2 PULP Platform: The Low-Power Parallel Computing Specialist 2
1.2.3 OpenTitan: The Security-First Root of Trust 3
1.3 Chipyard: the framework with Full-Stack Integration Capability 3
1.4 Core Selection within Chipyard: A Comparison of Rocket and BOOM 4
1.5 Research Objectives and Contributions 5
1.6 Significance and Impact 6
1.7 Thesis Structure and Organization 7

2 RISC-V Microprocessor Instruction Set Architecture 9

2.1 Overview of RISC-V 9
2.1.1 RISC-V 32-bit and the RV32I Base Instruction Set 11
2.1.2 RISC-V 64-bit and the RV64I Base Instruction Set 15
2.2 RISC-V Processor Cores 18
2.2.1 Pipeline Architecture in Microprocessors 18
2.2.2 Out-of-Order Architecture in Microprocessors 20
2.2.3 Open-Source RISC-V Processor Cores 20
2.3 System-on-Chip Design Framework: Chipyard - Rocket Chip 23

3 A Single-Core with Gemmini RoCC Accelerator RISC-V Processor System 27

3.1 The Chipyard Compilation and Elaboration Flow 27

3.2 System Design Methodology with Chisel . . . 29
3.2.1 Chisel and Scala . . . 29
3.2.2 Verilator Simulation Tool . . . 31
3.3 The Rocket Processor Core . . . 31
3.4 Rocket Chip SoC Structure . . . 32
3.4.1 Bus and Interconnect . . . 33
3.4.2 Memory and Cache . . . 37
3.4.3 Core-Complex . . . 38
3.4.4 Peripheral Components . . . 39
3.4.5 The RoCC Interface: A Hardware Co-Design Methodology . . . 42

4 Architectural Foundations of DNN Acceleration 49
4.1 The Data-Compute Divide: The Memory Wall . . . 49
4.2 A Classic Solution: The Systolic Principle . . . 50
4.3 Case Studies in Modern Systolic Design . . . 51
4.3.1 Google's TPUv1: A Datacenter-Scale Design . . . 52
4.3.2 NVIDIA NVDLA: An Open-Source Accelerator IP . . . 53
4.4 A Formal Framework: Dataflows . . . 54
4.5 The Gemmini Approach: A Full-Stack Research Platform . . . 56
4.6 Chapter Summary . . . 58

5 The Gemmini DNN Accelerator: An Architectural Overview 59
5.1 Top-Level Architectural Blueprint . . . 59
5.2 Analysis of Core Architectural Components . . . 59
5.2.1 The Systolic Compute Engine . . . 59
5.2.2 The On-Chip Memory Subsystem . . . 62
5.3 SoC Integration and System-Level Features . . . 63
5.3.1 The RoCC Coprocessor Interface . . . 63
5.3.2 Virtual Memory Support . . . 63
5.4 The Multi-Level Programming Model . . . 64
5.4.1 Low-Level: Gemmini ISA . . . 64
5.4.2 Mid-Level: Hand-Tuned C Library . . . 65
5.4.3 High-Level: Compiler Support (Conceptual) . . . 65
5.5 Summary . . . 65

6 Gemmini's Internal Architecture and Programming Model 67
6.1 The Command-Unrolling Hierarchy . . . 68
6.1.1 Level 1: Convolution Decomposition (`LoopConv`) . . . 68
6.1.2 Level 2: Tiled Matrix Multiplication (`LoopMatmul`) . . . 70
6.2 Decoupled Execution and Resource Management . . . 72
6.2.1 The Reservation Station: A Central Dispatch Hub . . . 72
6.2.2 Back-End Controllers: Load, Store, and Execute . . . 73
6.3 The Physical Hardware: Compute and Memory . . . 74
6.3.1 The Processing Element (PE) and Distributed Arithmetic . . . 74
6.3.2 The Tile and The Mesh: A Tale of Two Hierarchies . . . 75
6.3.3 Architectural Trade-offs in Mesh Configuration . . . 77
6.3.4 The Scratchpad: An Intelligent Memory Subsystem . . . 78
6.4 Chapter Summary . . . 79

7 System Implementation and Testing 81
7.1 Hardware Implementation: SoC Configuration . . . 81
7.1.1 The `Rocket64b1gem16jtag` Configuration . . . 83
7.2 Gemmini Integration via RoCC . . . 85
7.2.1 The RoCC Interface . . . 85
7.3 Peripheral Integration . . . 87
7.3.1 DDR Memory: High-Bandwidth Main Memory . . . 87
7.3.2 SD Card: AXI SD Card Controller and SD Mode . . . 88
7.3.3 UART: AXI UART Controller . . . 89
7.3.4 Ethernet: AXI Ethernet Controller . . . 90
7.3.5 System Monitoring: XADC and Fan Control . . . 91
7.4 Development Environment . . . 91
7.5 Software Stack and Build Process . . . 91
7.5.1 Boot Flow: OpenSBI, U-Boot, and Linux . . . 91
7.5.2 SoC Hardware Generation . . . 92
7.6 Gemmini Testing . . . 93

8 Implementation Results and Discussion 95
8.1 Hardware Implementation Analysis . . . 95
8.1.1 System Block Design and Address Map . . . 95
8.1.2 Vivado Implemented Design Results . . . 96
8.2 Software Stack Validation . . . 102

8.2.1 System Boot Sequence . . . 102
8.2.2 Debian Linux Environment . . . 103
8.3 Gemmini Accelerator Performance Evaluation . . . 104
8.3.1 Functional Correctness . . . 104
8.3.2 Performance Speedup and Analysis . . . 104
8.3.3 Real-World Workload Analysis: MobileNetV1 . . . 105
8.3.4 Discussion of Results . . . 106

9 Conclusion and Future Work **107**

LIST OF PUBLICATIONS **109**

BIBLIOGRAPHY **111**

A MÃ NGUỒN CỦA THIẾT KẾ **115**

B Detailed Processing Units **117**
B.1 ExecuteController Sub-Modules . . . 117
B.2 Scratchpad Integrated Processing Units . . . 118

C RoCC Interface and Gemmini Integration Details **119**
C.1 SoC Configuration and Accelerator Instantiation . . . 119
C.2 RoCC Hardware Interface Bundles . . . 119
C.3 Gemmini Custom Instruction Set . . . 121
C.4 CPU-Side Pipeline Integration Details . . . 121
C.4.1 Command Processing Flow . . . 122
C.4.2 Pipeline Stall and Hazard Management . . . 122
C.4.3 Response and Register File Write . . . 123
C.4.4 Status and Interrupt Handling . . . 123

D Time Measuring Methodology **125**

E Detailed Performance Evaluation Results **135**
E.1 Performance Benchmark Results . . . 135
E.1.1 Matrix Multiplication Performance Analysis . . . 135
E.2 Test Program Documentation and Configurations . . . 136
E.2.1 Matrix Multiplication Tests (bareMetalC/) . . . 136
E.2.2 Convolution Tests (bareMetalC/) . . . 137

E.2.3 Neural Network Models (imagenet/) 139
E.2.4 Multi-Layer Perceptron Models (mlps/) 139
E.2.5 Additional Tests 140

LIST OF FIGURES

2.1 RISC-V is managed by RISC-V International. Source [22] 9
2.2 Instruction encoding formats in RV32I. Source [30] 12
2.3 Instructions in the RV32I base instruction set. [30] 14
2.4 Instructions in the RV64I base instruction set. [30] 16
2.5 Instruction encoding format for ADDIW in 64-bit architecture. [30] . . 16
2.6 Encoding format for 64-bit I-type shift instructions. [30] 17
2.7 Encoding format for 64-bit R-type shift instructions for word operations. [30] 17
2.8 Encoding formats for 64-bit LOAD and STORE instructions. [30] . . . 18
2.9 Basic five-stage pipeline in a RISC machine. The vertical axis is successive instructions; the horizontal axis is time. So in the green column, the earliest instruction is in WB stage, and the latest instruction is undergoing instruction fetch. 19
2.10 Pipeline of Rocket core. Source [2] 21
2.11 Berkeley Out-of-Order Machine Processor Pipeline Architecture. [6] . . 22
2.12 Chipyard, developed by UC Berkeley. Source: [32] 23
2.13 Component structure of Chipyard. Source: [32] 24

3.1 The Chipyard Development and Compilation Flow 28
3.2 Chisel as a library within Scala. 29
3.3 The Chisel to HDL design elaboration flow 30
3.4 Structure of a Rocket Tile. 32
3.5 Proposed structure of a single-core RISC-V processor system with a Gemmini RoCC accelerator. 33
3.6 Connection model in TileLink and its interfaces. Source: [24] 34
3.7 Channels in a TileLink connection between two agents. Source: [24] . . 35
3.8 TileLink TL-UL connection structure. Source: [24] 37
3.9 Implementation structure of a heterogeneous multi-core processor system. 38
3.10 Peripherals in the Rocket Chip base system. 39
3.11 `RoCC` co-processor core and MMIO peripheral core in Rocket Chip. . . 40

3.12 Rocket Tile RoCC Interface Architecture with Accelerator Integration. The RoCC interface provides direct communication pathways between the Rocket Core's datapath and the attached accelerator, including command dispatch, memory requests, and response handling channels. 42
3.13 The RoCC Instruction Format. The `opcode` field is set to `custom3` (0b1111011). The `funct7` field specifies the Gemmini-specific operation, while the `xd`, `xs1`, and `xs2` bits act as control signals for the hardware interface. 43
3.14 Diagram illustrating the RoCC co-design and control flow. (1) A RoCC instruction is recognized at the Decode stage. (2) At the Writeback stage, the `RoCCCommand` is dispatched to Gemmini over a `ready/valid` channel. (3) A `busy` signal provides back-pressure to the CPU's Hazard Unit, stalling the pipeline. (4) The `resp` path provides a privileged write-back directly to the Register File. 44
3.15 The asymmetrical RoCC memory interface. The request channel (`mem_req`) uses a full `ready/valid` handshake, while the response channel (`mem_resp`) is a simpler `valid`-only interface. 46
3.16 Timing diagram for a single RoCC memory request. The diagram demonstrates the two-cycle latency of the memory interface, as the `mem_resp_valid` signal is asserted two clock cycles after the `mem_req_valid` signal. Note the absence of a `mem_resp_ready` signal, confirming the asymmetrical nature of the interface. [21] 47
3.17 Timing diagram for consecutive RoCC memory requests. This illustrates the 50% maximum throughput of the interface. The L1D$ can accept two back-to-back requests but then requires a one-cycle pause before accepting the next two, a critical performance characteristic for data-intensive accelerators. [21] 47

4.1 The Relative Energy Cost of Computation vs. Memory Access 50
4.2 Conventional vs. Systolic Processing 51
4.3 The fundamental principle of a systolic array for matrix multiplication. Input matrix A flows horizontally, while input matrix B flows vertically. This rhythmic dataflow allows for massive parallelism and data reuse. . 52
4.4 The Google TPUv1 Block Diagram 53
4.5 The NVDLA High-Level Architecture 54
4.6 Data Reuse Opportunities in DNNs 55

4.7 Taxonomy of Common DNN Dataflows 55
4.8 Architectural Flexibility of the Gemmini Generator 57
4.9 Comparison of OS and WS Dataflows 57

5.1 Gemmini Logo. [11]. 59
5.2 High-level overview of the Gemmini accelerator template integrated with a host CPU and system memory hierarchy. [11]. 60
5.3 Microarchitecture of Gemmini's Spatial Array 60
5.4 The im2col Transformation 62
5.5 Conceptual view of Gemmini's decoupled access-execute pipelines, managed by separate hardware command queues for Load, Execute, and Store operations. [11]. 64

6.1 Overview of the internal modules in the generated Gemmini hardware architecture, showing the flow of commands from the RoCC interface through various queues and controllers. 67
6.2 The `LoopConv` module. This is the first level of the command-unrolling hierarchy, responsible for decomposing a complex convolution operation into a series of matrix multiplication tasks. 69
6.3 The `LoopMatmul` module. This second-level unroller translates matrix multiplication tasks into a stream of smaller, tiled micro-operations for the physical hardware. 70
6.4 The primary back-end controllers that receive commands from the Reservation Station. 73
6.5 A single Processing Element (PE), the fundamental compute unit. It contains a Multiply-Accumulate (MAC) unit and the necessary logic to support multiple dataflows, such as Output Stationary and Weight Stationary. 75
6.6 The Multiply-Accumulate (MAC) unit architecture. It is implemented using the `Arithmetic.scala` typeclass, which provides a suite of operations and supports multiple data types, including integers and IEEE 754 floating-point numbers. 76
6.7 Different Mesh configurations, illustrating the trade-off between the number of tiles and the number of PEs per tile, while maintaining the same total number of 16 PEs. 77

6.8 The internal structure of the Scratchpad module, an intelligent memory subsystem with integrated DMA engines and data processing units that serves the systolic array. 78

7.1 The implemented Rocket SoC with Gemmini on the VC707 FPGA board. The Rocket Core is connected to a 16x16 Gemmini systolic array, and various peripherals are integrated via AXI interfaces. 82
7.2 Gemmini Configuration on `Rocket64b1gem16jtag` SoC. 85
7.3 SD Mode: Multi-Block Read Operation. [3] 88
7.4 SD Mode: Multi-Block Write Operation. [3] 88
7.5 SPI Mode: Multi-Block Read Operation. [3] 89
7.6 SPI Mode: Multi-Block Write Operation. [3] 89

8.1 Block Design of the System, showing the top-level integration of the RocketChip core, DDR, and IO subsystems. 96
8.2 Block Design of the IO Subsystem, illustrating the AXI interconnect for peripherals such as Ethernet, SD Card, and UART. 97
8.3 Block Design of the DDR Subsystem, detailing the Memory Interface Generator (MIG) and its AXI connection. 98
8.4 Address Editor view, confirming the memory map for the integrated peripherals. 98
8.5 Detailed Utilization Summary for the entire SoC on the VC707 FPGA. 99
8.6 Hierarchical Utilization of the Gemmini accelerator module. 99
8.7 Post-implementation on-chip power summary. 100
8.8 Hierarchical power report for the Gemmini accelerator. 101

LIST OF TABLES

2.1 RISC-V Standard 32-bit ABI Register Conventions (RV32I). [30] . . . 12
2.2 Common RISC-V Pseudo-instructions. [30] 13
2.3 Development status of extended RISC-V instruction sets. [30] 15

3.1 TileLink operational configurations. Source: [24] 34

8.1 Summary of Gemmini performance speedup across key operational categories. Detailed results for each test case are provided in Appendix E. 105
8.2 Cycle breakdown for a single inference pass of MobileNetV1 on the Gemmini accelerator. 106

E.1 Detailed Matrix Multiplication Performance 135
E.2 Matrix Multiplication Performance Analysis 135
E.3 Detailed Convolution Performance 135
E.4 Detailed Convolution with Pooling Performance 136
E.5 Detailed Residual Addition Performance 136
E.6 Detailed MLP Model Performance 136

Chapter 1: Introduction

This thesis presents the design, implementation, and evaluation of a RISC-V System-on-Chip (SoC) featuring a specialized Deep Neural Network (DNN) accelerator, conducted within the Computer Embedded Systems Laboratory (CESLAB) of the University of Science, VNUHCM (HCMUS). This work represents a significant step in our laboratory's exploration of the emerging RISC-V ecosystem and open-source SoC design automation solutions for future chip development and fabrication.

1.1 Background and Motivation

The rapid advancement of artificial intelligence and machine learning has created an unprecedented demand for specialized computing architectures. Deep Neural Networks, which form the backbone of modern AI applications, require enormous computational resources that traditional general-purpose processors struggle to provide efficiently. This computational challenge, combined with the exponential growth in model complexity and the need for energy-efficient solutions, has driven the development of custom hardware accelerators specifically designed for DNN workloads.

Traditionally, the CESLAB has been involved in designing and developing hardware accelerators (or IP Cores) on FPGA using Intel Quartus, particularly leveraging the Platform Designer tool with Nios II soft processors and Avalon Bus interfaces. While this approach has served well for FPGA-based prototyping, it presents significant limitations for our laboratory's strategic evolution toward chip fabrication and VLSI design. The Avalon Bus, though functional, lacks the sophistication and industry adoption of modern interconnect standards such as AXI4 and TileLink. More critically, the Intel Quartus ecosystem is primarily optimized for FPGA implementation and lacks the comprehensive VLSI design flow capabilities essential for ASIC development and chip fabrication.

As CESLAB envisions a future centered on chip prototyping and semiconductor fabrication, the limitations of our current toolchain have become increasingly apparent. The proprietary nature of Intel's tools not only imposes licensing costs but also restricts our ability to customize the design flow, integrate with modern VLSI tools, and contribute to the broader open-source hardware community. Furthermore, the transition from FPGA prototypes to actual silicon requires a design methodology that seamlessly bridges both domains—a capability that traditional FPGA-centric tools

cannot provide.

The emergence of the RISC-V instruction set architecture has fundamentally changed the landscape of processor design and SoC development, offering a pathway to address these limitations. As an open-source ISA, RISC-V eliminates the barriers imposed by proprietary architectures, providing unprecedented flexibility for customization and modification. More importantly, this openness has catalyzed the development of comprehensive design ecosystems that support the entire SoC development flow, from high-level specification through FPGA prototyping to final VLSI implementation and chip fabrication.

1.2 A Survey of Modern Open-Source SoC Frameworks

To address the limitations of our previous proprietary toolchain and select a platform aligned with CESLAB's strategic goal of VLSI development, we conducted a survey of prominent open-source System-on-Chip (SoC) design frameworks. The landscape is rich with powerful, mature options, each with a distinct design philosophy and target application domain. Key alternatives considered include the LiteX, PULP, and OpenTitan platforms.

1.2.1 LiteX: The Flexible SoC Builder

LiteX is a versatile framework that utilizes Migen, a Python-based hardware description language, to facilitate the rapid construction of custom SoCs [7, 15]. Its primary strength lies in its flexibility, offering a vast library of general-purpose IP and support for over 150 FPGA boards from a wide range of vendors [18]. While LiteX provides an exceptional environment for rapid FPGA prototyping and experimentation, its toolchain is not inherently optimized for a direct, end-to-end VLSI design flow in the same way as other frameworks. For our laboratory's specific goal of a seamless transition from FPGA prototyping to ASIC implementation, LiteX presented a less direct path.

1.2.2 PULP Platform: The Low-Power Parallel Computing Specialist

The PULP (Parallel Ultra-Low-Power) Platform, developed at ETH Zurich, is a leader in energy-efficient, parallel computing for edge AI and IoT applications [28]. The platform is built around a family of highly-optimized RISC-V cores and has

been validated through an impressive 51 silicon tape-outs, demonstrating its maturity and industrial relevance [27]. However, the entire PULP architecture is meticulously optimized for its specific domain of ultra-low-power, multi-core data processing. Our project required a general-purpose SoC architecture capable of integrating a large, tightly-coupled, memory-intensive DNN accelerator, a different architectural paradigm than PULP's primary focus.

1.2.3 OpenTitan: The Security-First Root of Trust

OpenTitan is a high-profile, open-source project managed by lowRISC to create a transparent and verifiable silicon "root of trust" (RoT) [26]. The framework, backed by a coalition including Google, provides a comprehensive suite of security-hardened IP, such as cryptographic accelerators and secure boot managers, built around the Ibex RISC-V core [19]. While OpenTitan represents a landmark achievement in open-source hardware security, its purpose is highly specialized. It is engineered to provide verifiable security, not high-performance computation. As our thesis is focused on DNN acceleration research, the OpenTitan framework was not a suitable foundation.

1.3 Chipyard: the framework with Full-Stack Integration Capability

Among the various RISC-V SoC design frameworks, Chipyard has emerged as the most comprehensive and mature solution for academic research and industrial VLSI development. Developed at UC Berkeley, Chipyard provides a complete, open-source SoC design environment that spans from high-level hardware generators written in Chisel to full VLSI implementation flows optimized for ASIC fabrication. This end-to-end capability is precisely what CESLAB requires to transition from FPGA-based prototyping to actual chip development.

The framework's technical superiority over traditional FPGA design flows is evident in several key areas. Unlike the proprietary Avalon Bus used in Intel's ecosystem, Chipyard leverages industry-standard interconnects including TileLink and AXI4. AXI4, in particular, has become the de facto standard for modern SoC design due to its superior performance characteristics, extensive tool support, and widespread adoption across the semiconductor industry. This adherence to industry standards ensures that designs developed using Chipyard can seamlessly integrate with commercial IP cores and industry-standard VLSI tools.

Most critically, Chipyard includes a fully integrated, open-source VLSI flow that

encompasses synthesis, place-and-route, and timing closure using industry-standard tools such as Cadence Genus and Innovus. This VLSI-optimized design flow enables the direct translation of high-level hardware descriptions into fabrication-ready layouts, supporting multiple process technologies including advanced nodes from foundries like TSMC and GlobalFoundries. This capability is essential for CESLAB's strategic goal of developing chip prototypes and eventual commercial silicon products.

A key innovation of Chipyard is its emphasis on *full-stack integration*. Unlike traditional approaches that design and evaluate accelerators in isolation, Chipyard enables the creation of complete, Linux-capable systems that capture real-world effects such as operating system overhead, memory hierarchy interactions, and system-level resource contention. This holistic approach is crucial for understanding the true performance characteristics of specialized accelerators when deployed in practical computing environments—insights that are invaluable when transitioning from FPGA prototypes to optimized ASIC implementations.

The framework includes the Gemmini accelerator generator, an open-source DNN accelerator specifically designed for systolic array-based matrix computations. Gemmini embodies the principles of efficient DNN acceleration through its configurable systolic array architecture, which maximizes data reuse and minimizes the costly movement of data between computational units and external memory—a fundamental challenge known as the "memory wall" in computer architecture.

1.4 Core Selection within Chipyard: A Comparison of Rocket and BOOM

A key advantage of the Chipyard framework is that it is a generator capable of instantiating different processor cores. This presented a critical design choice for our system: selecting the CPU core that best aligned with the project's objectives. The two primary candidates developed at UC Berkeley and available within Chipyard are the Rocket Core and the Berkeley Out-of-Order Machine (BOOM).

- The Rocket Core is a mature, stable, and widely-used 5-stage, single-issue, in-order processor core. It is highly configurable and provides a solid foundation for general-purpose computing, capable of booting full operating systems like Linux. Its relative simplicity and extensive documentation make it a reliable and robust choice for systems where the primary focus is on integrating other components, such as custom accelerators [2].
- The Berkeley Out-of-Order Machine (BOOM) is a synthesizable, parameter-

ized, out-of-order RISC-V core designed for high-performance applications [4, 6]. It features a much more complex microarchitecture than Rocket, with a deeper pipeline and sophisticated hardware for dynamic scheduling, register renaming, and speculative execution. BOOM is designed to compete with high-performance commercial cores, targeting performance levels suitable for desktop or server-class applications [33].

For this thesis, the Rocket Core was undoubtedly the correct choice. While BOOM offers superior standalone CPU performance, our project's primary goal was to design an SoC, integrate, and evaluate a specialized DNN accelerator (Gemmini). The vast majority of the performance uplift for our target workload was expected to come from the accelerator, not the host CPU. Therefore, the selection criteria for the CPU core prioritized stability, reliability, and ease of integration over raw CPU performance.

1.5 Research Objectives and Contributions

This thesis aims to demonstrate the capabilities of the Chipyard framework through the implementation of a practical RISC-V SoC that integrates a Rocket processor core with the Gemmini DNN accelerator, while establishing a comprehensive foundation for CESLAB's transition toward chip prototyping and VLSI-based development. The specific objectives of this work include:

1. **Framework Evaluation and Migration**: Comprehensively evaluate the Chipyard framework as a replacement for Intel Quartus-based design flows, specifically assessing its VLSI capabilities, modern interconnect support (Tile Link/AXI4), and suitability for ASIC development workflows.

2. **System Architecture Design**: Develop a comprehensive understanding of the Chipyard framework and utilize it to configure a custom SoC featuring a 64-bit Rocket core coupled with a 16×16 Gemmini systolic array accelerator (Rocket64b1gem16jtag configuration), demonstrating the framework's flexibility and configurability.

3. **Full-Stack Implementation**: Implement the complete system stack, including hardware generation, software compilation, bootloader configuration, and Linux operating system integration, demonstrating the end-to-end capability of the open-source toolchain from high-level design to functional system.

4. **FPGA Prototyping and Validation**: Successfully deploy and validate the designed SoC on a Xilinx VC707 FPGA board, providing a realistic hardware platform for system evaluation and serving as a crucial stepping stone toward future ASIC implementation.

5. **Accelerator Integration and Testing**: Demonstrate the integration of the Gemmini accelerator as a Rocket Custom Coprocessor (RoCC), validate its functionality through comprehensive testing, and showcase how specialized computational units can be seamlessly incorporated into RISC-V systems using modern interconnect standards.

6. **Documentation and Knowledge Transfer**: Create comprehensive documentation of the design process, implementation challenges, and solutions to serve as a foundational reference for future CESLAB projects targeting chip prototyping and VLSI development.

1.6 Significance and Impact

This work represents the first comprehensive exploration of the RISC-V ecosystem within CESLAB and, more importantly, establishes a critical foundation for our laboratory's strategic transition toward chip prototyping and semiconductor fabrication. By successfully implementing a complete, functioning system that combines a RISC-V processor with a state-of-the-art DNN accelerator using industry-standard design flows, this thesis demonstrates the maturity and practicality of open-source hardware design methodologies for serious VLSI development.

The strategic significance of this work extends far beyond the immediate technical achievements. This thesis serves as a comprehensive feasibility study and implementation roadmap for CESLAB's evolution from FPGA-based prototyping to actual chip development. The successful demonstration of the Chipyard framework's VLSI capabilities, coupled with its support for modern interconnect standards and integration with industry-standard EDA tools, validates our laboratory's decision to adopt this platform as the foundation for future chip development projects.

Furthermore, this work establishes a new research paradigm within our laboratory that emphasizes open-source collaboration, reproducible research, and the development of freely available intellectual property. This approach not only reduces the barriers to advanced hardware research but also positions CESLAB as a contributor

to the global open-source hardware movement, potentially leading to collaborative opportunities with industry and academic partners pursuing similar chip development goals.

The comprehensive documentation generated by this thesis—covering everything from framework setup and configuration to FPGA deployment and system validation—serves as an invaluable knowledge base for future CESLAB researchers and students. This documentation addresses the practical challenges of implementing complex SoC designs using open-source tools, providing detailed solutions to common problems and establishing best practices for our laboratory's future chip development projects.

Most critically, the full-stack nature of this implementation provides valuable insights into the complex interactions between hardware accelerators, operating systems, and application software—interactions that are often overlooked in studies that focus solely on isolated hardware components. These insights are crucial for developing practical, deployable AI systems that can deliver their theoretical performance benefits in real-world environments, and they will inform our laboratory's approach to designing optimized ASIC implementations in future projects.

1.7 Thesis Structure and Organization

This thesis is organized into nine chapters that systematically present the foundational argument, technical background, design, implementation, and evaluation of the RISC-V SoC with a Gemmini DNN accelerator.

Chapter 2: RISC-V Microprocessor Instruction Set Architecture provides the necessary technical background for this thesis. It begins with a comprehensive overview of the RISC-V ISA and its key architectural features. It then presents a detailed description of the Chipyard SoC design framework, the central development platform used in this work.

Chapter 3: A Single-Core with Gemmini RoCC Accelerator RISC-V Processor System introduces the specific architecture of the system designed and implemented in this project. It details the high-level system structure, the role of Chisel/Scala in hardware design, and the overall methodology for constructing our custom SoC within the Chipyard framework.

Chapter 4: Architectural Foundations of DNN Acceleration explores the fundamental challenges of Deep Neural Network acceleration, including the memory

wall problem and the principles of systolic array architectures. This chapter provides the theoretical context for understanding why specialized accelerators like Gemmini are necessary for efficient DNN computation.

Chapter 5: The Gemmini DNN Accelerator: An Architectural Overview presents a detailed analysis of the Gemmini accelerator's high-level architecture, including its systolic compute engine, memory hierarchy, and integration model within the SoC framework.

Chapter 6: Gemmini's Internal Architecture and Programming Model performs a deeper dive into the microarchitectural details of Gemmini. It examines the internal command and control flow, the instruction processing pipeline that enables efficient DNN computation.

Chapter 7: System Implementation and Testing documents the complete implementation process, from SoC configuration, integration of all system components, and hardware generation to FPGA deployment and system validation. This chapter provides practical insights into the entire development workflow and demonstrates the full-stack, open-source design process.

Chapter 8: Implementation Results and Discussion presents the results of the project. It includes an analysis of hardware resource utilization, validates the successful boot of a full Linux operating system, and provides a quantitative performance evaluation of the Gemmini accelerator compared to a software-only implementation.

Chapter 9: Conclusion and Future Work summarizes the key achievements of this thesis, evaluates the success of the implementation against the stated objectives, acknowledges the limitations of this work, and outlines potential directions for future research and development.

This structured approach ensures a comprehensive understanding of both the theoretical principles underlying modern SoC design and the practical challenges involved in implementing complex, multi-component systems using open-source tools and methodologies.

Chapter 2: RISC-V Microprocessor Instruction Set Architecture

2.1 Overview of RISC-V

Designing and developing a microprocessor, or Central Processing Unit (CPU), is a complex process requiring the collaboration of many experts in fields such as digital logic design, compiler development, and operating systems, along with significant financial resources. Major vendors like ARM, with prominent processor lines such as Cortex-M and Cortex-A, often impose high licensing fees and strict confidentiality requirements on their designs. This creates considerable barriers to hardware and software research and customization, particularly for academic institutions or small businesses. In response to these limitations, the RISC-V architecture [30, 29] was developed and introduced as an open-source solution, providing a flexible platform that eliminates licensing costs and allows for need-based customization. By fostering innovation and opening up a freer research ecosystem, RISC-V is reshaping the traditional approach to microprocessor development, aiming for more efficient and comprehensive solutions.

Figure 2.1: RISC-V is managed by RISC-V International. Source [22]

Essentially, RISC-V is an open-source microprocessor instruction set architecture (ISA), designed and developed based on the Reduced Instruction Set Computer (RISC) methodology, and it represents the fifth generation of this design approach. For a program executing on a microprocessor, the RISC design methodology stipulates that the microprocessor executes only simple assembly (machine code) instructions, which have a fixed length and are executed within a single, uniform clock cycle. A complex task is performed by multiple assembly instructions, making the instruction set simple, compact, and easy to develop hardware for rapidly. This is entirely different from the Complex Instruction Set Computer (CISC) architecture, which aims to complete a task with the fewest possible assembly instructions. To achieve this, CISC microprocessors must be designed to execute structurally complex assembly instructions, capable of performing multiple specialized functions sequentially and executing over multiple clock cycles.

Originating as a project developed by researchers and graduate students at the

University of California, Berkeley in 2010, the initial purpose of RISC-V was to create a practical ISA to support research and teaching in computer architecture, deployable on both hardware and software without licensing requirements. The original authors of RISC-V were individuals with considerable experience in computer design. Consequently, for stable development, RISC-V aimed for several goals, most of which have been achieved, including:

- A complete, fully open-source ISA, free for academic, educational, and industrial use, with practical applicability suitable not only for experimental simulation but also for hardware implementation.
- An ISA not oriented towards complex microarchitectures or fixed by any specific implementation technology (FPGA, ASIC, full-custom, etc.) but still allowing for efficient implementation in any of these directions.
- An ISA divided into base ISAs, usable for developing hardware accelerators or for educational purposes, and optional standard extensions to support general software development.
- An ISA supporting extensive instruction set extensions with specialized functionalities and numerous variants.
- Support for the IEEE 754-2008 floating-point standard.
- Support for parallel, multi-core, and heterogeneous many-core implementations.
- Both 32-bit and 64-bit address space variants developed for applications, operating system kernels, and hardware implementations.

Due to its open-source nature, RISC-V allows developers to freely use the design architecture without paying intellectual property and licensing fees. This creates development opportunities for RISC-V itself as it is adopted by the research community and educational institutions, gradually becoming popular in industry, and attracting significant attention from many large companies, corporations, and even governments of various countries. Furthermore, designers using the RISC-V ISA can utilize it without limitations in both hardware and software. This means developers are encouraged, but not required, to publicly disclose their designs and projects developed using RISC-V.

Currently, the development process and rights regarding the publication, release, and maintenance of intellectual property and official documentation related to RISC-V are managed by RISC-V International, based in Switzerland. This meets the standardization requirements for designers and users for commercial purposes, who demand a system using the RISC-V ISA that can operate and exist for many years.

To suit various implementers and ensure flexibility in designs with specialized characteristics, the RISC-V instruction set is divided and standardized into two main parts: the base instruction set and standard extensions. The RISC-V instruction set supports 32-bit, 64-bit, and 128-bit architectures with the respective base instruction sets RV32I, RV32E, RV64I, and RV128I. Among these, RV32I is the most important and common set for 32-bit microprocessors, featuring 40 integer manipulation instructions. The fact that RISC-V only standardizes and provides the instruction set allows hardware designers the freedom to choose how to implement their designs.

2.1.1 RISC-V 32-bit and the RV32I Base Instruction Set

RV32I [30] is the base instruction set present in almost all 32-bit RISC-V microprocessor architectures. The assembly instructions in this set use 32 32-bit registers, named `x0` - `x31`, to store integers. Each register, in addition to being numbered from 0 to 31, is also referred to by its Application Binary Interface (ABI) name, which reflects its specific function. For example, register `x0` is a read-only register containing the constant value "0", and register `x1` is used as the return address register, storing the return address when a function is executed. Table 2.1 lists these registers and their functions in detail.

The assembly instructions in RV32I use six different 32-bit instruction formats, depending on the function of the instruction. These include R, I, S, U formats, as well as two additional formats for immediate values: B and J. These formats are clearly shown in Figure 2.2. Notably, the source registers (`rs1` and `rs2`) and the destination register (`rd`) are kept in the same position in most formats, simplifying instruction decoding. The only discernible difference lies in how `rs2`, `rd`, and immediate values are organized within the instruction.

Figure 2.3 details all instructions in the RV32I base instruction set with their respective formats, specific opcode values, and function codes (`funct3` and `funct7`). Functionally, the RV32I set is divided into five distinct groups:

- 21 integer computation instructions: Perform arithmetic (addition, subtraction, bit shifts), logic (AND, OR, XOR, NOT, etc.), and comparison operations

Table 2.1: RISC-V Standard 32-bit ABI Register Conventions (RV32I). [30]

Register	ABI Name	Description
x0	zero	Constant "0"
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5 → x7	t0 → t2	Temporary registers
x8	s0/fp	Memory/frame pointer
x9	s1	Saved register
x10, x11	a0, a1	Function parameter/return value
x12 → x17	a2 → a7	Function parameter
x18 → x27	s2 → s11	Saved register
x28 → x31	t3 → t6	Temporary registers

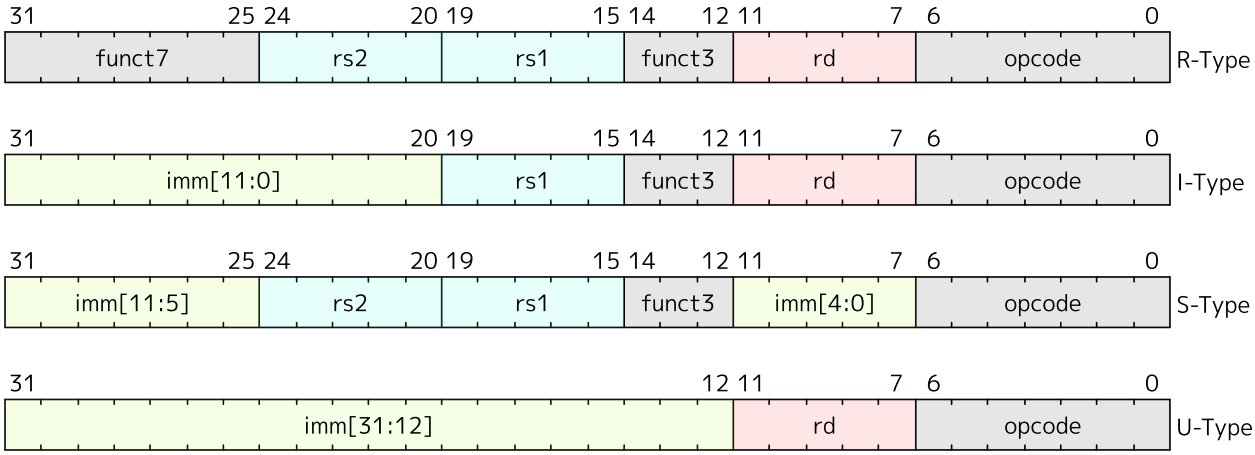


Figure 2.2: Instruction encoding formats in RV32I. Source [30]

on signed and unsigned numbers. Typically use R-type and I-type formats.

- 8 load-store instructions: LOAD and STORE byte, half-word, word (signed and unsigned), typically using I-type and S-type formats.
- 8 control-flow/branch instructions: Conditional branches, performing comparisons between two registers to decide whether to branch (BEQ/BNE, BLT, BGE), typically using I-type format.
- 2 system access instructions for the execution environment - ECALL and breakpoint - EBREAK, using I-type format.
- 1 memory access ordering instruction - FENCE, used by other processing cores

(in multi-core scenarios) or other peripheral devices in the system. The I-type format is used for this instruction with some specific bit encoding rules and opcodes.

With these 40 instructions, the microprocessor can perform most basic operations on integers. Additionally, several standard pseudo-instructions are defined to add tasks for which there are no dedicated instructions in the set. For example, the MOVE instruction - "`mv rd, rs`" - performs the function of moving/copying data between two registers. In a program, this instruction might be generated in the assembly code by a RISC-V compiler but is actually converted by assemblers into a standard instruction when translating to machine code, namely ADDI - Add Immediate "`addi rd, rs, 0`", which adds the source register to the constant "0" and assigns it to the destination register, achieving a similar function. Some other pseudo-instructions and their equivalent standard instructions are shown in Table 2.2.

Table 2.2: Common RISC-V Pseudo-instructions. [30]

Pseudo-instruction	**Corresponding Base Instruction**	**Meaning**
`nop`	`addi x0, x0, 0`	No operation
`li rd, imm`	`addi rd, x0, imm`	Load immediate
`mv rd, rs`	`addi rd, rs, 0`	Move register (copy)
`not rd, rs`	`xori rd, rs, -1`	One's complement (bitwise NOT)
`neg rd, rs`	`sub rd, x0, rs`	Two's complement (negate)
`seqz rd, rs`	`sltiu rd, rs, 1`	Set if equal to zero
`snez rd, rs`	`sltu rd, x0, rs`	Set if not equal to zero
`sltz rd, rs`	`slt rd, rs, x0`	Set if less than zero
`sgtz rd, rs`	`slt rd, x0, rs`	Set if greater than zero
`beqz rs, offset`	`beq rs, x0, offset`	Branch if equal to zero
`bnez rs, offset`	`bne rs, x0, offset`	Branch if not equal to zero
`blez rs, offset`	`bge x0, rs, offset`	Branch if less than or equal to zero
`bgez rs, offset`	`bge rs, x0, offset`	Branch if greater than or equal to zero
`bltz rs, offset`	`blt rs, x0, offset`	Branch if less than zero
`bgtz rs, offset`	`blt x0, rs, offset`	Branch if greater than zero
`j offset`	`jal x0, offset`	Jump to label (offset)
`jal offset`	`jal x1, offset`	Jump and link (call)
`ret`	`jalr x0, 0(x1)`	Return from subroutine

As mentioned, a goal of RISC-V is to support implementations requiring more

RV32I Base Instruction Set						
imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20|10:1|11|19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND
fm	pred succ	rs1	000	rd	0001111	FENCE
1000	0011 0011	00000	000	00000	0001111	FENCE.TSO
0000	0001 0000	00000	000	00000	0001111	PAUSE
000000000000		00000	000	00000	1110011	ECALL
000000000001		00000	000	00000	1110011	EBREAK

Figure 2.3: Instructions in the RV32I base instruction set. [30]

features beyond the base instruction set. This is reflected in RISC-V defining extensions [30] for its instruction set architecture. These extensions define additional instruction encodings for various functionalities. For example, the standard M extension adds instructions for integer multiplication and division, while the standard F extension defines instructions for single-precision floating-point computation and manipulation.

Table 2.3: Development status of extended RISC-V instruction sets. [30]

Instruction Set	**Function Description**	**Version**	**Status**
M	Integer multiplication and division instruction set	2.0	Approved
A	Atomic memory operation instruction set	2.1	Approved
F	Single-precision floating-point computation	2.2	Approved
D	Double-precision floating-point computation	2.2	Approved
Q	Quad-precision floating-point computation	2.2	Approved
C	Compressed instruction set	2.0	Approved
Zifence	Instruction-Fetch Fence instruction set	2.0	Approved
Zicsr	Control and Status Registers (CSRs) instruction set	2.0	Approved
Ztso	Total Store Ordering instruction set	1.0	Approved

Table 2.3 above clearly lists the standard extension sets developed and published by RISC-V International. It can be seen that many instruction sets have been ratified and can be used in designs. When implemented, architectures supporting the RV32I base instruction set (or RV64I depending on the architecture) combined with the standard extensions M, A, F, D, Zicsr, and Zifencei are often summarized and referred to as the RV32G (or RV64G) instruction set, named for its general-purpose utility.

2.1.2 RISC-V 64-bit and the RV64I Base Instruction Set

Similar to RV32I, RV64I is the base instruction set for 64-bit microprocessor architectures, featuring 64-bit wide registers and supporting 64-bit addressing (XLEN=64), though instructions remain 32-bits wide. Architectures compliant with RV64I are compatible with RV32 instructions and include additional instructions to support operations between 32-bit and 64-bit data. For example, the `LW` instruction loads a 32-bit value and sign-extends it into a 64-bit register, while the new `LD` instruction loads a 64-bit word. The `ADD` instruction now performs 64-bit addition, while the `ADDW` instruction handles 32-bit addition, ignoring the upper 32 bits and returning a 32-bit signed value, sign-extended to 64 bits. These instructions enabling backward compatibility typically have a "W" suffix to denote 32-bit wide operations. These

instructions, along with new 64-bit load and store operations, are summarized in Figure 2.4.

RV64I Base Instruction Set (in addition to RV32I)						
imm[11:0]		rs1	110	rd	0000011	LWU
imm[11:0]		rs1	011	rd	0000011	LD
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011	SD
000000	shamt	rs1	001	rd	0010011	SLLI
000000	shamt	rs1	101	rd	0010011	SRLI
010000	shamt	rs1	101	rd	0010011	SRAI
imm[11:0]		rs1	000	rd	0011011	ADDIW
0000000	shamt	rs1	001	rd	0011011	SLLIW
0000000	shamt	rs1	101	rd	0011011	SRLIW
0100000	shamt	rs1	101	rd	0011011	SRAIW
0000000	rs2	rs1	000	rd	0111011	ADDW
0100000	rs2	rs1	000	rd	0111011	SUBW
0000000	rs2	rs1	001	rd	0111011	SLLW
0000000	rs2	rs1	101	rd	0111011	SRLW
0100000	rs2	rs1	101	rd	0111011	SRAW

Figure 2.4: Instructions in the RV64I base instruction set. [30]

The compiler and calling conventions maintain the invariant that all 32-bit values are stored in sign-extended format within 64-bit registers. Even 32-bit unsigned integers will have bit 31 extended into bits 63 through 32. Consequently, converting between 32-bit signed and unsigned integers is a no-operation, as is converting from a 32-bit signed integer to a 64-bit signed integer. Existing 64-bit instructions like `SLTU` (set less than unsigned) and branch comparisons continue to operate correctly on 32-bit unsigned integers under this rule. Similarly, 64-bit logical operations on 32-bit sign-extended integers preserve the sign-extension property. A few new instructions (`ADD[I]W/SUBW/SxxW`) are necessary for addition and shifts to ensure reasonable performance for 32-bit values.

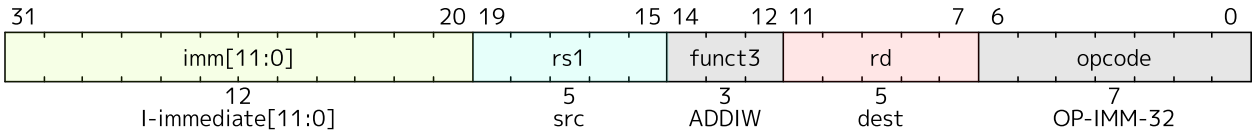


Figure 2.5: Instruction encoding format for ADDIW in 64-bit architecture. [30]

Some distinctly different instructions found only in RV64I include the `ADDIW` instruction, as shown in Figure 2.5. `ADDIW` performs an addition between a 12-bit sign-

extended immediate value and register `rs1`, producing a proper 32-bit sign-extended result in `rd`. Overflows are ignored, and the result consists of the lower 32 bits of the sum, sign-extended to 64 bits. Note, the instruction "`addiw rd, rs1, 0`" writes the sign-extension of the lower 32 bits of register `rs1` into register `rd` (pseudo-instruction `SEXT.W`).

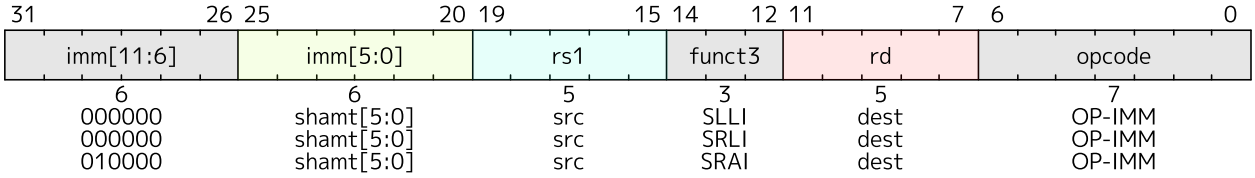


Figure 2.6: Encoding format for 64-bit I-type shift instructions. [30]

I-type shift operations in RV64I (Figure 2.6) use the same opcode as RV32I with some modifications. Register `rs1` contains the operand, and the shift amount is encoded in the lower 6 bits of the I-immediate field. Bit 30 determines the shift type: `SLLI` (logical left shift, zeros shifted into lower bits), `SRLI` (logical right shift, zeros shifted into upper bits), and `SRAI` (arithmetic right shift, sign bit copied into vacated upper bits).

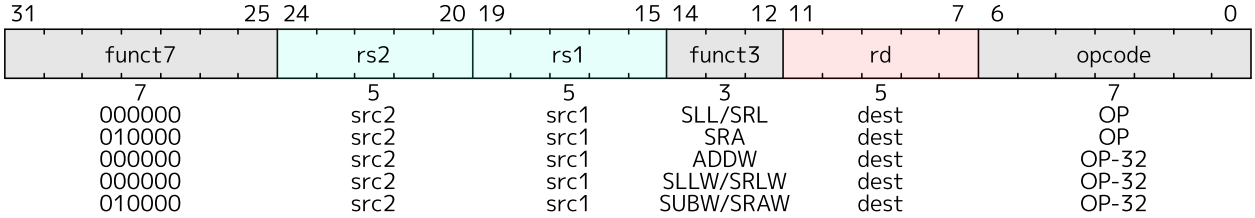


Figure 2.7: Encoding format for 64-bit R-type shift instructions for word operations. [30]

The `ADDW` and `SUBW` instructions in RV64I, similar to `ADD` and `SUB` for 64-bit operations, operate on 32-bit data and produce a 32-bit signed result, which is then sign-extended to 64-bits before being written to the destination register. Overflows are also ignored. The `SLL`, `SRL`, and `SRA` instructions perform logical left, logical right, and arithmetic right shifts on the value in `rs1`, with the shift amount determined by the lower 6 bits of `rs2`. The `SLLW`, `SRLW`, and `SRAW` instructions (Figure 2.7), unique to RV64I, operate on 32-bit values and use the lower 5 bits of `rs2` to determine the shift amount.

Finally, load-store instructions can be mentioned. RV64I extends the address space to 64 bits, but the number of accessible addresses may vary depending on the execution environment.

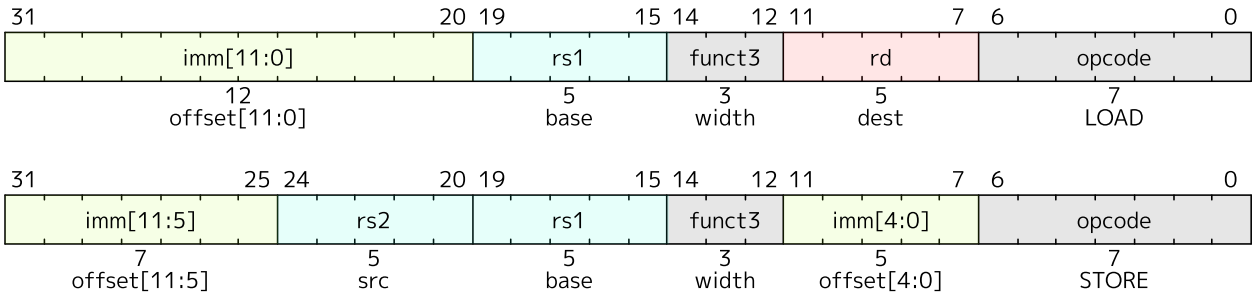


Figure 2.8: Encoding formats for 64-bit LOAD and STORE instructions. [30]

The instruction formats are shown in Figure 2.8. The `LD` instruction here will load a 64-bit value, `LW` loads a 32-bit value (sign-extended to 64-bits) from memory into a register, and `LWU` zero-extends a 32-bit value from memory into register `rd`. Similarly, `LH/LHU` and `LB/LBU` handle 16-bit and 8-bit values. The `SD`, `SW`, `SH`, and `SB` instructions store 64-bit, 32-bit, 16-bit, and 8-bit values from register `rs2` to memory, respectively.

2.2 RISC-V Processor Cores

With its open-source nature, system developers can freely implement designs based on the RISC-V architecture, supported by an increasingly robust and growing ecosystem. The RISC-V ISA ecosystem mentioned here includes a diverse range of components from hardware design tools, low-level firmware development, bootloaders, to fully functional operating system kernels. Currently, development tools such as GCC/LLVM compilers, QEMU and Verilator simulators, and even the Linux operating system have incorporated RISC-V architecture and are under active development. As of the time of writing this thesis, there are over 100 open-source hardware processor cores and more than 40 open-source SoC platforms using the RISC-V ISA that have been and continue to be developed.

This section will provide a general overview of pipeline, in-order, and out-of-order architectures commonly implemented in microprocessors. Subsequently, a brief survey of open-source RISC-V processor cores will be conducted to select the most suitable design for implementation.

2.2.1 Pipeline Architecture in Microprocessors

A modern general-purpose CPU is often designed using a pipeline architecture. This technique draws inspiration from industrial assembly lines, dividing the process-

ing of an assembly instruction into multiple stages or segments, each performed by different specialized hardware units connected sequentially throughout the process and capable of continuous operation. This architecture is commonly found in many microprocessor designs using RISC in general and RISC-V ISA in particular, due to its higher performance compared to processing instructions sequentially and waiting for results.

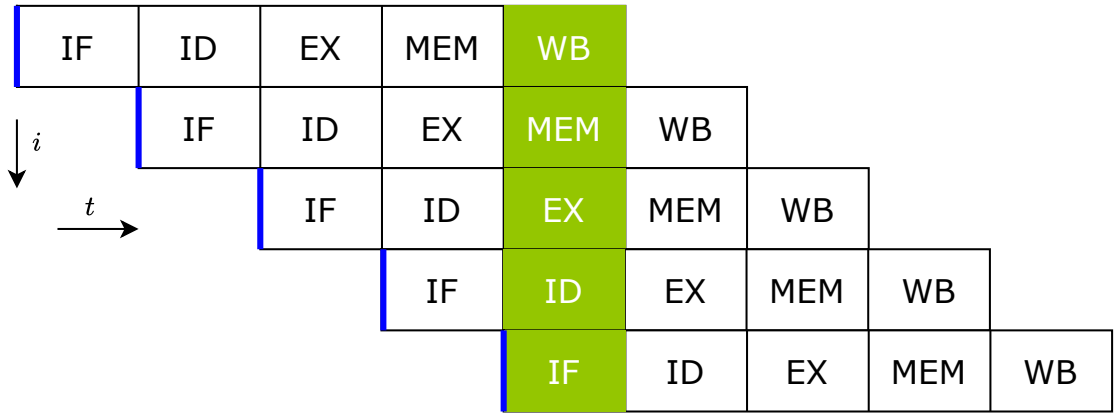


Figure 2.9: Basic five-stage pipeline in a RISC machine. The vertical axis is successive instructions; the horizontal axis is time. So in the green column, the earliest instruction is in WB stage, and the latest instruction is undergoing instruction fetch.

The concept of pipelining is illustrated in Figure 2.9. While one assembly instruction is being processed in later stages, another assembly instruction begins processing in earlier stages, in parallel with previous instructions. It can be seen that at the fourth clock cycle, the pipeline is processing four different instructions in parallel, each in a distinct stage. A basic microprocessor pipeline consists of five stages. However, depending on the implementation and design, a microprocessor may use more or fewer stages, but still follows the sequence of stages explained below:

- **Instruction Fetch (IF):** Retrieves instructions from the instruction memory, also known as program memory. Each instruction code that the CPU must execute is stored in the program memory. These instruction codes are binary sequences compiled from C/C++ code or assembly code.
- **Instruction Decode (ID):** The binary of each instruction is decoded, and corresponding control signals are activated.
- **Execute (EX):** Performs calculations and processing corresponding to the decoded instruction, such as addition, subtraction, multiplication, division, shifts, assignments, branches, etc.

- **Memory Access (MEM):** Accesses memory and performs read/write operations of data from/to memory.
- **Writeback (WB):** Saves the results after instruction execution to the location specified by the instruction, such as registers.

2.2.2 Out-of-Order Architecture in Microprocessors

Alternatively, out-of-order execution architecture is a crucial paradigm in modern CPU design, delivering significant performance gains by allowing instructions to be executed as soon as their input operands are ready, rather than strictly adhering to the program order as in a pipeline. This architecture is also a core element in high-performance systems used in scientific computing and artificial intelligence research. This flexible execution model addresses performance bottlenecks caused by data dependencies, cache latencies, and pipeline stalls, ensuring more efficient utilization of computational resources.

Key components of this architecture include the instruction window, which stores multiple instructions for dependency analysis, and dynamic scheduling mechanisms such as Tomasulo's algorithm or scoreboarding, which track dependencies and resource availability. Another critical feature is the Re-order Buffer (ROB), which helps maintain program order during instruction retirement, ensuring correct architectural state updates and precise exception handling. By decoupling instruction fetch and execution stages, out-of-order processors achieve higher instruction throughput and sustain the performance of execution units, optimizing efficiency across various applications.

Despite its many strengths and implementation benefits, out-of-order processors also present significant design challenges. The hardware blocks required for components like the ROB and branch prediction increase complexity, chip area, and power consumption, making energy efficiency optimization an important research area. Furthermore, extending out-of-order architecture to meet the demands of heterogeneous processing and large-scale parallel computing requires innovative approaches in dynamic scheduling and resource management.

2.2.3 Open-Source RISC-V Processor Cores

Rocket [2], or Rocket Core, is a RISC-V instruction set architecture microprocessor core designed by the Berkeley Architecture Research group. Rocket features

a 5-stage, in-order architecture and implements the RISC-V RV32G and RV64G instruction sets.

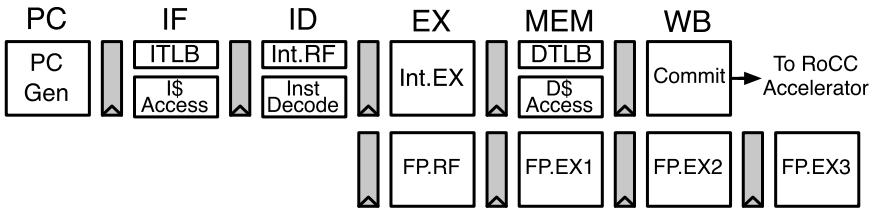


Figure 2.10: Pipeline of Rocket core. Source [2]

The design of this microprocessor integrates many advanced features, including a Memory Management Unit (MMU) that supports virtual memory, enabling efficient memory access and address translation. Besides its role as a general-purpose microprocessor, Rocket also serves as a library of reusable components, including functional units, caches, Translation Lookaside Buffer (TLB), Page Table Walker (PTW), and Control and Status Registers (CSRs), which are reused in many other microprocessor designs. This makes Rocket Core a foundational component in Chipyard - RocketChip, allowing for the rapid development and creation of experimental systems ranging from simple to complex.

Rocket is considered the most prominent microprocessor core in the open-source RISC-V development community due to its well-updated and stably supported development documentation. It is built using Chisel, a relatively new hardware construction language based on Scala, employing an object-oriented approach similar to software development. Therefore, the flexibility of the system development process and the applicability of Rocket Core are very high, making it suitable for implementing multi-core systems.

Along with Rocket Core, the Berkeley Out-of-Order Machine (BOOM) [5, 4, 6] is also a prominent open-source RISC-V core, designed for high performance and out-of-order execution. Also developed within the microprocessor architecture research group at the University of California, Berkeley, BOOM is aimed at applications in advanced academic research and industry, featuring a flexible design and high operational performance. BOOM is also written in the Chisel language as a generator, like Rocket Core, allowing for rapid deployment and customization of the core according to the RV64GC RISC-V architecture. The latest version, SonicBOOM [33] (also known as BOOMv3), achieves a performance of 6.2 CoreMarks/MHz, capable of competing directly with high-performance commercial processor cores, setting a

high standard for open-source processor design.

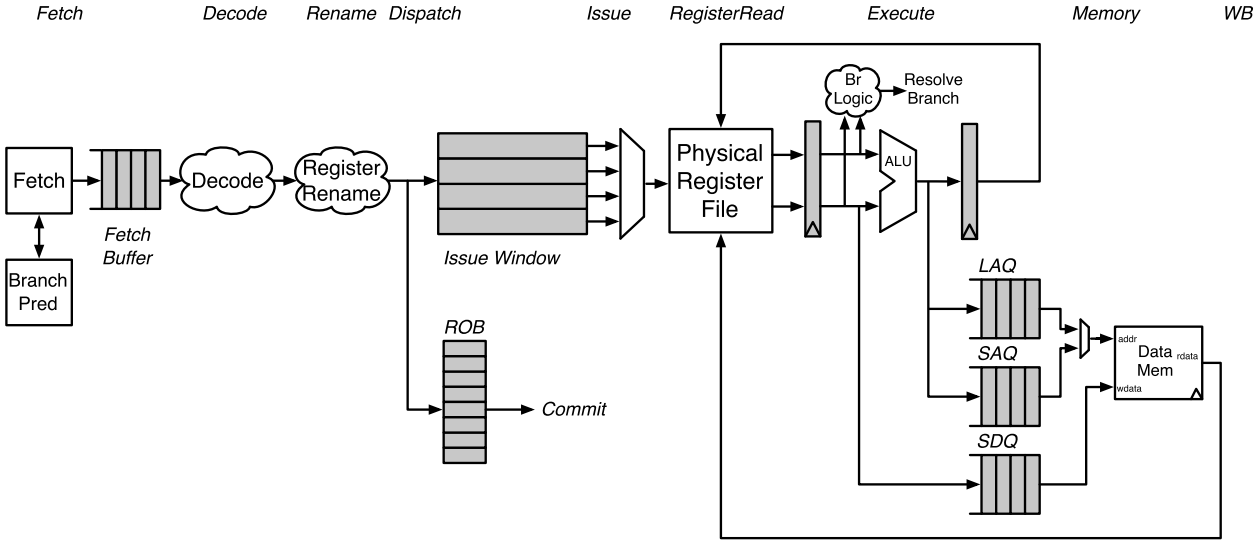


Figure 2.11: Berkeley Out-of-Order Machine Processor Pipeline Architecture. [6]

Structurally, BOOM features a 10-stage pipeline architecture including Fetch, Decode, Register Rename, Dispatch, Issue, Register Read, Execute, Memory, Writeback, and Commit. These stages allow for detailed control and optimization of the instruction flow, maintaining the "out-of-order" functionality at the high speeds necessary for modern processors. Specifically, each stage performs as follows:

- **Fetch:** Instructions are fetched from instruction memory and stored in a fetch buffer to prepare for processing.
- **Decode:** Instructions are converted into simpler micro-ops to optimize execution within the pipeline.
- **Register Rename:** Logical registers in the ISA are renamed to independent physical registers, reducing data conflicts and increasing processing efficiency.
- **Dispatch:** Micro-ops from the decode stage are sent to the Issue window, awaiting necessary operands.
- **Issue:** Instructions in the Issue window are checked for operand readiness, and when conditions are met, micro-ops are sent to an execution unit.
- **Register File (RF) Read:** Micro-ops read operands from the physical register file or bypass network, preparing data for the next stage.

- **Execute:** Micro-ops are processed by functional units, performing computational tasks and memory address calculations.
- **Memory:** Manages memory operations via Load Address Queue (LAQ), Store Address Queue (SAQ), and Store Data Queue (SDQ), performing loads when addresses are ready and stores when both address and data are available.
- **Writeback:** Results from operations are written back to the physical register file, ensuring updated values are stored for future instructions.
- **Commit:** The Reorder Buffer tracks the status of instructions in the pipeline and commits results when instructions complete, including storing data to memory.

Similar to Rocket Core, this is also considered one of the most prominent open-source microprocessor cores. However, BOOM's design is more geared towards ASIC implementation, and its numerous features along with the use of Chisel for implementation might pose difficulties in creating a multi-core design in Verilog and deploying it on FPGAs.

2.3 System-on-Chip Design Framework: Chipyard - Rocket Chip

The Rocket Chip project, also developed by the Berkeley Architecture Research (BAR) group at the University of California, Berkeley, marked a significant advancement in open-source processor design. This project provides a flexible, scalable, and configurable platform, enabling the creation of custom RISC-V microprocessors tailored to diverse performance and application requirements. Rocket Chip is essentially a library of hardware generators that can be configured and interconnected in various ways, producing diverse SoC designs. This library can generate an in-order core (Rocket) or an out-of-order core (BOOM), and these cores can be connected to co-processors via an interface named RoCC (Rocket Custom Coprocessor) [2].

Figure 2.12: Chipyard, developed by UC Berkeley. Source: [32]

Subsequently, to develop and expand the capabilities of Rocket Chip, Chipyard was created. Chipyard is an open-source framework that integrates system design, simulation, and implementation for SoC development, and it has become one of the most actively embraced and developed frameworks by the community. Chipyard, also originating from UC Berkeley, is the result of continuous development and integration of Chisel, Rocket Chip, and the Rocket Core.

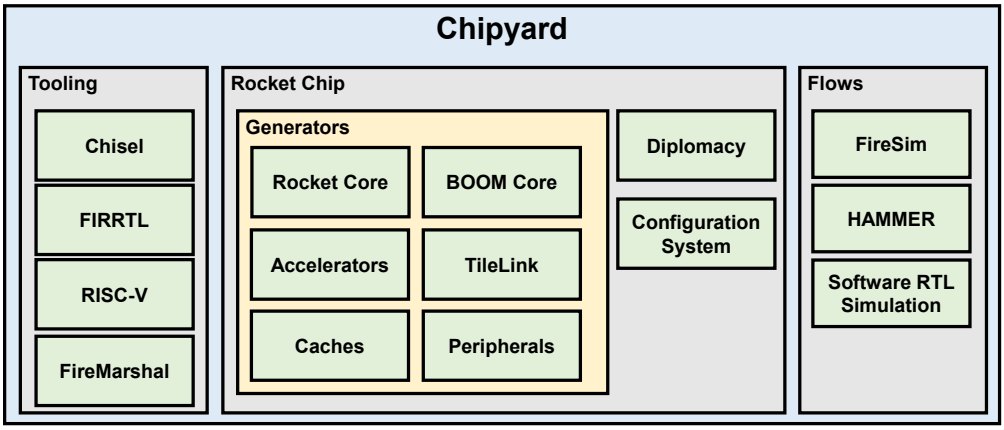


Figure 2.13: Component structure of Chipyard. Source: [32]

By utilizing Chisel, the FIRRTL compiler, the Rocket Chip generator, and other tools, Chipyard facilitates the development of highly customizable RISC-V SoCs. Chipyard includes a diverse set of processor cores (Rocket Core, BOOM, Ariane), accelerators (Hwacha, Gemmini, SHA3), along with comprehensive memory systems, peripheral devices, and tools to support the construction of a full-featured SoC. Chipyard supports multiple development flows, allowing for flexible hardware extension and experimentation. These development flows include software-based Register Transfer Level (RTL) simulation, FPGA-accelerated simulation using FireSim, automated VLSI flows with Hammer, and FireMarshal, which enables the creation and loading of software on both bare-metal systems (similar to microcontrollers) and Linux-based systems.

Rocket Chip supports deployment on FPGAs, with operational frequencies reaching 25-100 MHz depending on the configuration and the type of FPGA used. Currently, Rocket Chip is being utilized by SiFive [23], a company founded by some of the authors of RISC-V. SiFive produces RISC-V processors on silicon based on Rocket and provides tools for developing, testing, and evaluating their designs.

Using Chipyard (or Rocket Chip) as a foundation for system construction can pose several challenges due to its complexity in terms of the number of components and the

design language used—Chisel. However, overcoming this barrier yields numerous benefits due to Rocket Chip's high flexibility and system customization capabilities. Its components are developed independently, have detailed development documentation, and are easily extensible. Furthermore, the Rocket Chip support community is very active, with more frequent updates and improvements compared to other frameworks.

Having established the theoretical underpinnings of the RISC-V ISA and identified the Chipyard framework as our chosen development platform, the next step is to detail the architecture of our specific implementation. The following chapter will describe the design of the single-core RISC-V processor system, its integration with the Gemmini accelerator, and the high-level compilation and co-design flow that makes such a complex system possible.

Chapter 3: A Single-Core with Gemmini RoCC Accelerator RISC-V Processor System

From the surveys and analyses in the preceding sections regarding processor cores and the two SoC development frameworks, it is evident that each of these research subjects possesses distinct advantages and limitations. However, for the research orientation and implementation of a processor system featuring a custom accelerator, the Chipyard framework and its library components are deemed most suitable for the objectives of this thesis. Not only does Chipyard effectively meet current requirements, but it also holds the potential for expansion to support more complex systems in the future.

This chapter will present a more detailed exposition of the component structure and the process of constructing an SoC system, while also delving into the architecture of the Rocket processor core. The thesis will utilize high-level design languages Chisel/Scala to implement a single-core processor system integrated with the Gemmini RoCC accelerator, focusing on leveraging the components within an SoC system, and building the hardware and software design based on the Chipyard framework.

3.1 The Chipyard Compilation and Elaboration Flow

A key advantage of the Chipyard framework is its unified compilation and elaboration flow, which decouples high-level hardware design from low-level simulation and implementation targets [1]. The process begins with a `Custom SoC Configuration`, a set of Scala classes that define the parameters for the entire system. These parameters are consumed by the `RTL Generators`—hardware libraries written in Chisel—which programmatically construct the SoC design. Rather than directly emitting Verilog, the Chisel compiler generates a circuit description in the **FIRRTL Intermediate Representation**. This crucial IR acts as a pivot point for the entire framework, allowing a single design description to be consumed by various backend "transforms" tailored for specific goals, as illustrated in Figure 3.1.

For instance, the `FireSim Transforms` can apply modifications necessary for mapping the design onto cloud FPGAs for fast, cycle-exact simulation. The `VLSI Transforms` prepare the design for an ASIC implementation flow by replacing logical memories with technology-specific macros and preparing the hierarchy for place-and-route tools.

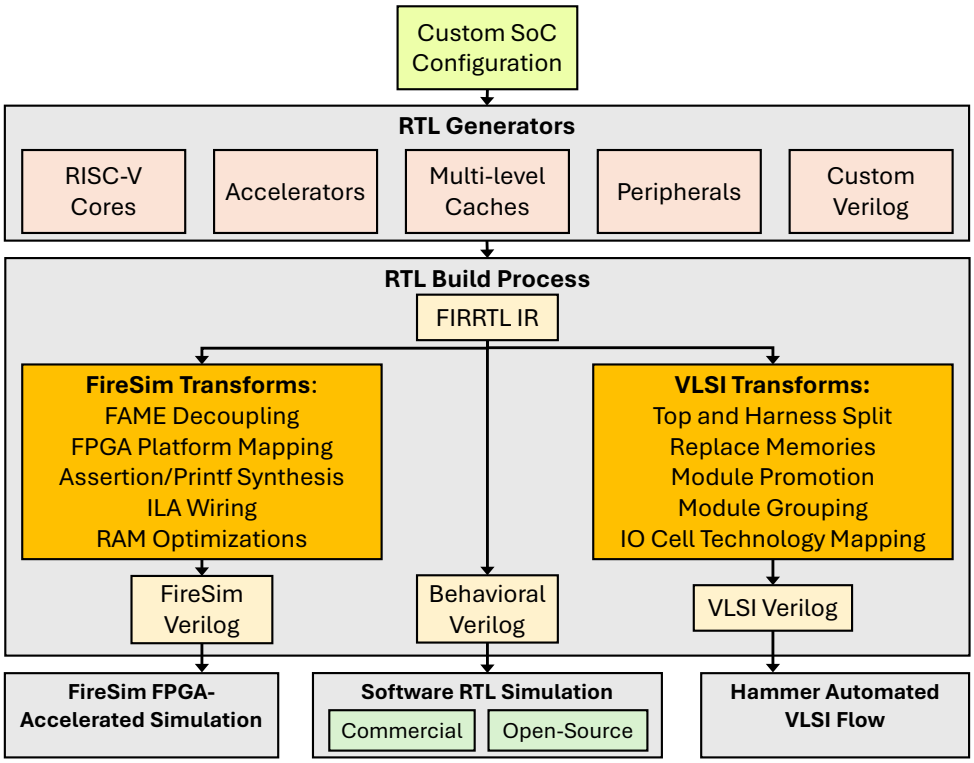


Figure 3.1: The overall development flow within Chipyard. Design begins with high-level configuration and Chisel generators, which produce FIRRTL. The FIRRTL IR acts as a pivot point, enabling different transformation paths for software simulation, FPGA-accelerated simulation, or VLSI layout. [1].

For the purposes of this thesis, this flow is primarily used to generate synthesizable Verilog for two key targets:

1. **Software RTL Simulation:** The FIRRTL is compiled into C++ code for cycle-accurate simulation using Verilator, enabling functional verification and initial performance analysis.

2. **FPGA Implementation:** The generated Verilog is consumed by the Xilinx Vivado toolchain to be synthesized, placed, and routed for deployment on the target VC707 FPGA board.

This structured flow demonstrates the power of a generator-based approach, allowing a single, high-level design description to target multiple platforms while maintaining design consistency.

3.2 System Design Methodology with Chisel

As previously introduced, this thesis undertakes system design based on the components and tools within Chipyard. This framework and its library components are constructed from a diverse range of elements, all based on Chisel/Scala.

3.2.1 Chisel and Scala

Figure 3.2: Chisel as a library within Scala.

Chisel is a Hardware Construction Language (HCL) developed at UC Berkeley, initially intended to enable hardware design through highly parameterized and hierarchically specific code generators. Chisel is used to define these generators, which then produce designs in Verilog, suitable for industry-standard RTL design flows. Chisel is classified as a Domain-Specific Language (DSL); more precisely, it is implemented as a library package for Scala, a high-level, multi-paradigm programming language that integrates features of both Object-Oriented Programming (OOP) and functional programming on the Java Virtual Machine (JVM). Furthermore, Scala's functional programming capabilities support higher-order functions and immutable data structures, facilitating more maintainable code, which is crucial for reproducibility and collaboration in scientific research.

Chisel's design aims to address the lack of object-oriented programming and preprocessing structures in existing Hardware Description Languages (HDLs), supporting the construction of more flexible generators. By being based on Scala, Chisel gains the flexibility to describe hardware with high levels of abstraction and modularity. This helps hardware designers describe circuits in a way that is easy to maintain, modify, and extend, especially for large and complex designs such as microprocessors and specialized digital signal processing cores. Scala's robust type system, diverse numerical representations and variables, immutability, and support for functional programming make it an ideal foundation for Chisel, aiding in error checking during hardware and circuit description and encouraging reusable design patterns.

In the context of hardware design based on Chisel and Scala, `sbt` (Simple Build Tool) plays a crucial role, optimizing the design management and development process. Built specifically for Scala and Java projects, `sbt` is the build tool used by

the majority of Scala developers. Essentially, `sbt` acts as the compiler when working with Chisel and Scala. Using `sbt` simplifies the process of structuring projects for hardware design with Chisel in Chipyard, as well as enabling integration with many other open-source projects.

The design elaboration process using Chisel is illustrated in Figure 3.3. A design written in Chisel is first executed by a Scala program, which invokes the Chisel compiler to generate a circuit representation in FIRRTL. This intermediate representation is then processed by a series of FIRRTL transformers and optimizers, which ultimately produce synthesizable Verilog or VHDL code. This final HDL can then be consumed by standard industry tools for synthesis and physical design, leading to a final chip layout. This flow abstracts away many tedious aspects of hardware design, allowing architects to focus on the high-level structure and parameterization of their designs.

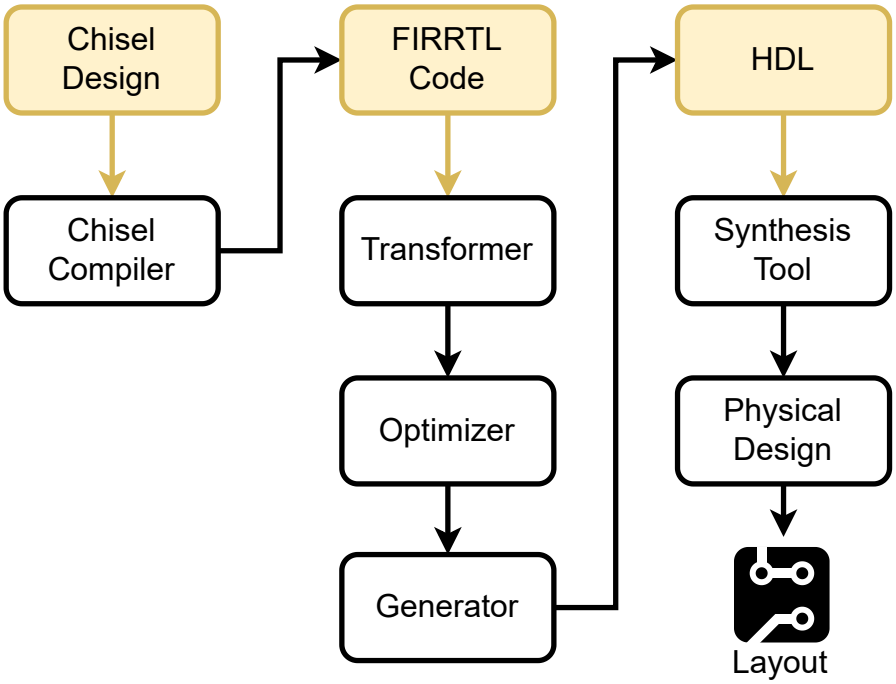


Figure 3.3: The Chisel to HDL design elaboration flow. A design is described in Chisel, compiled to FIRRTL, and finally transformed into a standard HDL for synthesis and physical design. [16]

3.2.2 Verilator Simulation Tool

In the hardware design process, evaluating the functionality of the design is critically important. With Chipyard, system designs, from simple to complex structures, can be effectively realized and tested thanks to integrated tools and RTL simulators like Verilator, which are readily available within the project framework.

Verilator is an open-source program, prominent in the field of RTL simulation due to its unique method of converting ("Verilating") `Verilog` or `SystemVerilog` into `C++` or `SystemC`, creating a highly optimized executable program. Unlike traditional simulators, Verilator does not directly interpret `Verilog` during simulation. Instead, it generates `C++` or `SystemC` program files by parsing the hardware code, performing syntax checking, linting, and optionally inserting debug points and coverage analysis to enhance the correctness and accuracy of evaluation and verification. This "Verilated" output file can be configured to run in single-threaded or multi-threaded mode on a development server, offering an advantage for simulating complex or computationally demanding designs. After the "Verilated" code is generated, it is compiled by a standard `C++` compiler such as `GCC`, `Clang`, or `MSVC++`, and is typically combined with a user-provided wrapper file to manage the initialization of the Verilated model.

Verilator offers high simulation performance, especially for designs requiring substantial resources, due to its high speed and cycle accuracy. Verilator's compilation approach makes it up to 100 times faster than interpretive simulators like Icarus Verilog (which is also open-source), and with multi-threading, performance can increase by an additional 2-10 times, achieving a total improvement of up to 1000 times. This makes Verilator suitable for RISC-V multi-core processor designs ranging from simple to complex, and competitive even with commercial simulators such as Synopsys VCS, Cadence Incisive, or Mentor ModelSim.

3.3 The Rocket Processor Core

As introduced in the previous chapter, Rocket is a 5-stage, in-order processor core compliant with the RISC-V instruction set architecture, with RV32G and RV64G variants. Currently, the development of the Rocket core is maintained by the CHIPS Alliance. Rocket is written in the Chisel hardware design language, based on the Scala programming language, and is a key component of the Rocket Chip SoC Generator toolchain.

When implemented, each Rocket core is combined with L1 data and instruction

caches. This memory operates locally for each core and, in a multi-core system, is responsible for storing temporary variables generated during the execution of the pipeline, helping to reduce congestion when multiple cores attempt to store data. Together with the Page Table Walker (`PTW`), a Tile within the SoC is formed. This Rocket Tile (Figure 3.4) is considered a fundamental unit in the Rocket Chip SoC system and can be replicated to create multi-core processor systems, meeting the demands for performance and flexibility in modern SoC design.

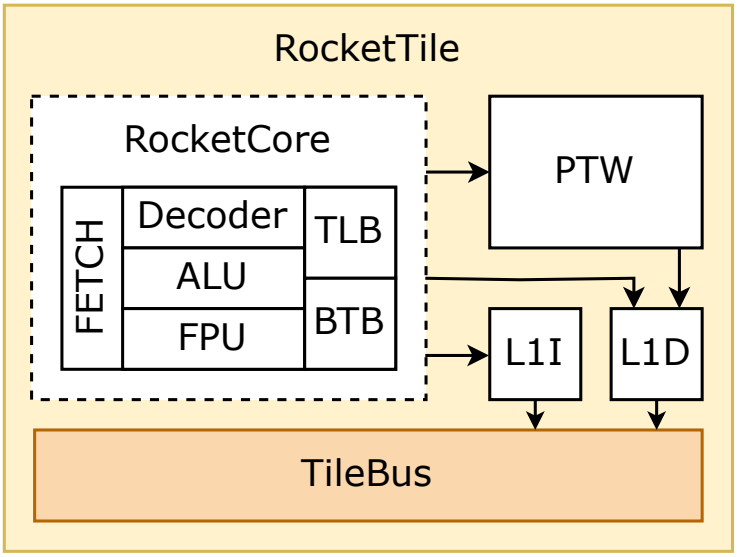


Figure 3.4: Structure of a Rocket Tile.

Due to the object-oriented nature of Chisel, the language used for its design, Rocket itself is written from object classes that can be defined and reconfigured according to implementation needs, such as `WithNBigCore()`, `WithNMedCore()`, `WithNSmallCore()`, and `WithNTinyCore()`. Additionally, instead of implementing the Rocket core, the BOOM core can also be integrated in its place, and many other peripherals can be integrated into a Tile unit.

3.4 Rocket Chip SoC Structure

This system generator includes multiple components, not just the processor core but also the on-chip system bus (SoC bus), specifically TileLink. TileLink comprises a peripheral bus, control bus, and memory bus, along with arbiters and system controllers. Additionally, the system integrates peripheral components such as DRAM, GPIO, UART, and, critically for this thesis, supports tightly-coupled accelerators like Gemmini via the RoCC interface. Figure 3.5 below depicts the generalized structure

of the single-core RISC-V processor system with a Gemmini RoCC accelerator, which forms the basis for the development in this thesis.

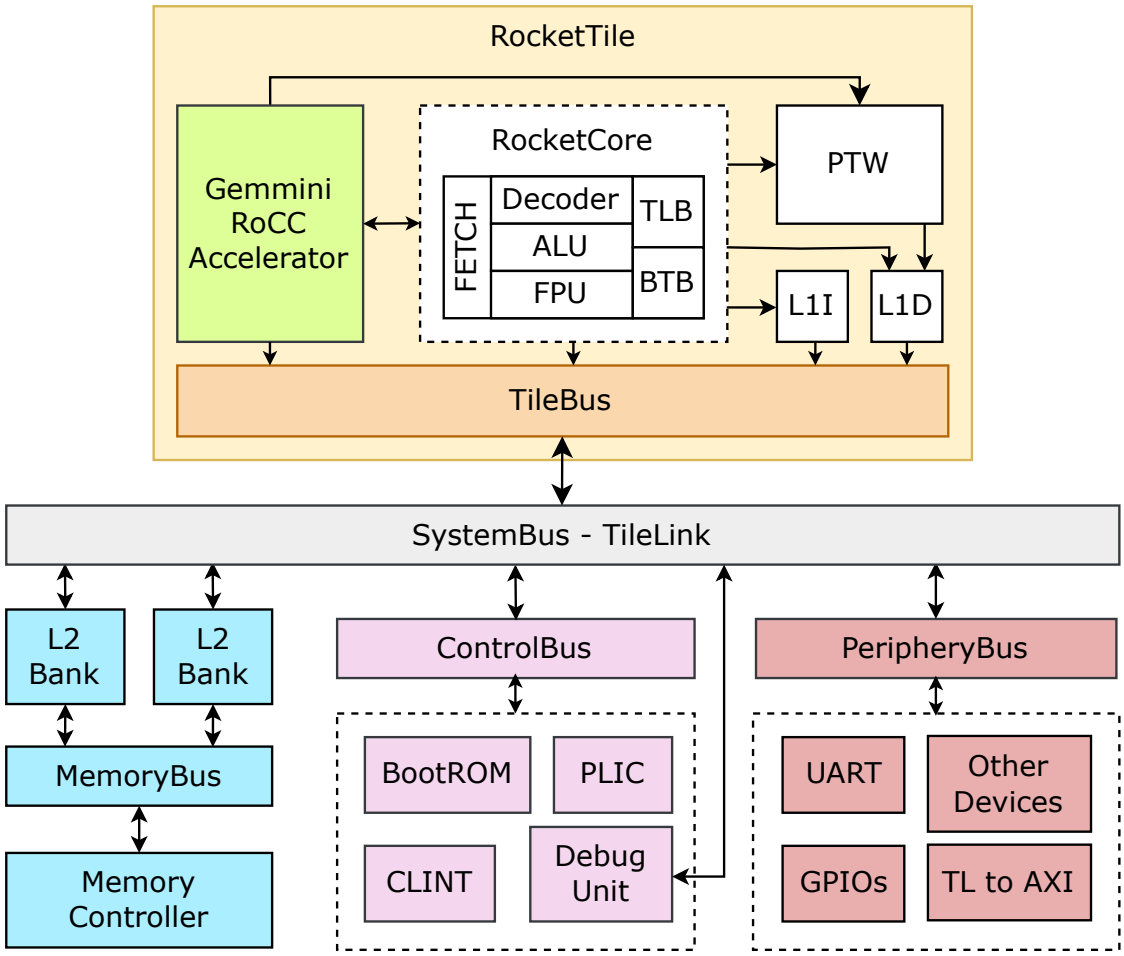


Figure 3.5: Proposed structure of a single-core RISC-V processor system with a Gemmini RoCC accelerator.

3.4.1 Bus and Interconnect

In the design of Systems-on-Chip (SoCs), bus architecture plays a backbone role, connecting and coordinating the operation of all components within the system. Particularly for modern multi-core processor systems, development trends focus on using crossbar-type bus architectures instead of Time Shared Buses. A crossbar, specifically a fully-connected crossbar structure, is considered a form of hardware switching fabric, allowing any input port to connect directly to any output port simultaneously. The outstanding advantage of this architecture lies in its ability to provide high bandwidth and low latency, effectively meeting performance requirements in applications such as networking equipment, multi-core processors, and high-performance storage

systems. The optimization in data transmission offered by this architecture is a key factor in improving the overall performance of modern SoC systems.

For Rocket Chip, the TileLink [24] system communication bus was developed concurrently with the project and the RISC-V instruction set, and is widely used in open-source system research at present.

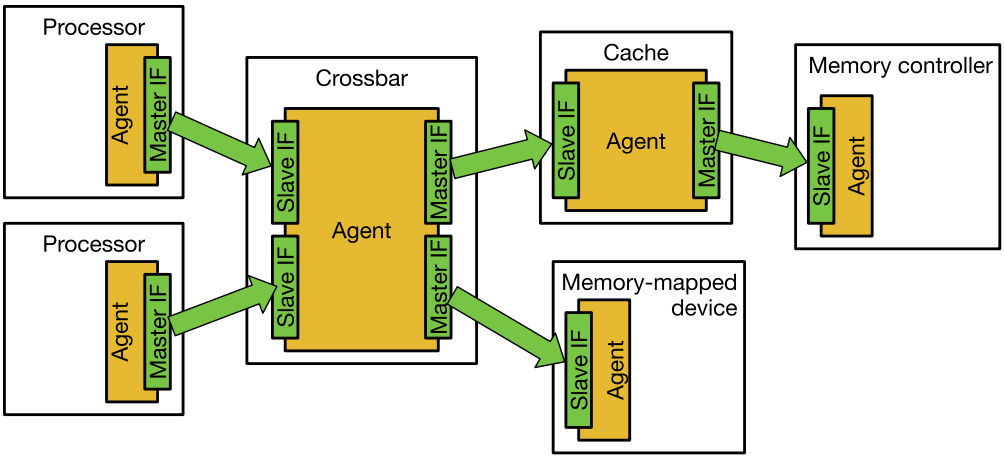


Figure 3.6: Connection model in TileLink and its interfaces. Source: [24]

TileLink is a highly parameterizable, open-source, shared memory interconnect protocol based on the aforementioned crossbar structure, designed to connect diverse modules for SoCs. TileLink is implemented via a hierarchical, point-to-point network that can be easily scaled. This protocol provides memory-mapped access for multiple controllers, ensuring memory coherency while supporting efficient communication between processors, DMAs, and peripheral devices.

A TileLink bus system can support a combination of multiple communicating agents, each capable of supporting different configured subsets of the protocol. The TileLink specification includes three levels of operational configuration for agents, presented in Table 3.1.

Table 3.1: TileLink operational configurations. Source: [24]

	TL-UL	**TL-UH**	**TL-C**
Read/Write operations	y	y	y
Multibeat messages	.	y	y
Atomic operations	.	y	y
Hint operations	.	y	y
Cache block transfers	.	.	y
Channels B+C+E	.	.	y

TileLink clearly distinguishes between cached and uncached communications. The simplest level is TileLink Uncached Lightweight (TL-UL), which only supports basic read and write operations to memory, for individual words. The next, more complex level is TileLink Uncached Heavyweight (TL-UH), which adds features such as "hints," "atomic" operations, and burst accesses but does not support coherent caching. Here, "hints" refers to a feature where a request command seeks data information external to the executing command from masters, and "atomic" refers to operations that lock access rights in a system with multiple masters, which could be microprocessors. Finally, TileLink Cached (TL-C) is the full protocol, supporting the use of coherent caches within the system.

For integrating peripheral hardware, all three bus configuration levels can be used. The communication connection diagram, at its most complete level, between a custom peripheral hardware and an agent is shown in Figure 3.7.

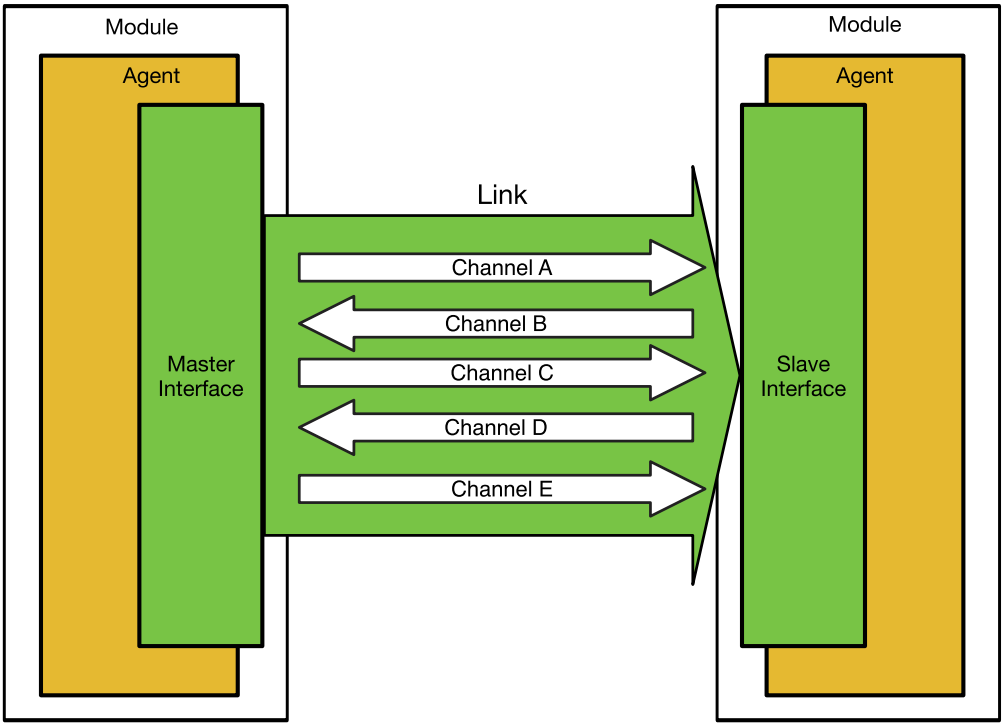


Figure 3.7: Channels in a TileLink connection between two agents. Source: [24]

In this TileLink protocol, the defined communication channels include `A`, `B`, `C`, `D`, and `E`, each undertaking specific roles in the interaction process between a master (M) and a slave (S), and all have a defined direction of transmission as depicted in the figure above.

- **Channel A:** This is the initiation channel, where M sends requests to S,

establishing the initial step of the communication process.

- **Channel B:** This channel is used when M requests a cache block from S, ensuring necessary data is queried.

- **Channel C:** Used for S to respond to M's request for a cache block, playing a crucial role in providing necessary data to M.

- **Channel D:** This is the general response channel, where S sends the final response to M after processing requests.

- **Channel E:** This channel ensures transaction completion by performing the final handshake for cache block transfer between the two parties.

The priority order for these channels is as follows: `A` $<$ `B` $<$ `C` $<$ `D` $<$ `E`, where channel `A` has the lowest priority and `E` has the highest. Prioritization in the TileLink bus system ensures that messages transmitted through the system do not fall into a hold-and-wait loop. In other words, the message flow through all channels between agents is maintained as a directed acyclic graph, enabling the TileLink protocol to operate without encountering deadlocks, ensuring continuous and uninterrupted data flow within the system.

The choice of TileLink bus configuration is relevant for connecting various memory-mapped peripherals within the system. For general-purpose MMIO peripherals, which might include cryptographic algorithm accelerators designed in Verilog HDL (if they were to be integrated as MMIO devices), the TL-UL configuration, comprising only channels `A` and `D` between M and S (as shown in Figure 3.8), is often sufficient. However, for the primary accelerator in this thesis, the Gemmini core, integration is achieved through the RoCC interface, which provides a more tightly-coupled path to the processor core, rather than as a TileLink-attached MMIO peripheral. Details on the RoCC interface are discussed further in Section 3.4.4. For other standard MMIO peripherals, a wrapper from the Rocket Chip library, using Chisel/Scala, can be inherited and customized to connect their signals to the appropriate system bus, typically the peripheral bus.

At the system configuration level, a wrapper from the Rocket Chip library, using Chisel/Scala, can be inherited and customized to convert the signals of the accelerator core to the system bus, in this case, the peripheral bus. Details for this interface will be further elaborated in subsequent sections of this thesis.

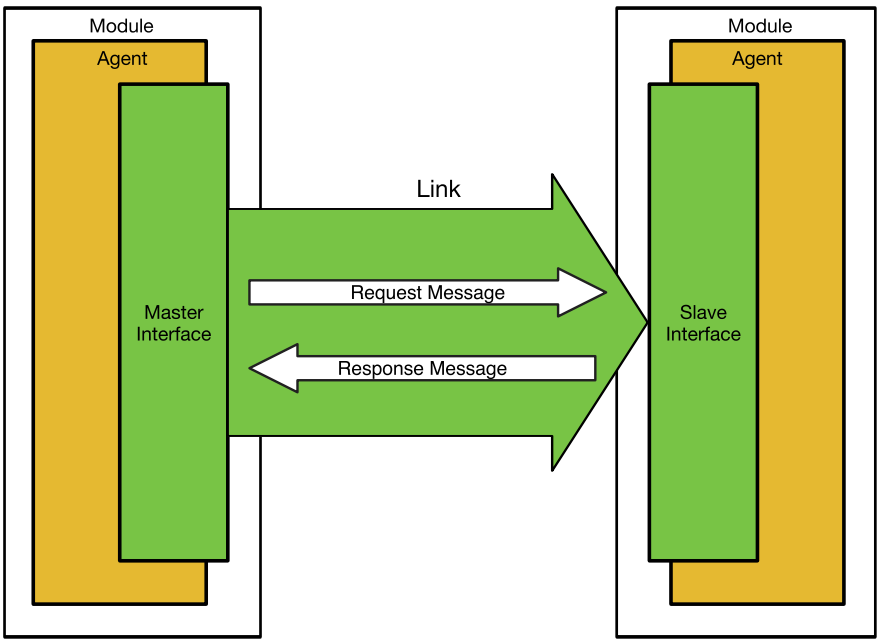


Figure 3.8: TileLink TL-UL connection structure. Source: [24]

3.4.2 Memory and Cache

Each Tile in the system is integrated with a processor core and an L1 cache that is flexibly configurable. By default design with Chisel, the L1 cache has a size of 16 KiB, organized multi-dimensionally with sets and ways. The L1 cache uses a set-associative cache structure, dividing the memory region into sets, each set containing a fixed number of data blocks (ways). When a memory address access request occurs, the index derived from the address maps it to a specific set. The cache controller then searches for the data block within the ways of that set. This design supports both instruction and data storage, optimizing access performance and increasing the hit rate compared to caches with lower associativity.

In addition to the L1 cache, the system also integrates a shared L2 cache, connected to all cores via the TileLink bus system. The L2 cache acts as an intermediate storage repository, allowing processor cores and other components in the system to perform read/write data requests. This is a coordination mechanism that helps reduce pressure on the main memory and improves overall processing performance. The L2 cache controller also ensures fairness in resource allocation, using a queuing system to manage access requests.

At a higher level, the system also supports the integration of external memory, such as DRAM or SRAM, through open-source controllers or specialized IPs from FPGA implementation toolkits. Additionally, an interface with DRAMSim is sup-

ported for integration when performing full-system simulation. This provides flexibility and scalability, meeting various requirements in practical applications.

3.4.3 Core-Complex

As previously mentioned, the Chipyard framework and library consist of many component definition classes that can form a Rocket Chip system. This allows for the formation of a complex system with diverse processor cores and peripherals, specifically heterogeneous architecture systems comprising multiple different processors like Rocket and BOOM coexisting on the system.

In these multi-core processor structures, each individual core will be uniquely identified with a different hart ID, ordered according to the system configuration. An example can be referenced below with a configuration comprising one Rocket core and one BOOM core. The corresponding block diagram configuration when generating the system will be as shown in Figure 3.9 below.

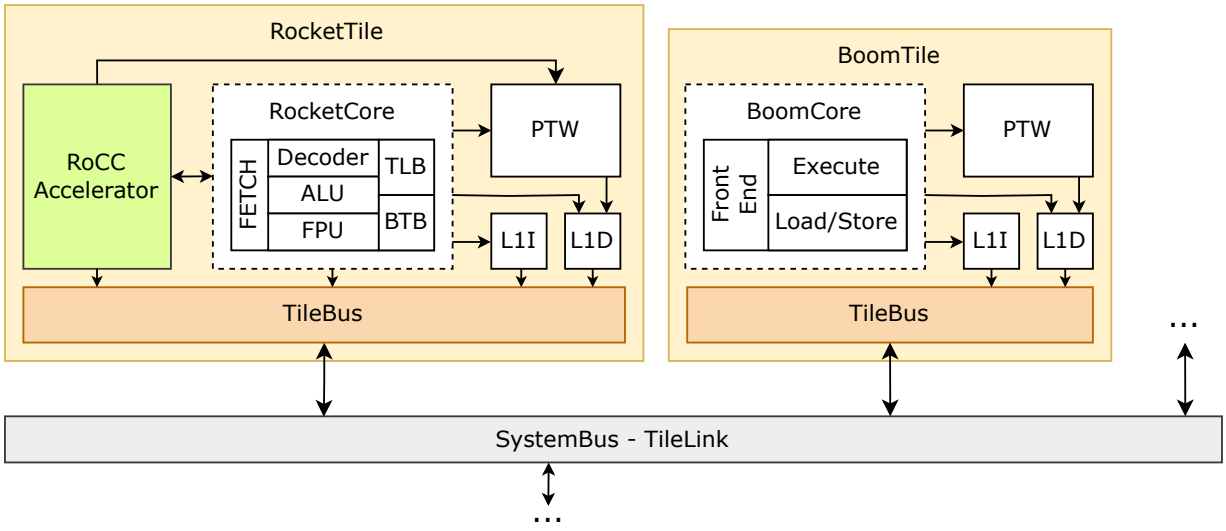


Figure 3.9: Implementation structure of a heterogeneous multi-core processor system.

A heterogeneous SoC system is an implementation approach also considered in this thesis to combine different processor cores to optimize performance and energy efficiency. At the application scale, lightweight cores can handle simple tasks with low power consumption, while high-performance cores support hyper-threading and out-of-order execution to process complex tasks. This design ensures flexibility, resource optimization, and superior performance for diverse applications.

3.4.4 Peripheral Components

In the base system configuration class of Rocket Chip, many peripheral components are initialized and integrated by default to support both simulation and deployment on FPGA hardware for the SoC system. This configuration includes basic I/O devices such as PWM, interrupt controllers, JTAG, ROM, external memory, and standard peripheral interfaces, ensuring that basic communication, storage, and control requirements are met. In particular, common peripheral devices like UART, SPI Flash, and GPIO are implemented to support serial communication, storage management, and flexible control within the system.

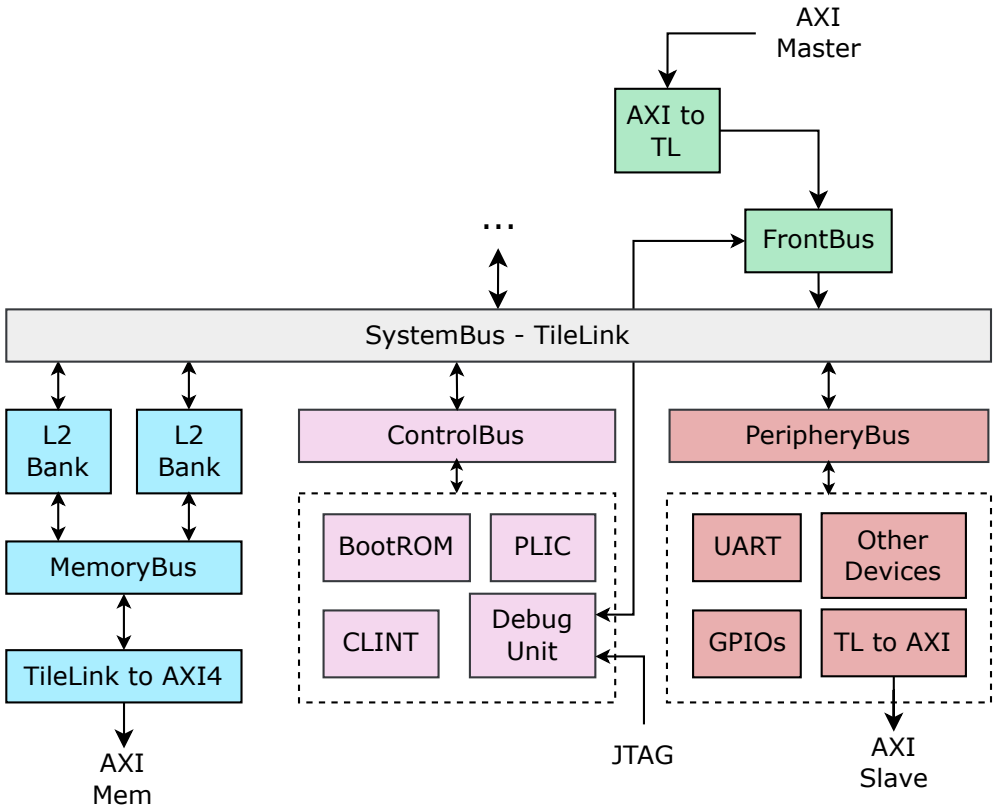


Figure 3.10: Peripherals in the Rocket Chip base system.

All these components are developed based on the open-source Rocket Chip library, with hardware designs like Rocket and BOOM realized on the Chisel platform. When external memory is needed in design and simulation, the `AXI4` (Advanced eXtensible Interface 4) interface can be integrated as an intermediate layer, allowing effective connection with DRAM or SPI Flash models. Concurrently, serial protocols like JTAG are integrated to ensure accurate debugging capabilities and efficient serial communication, configured to connect with the TileLink bus. The Rocket Chip

system also supports flexible configuration through special control ports, including custom boot pins and chip identification ports. Furthermore, the Rocket Chip system provides the capability to directly map bus interfaces such as `AXI4` Memory and MMIO (Memory-Mapped IO), allowing for expanded compatibility and optimized performance in complex SoC applications.

To expand the capability of integrating components and peripheral cores to form highly parameterized custom systems suitable for specific applications, the Rocket Chip architecture provides two flexible methods for peripheral integration. Specifically, the system can integrate a peripheral core as a `RoCC` (Rocket Custom Coprocessor), located within each tile, or integrate at the system communication level via the MMIO mechanism. These methods facilitate in-depth customization, meeting various requirements for each specific application.

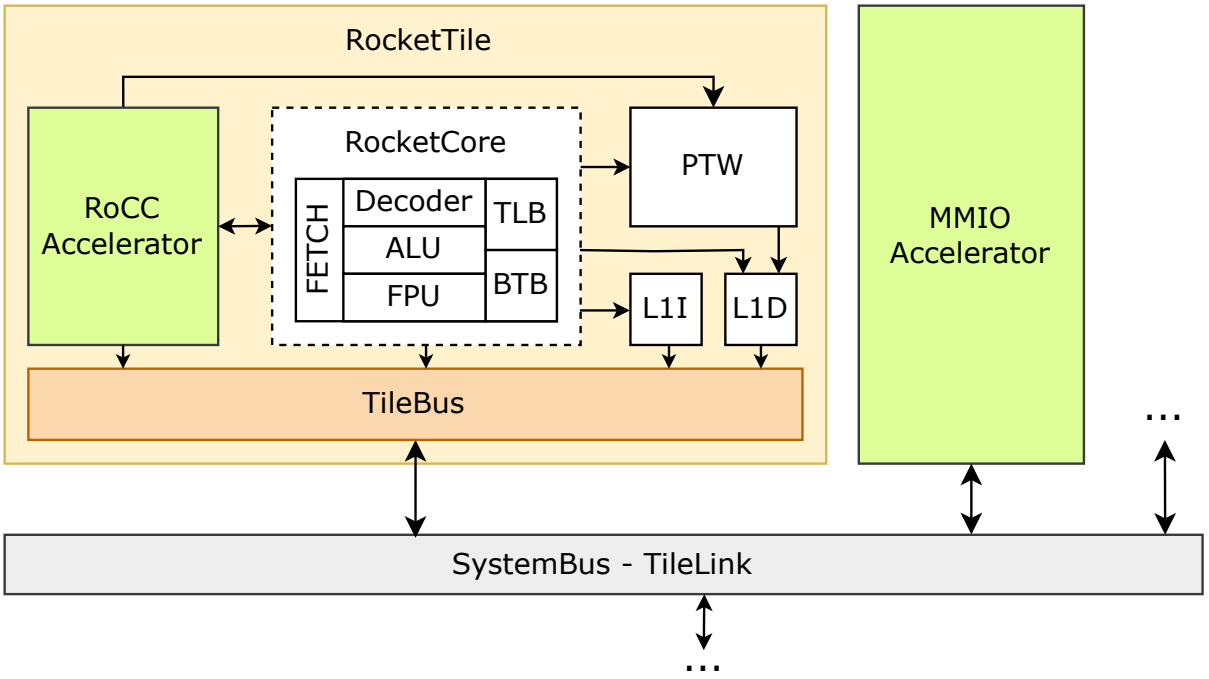


Figure 3.11: `RoCC` co-processor core and MMIO peripheral core in Rocket Chip.

MMIO Peripherals are custom peripherals attached directly to the TileLink bus, while Tightly-Coupled `RoCC` Accelerators are custom peripherals attached directly to the arbiter and processor within each Tile, in parallel with the processor core. With the TileLink-Attached MMIO method, the processor communicates with MMIO devices through registers mapped into memory at addresses on the system. Through these registers, control mechanisms or data transfer can be customized to suit the application of the peripheral core. Conversely, in the `RoCC` Accelerator method, com-

munication is performed via a custom protocol and non-standard ISA instructions, which are configured and defined separately within the encoding space of the RISC-V ISA. For Rocket and BOOM, each core can control up to four accelerators through custom opcodes, sharing resources with the microprocessor. Instructions executed with `RoCC` cores have the format `customX rd, rs1, rs2, funct`. Here, `X` is a number from 0-3, determining the opcode that controls the routing of the instruction to a specific accelerator. The fields `rd`, `rs1`, `rs2` are the register numbers for the destination register and two source registers, and the `funct` field is a 7-bit integer that the accelerator can use to differentiate various instructions.

Peripherals connected as `RoCCs` have a distinct advantage due to their high customizability and direct connection and control via custom ISA instructions. This allows for specialized performance optimization, particularly useful for cores requiring large, complex computational workloads or applications needing extremely fast response times. Communication via a custom protocol and leveraging the RISC-V ISA helps `RoCC` utilize CPU resources efficiently, while also offering flexibility in designing specialized accelerator processors. The Gemmini accelerator, central to this thesis, is a prime example of a `RoCC`-based component, designed for high-throughput matrix multiplication and tightly integrated with the Rocket core.

While `RoCC` implementation can require a custom software toolchain for its specialized instructions, the performance benefits for demanding applications like those targeted by Gemmini often justify this investment. For this thesis, the integration of the Gemmini accelerator via the `RoCC` interface is deemed the most appropriate approach to achieve the desired performance for deep learning workloads.

Conversely, the MMIO TileLink-Attached method is widely used for integrating general-purpose memory-mapped peripheral devices. In this model, peripheral devices provide a set of registers (e.g., Chisel Registers) to communicate with microprocessors or other masters on the system bus. This method is suitable for peripherals where the tight coupling and custom instruction set of `RoCC` are not necessary.

To minimize design complexity for MMIO peripherals, the Rocket Chip architecture provides the `regmap` interface, which allows for the automatic generation of much of the necessary linking logic. Designers can use `TLRegisterRouter` or create a `LazyModule` and initialize `TLRegisterNode` to integrate such devices. Since TileLink is the official protocol used in SoCs based on Rocket Chip and Chipyard, designing MMIO peripheral devices (other than the primary `RoCC` accelerator) would typically focus on integration with this protocol.

3.4.5 The RoCC Interface: A Hardware Co-Design Methodology

The primary conduit for communication between the Rocket Core and the Gemmini accelerator is the RoCC (Rocket Custom Coprocessor) interface. This interface, however, is not merely an external bus; it is a profound hardware **co-design methodology** that fundamentally alters the CPU's own pipeline. When the `WithGemmini` trait is active in the SoC configuration, the Chipyard generator does more than just instantiate the accelerator; it injects specific logic directly into the Rocket Core's decoder, hazard unit, and writeback stage.

The architectural philosophy of the RoCC interface is designed for high-throughput, decoupled accelerators. As it is were found in prior work by Pala, the interface's high-latency profile for CPU-to-accelerator register reads makes it best suited for offloading large, autonomous tasks rather than for frequent, small operations that require constant communication between CPU and accelerator [21]. Gemmini, which operates on large data streams for DNN computation, is a canonical example of this intended use case.

This section will trace the lifecycle of a RoCC instruction to demonstrate how these code-level modifications create a tightly-coupled, low-latency communication channel that is essential for high-performance acceleration.

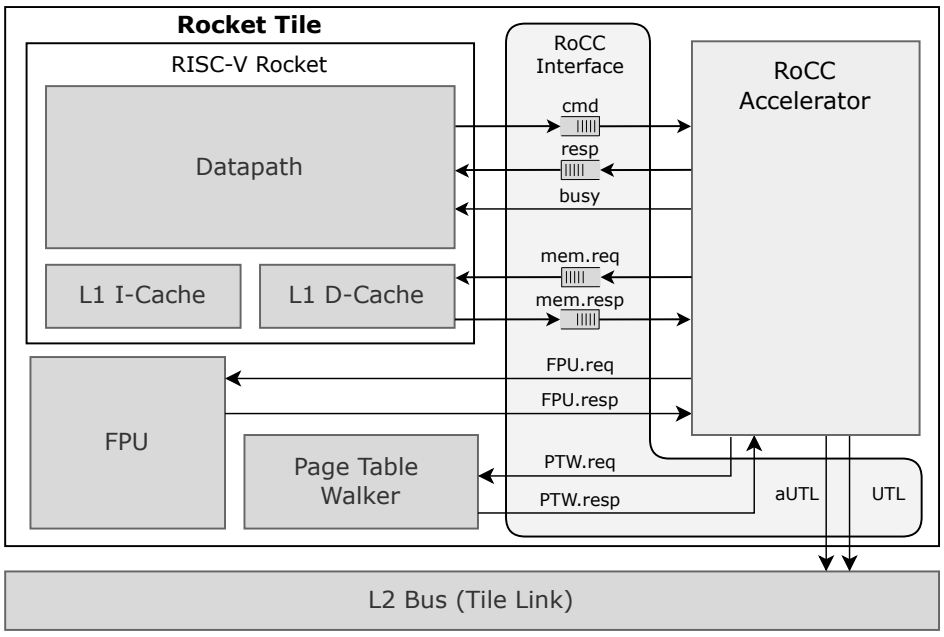


Figure 3.12: Rocket Tile RoCC Interface Architecture with Accelerator Integration. The RoCC interface provides direct communication pathways between the Rocket Core's datapath and the attached accelerator, including command dispatch, memory requests, and response handling channels.

3.4.5.1 The RoCC Instruction Format

The entire co-design process is initiated by a specific instruction format recognized by the CPU. The RISC-V ISA reserves four major opcodes for custom extensions, and the Gemmini accelerator is assigned the `custom3` opcode space. Any instruction using this opcode follows the R-type format, modified for RoCC communication as shown in Figure 3.13.

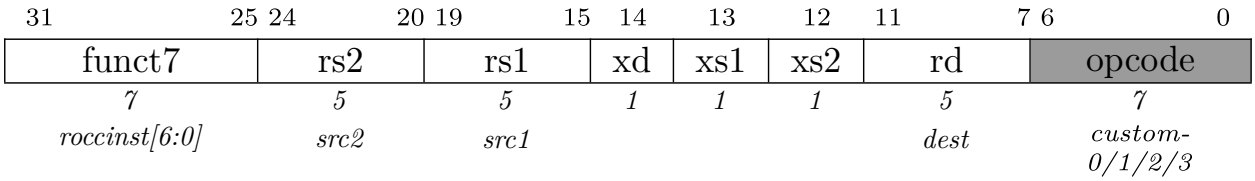


Figure 3.13: The RoCC Instruction Format. The `opcode` field is set to `custom3` (0b1111011). The `funct7` field specifies the Gemmini-specific operation, while the `xd`, `xs1`, and `xs2` bits act as control signals for the hardware interface.

The fields in this instruction format are critical for orchestrating the hardware interaction:

- `opcode`: Identifies the instruction as a RoCC command, directing it away from the standard execution units.
- `funct7`: This 7-bit field is passed directly to Gemmini, which uses it to select the specific operation to perform (e.g., `LOOP_CONV_WS`, `CONFIG_LOAD`).
- `xd, xs1, xs2`: These single-bit fields are essential hardware control signals. They function as "valid" bits for the register operands. `xs1` and `xs2` signal that valid data is present on the `rs1` and `rs2` busses, while `xd` signals that the CPU expects a result for the destination register `rd` and must stall until a response arrives.

3.4.5.2 Modifying the CPU's Instruction Decoder

For the co-design to function, the CPU must first be taught to recognize the new instruction format. The Chipyard generator achieves this by programmatically modifying the main decoder table in the Rocket Core, as shown in Listing 3.1.

```
1 // From rocket-chip/src/main/scala/rocket/IDecode.scala
2 CUSTOM3 -> List(Y,N,Y,N,N,N,N,N,A2_ZERO,A1_RS1,IMM_X,DW_XPR,...),
```

Listing 3.1: Modification of the Rocket Core's instruction decode table.

This modification is the first and most critical step. It instructs the decoder that the `custom3` opcode is a valid instruction that uses the register file (as specified by `A1_RS1`) but not the standard ALU. This prevents the CPU from triggering an illegal instruction trap and instead flags the instruction internally as a `rocc` command, allowing it to propagate down the pipeline for specialized handling, as illustrated in Figure 3.14.

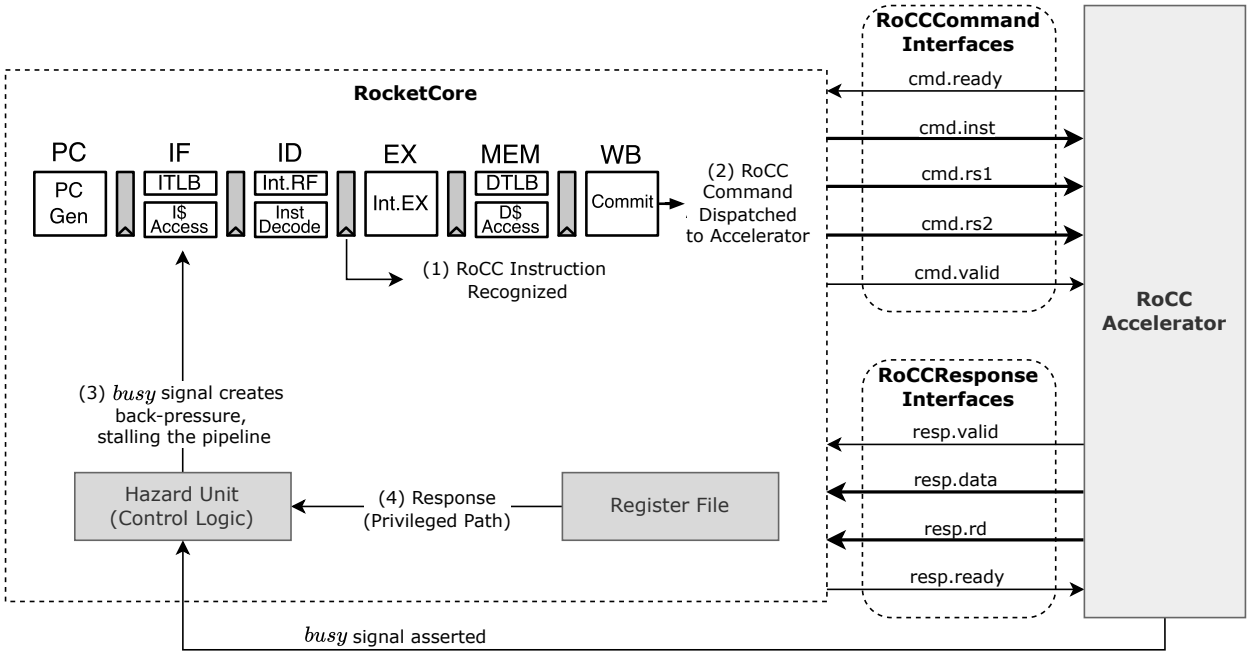


Figure 3.14: Diagram illustrating the RoCC co-design and control flow. (1) A RoCC instruction is recognized at the Decode stage. (2) At the Writeback stage, the `RoCCCommand` is dispatched to Gemmini over a `ready/valid` channel. (3) A `busy` signal provides back-pressure to the CPU's Hazard Unit, stalling the pipeline. (4) The `resp` path provides a privileged write-back directly to the Register File.

3.4.5.3 Pipeline Integration and Hazard Management

The tight coupling between the CPU and Gemmini necessitates new hardware interlocks to manage pipeline hazards. These interlocks are injected directly into the CPU's control logic. If Gemmini is busy and cannot accept a new command, it exerts **back-pressure** on the CPU. The hardware logic shown in Listing 3.2 generates a stall signal that freezes the CPU pipeline until the accelerator is ready.

```
1 // Source: rocket-chip/src/main/scala/rocket/RocketCore.scala
2
3 // Stall decode if RoCC is not ready for a new command
```

```
val rocc_blocked = Reg(Bool())
rocc_blocked := !wb_xcpt && !io.rocc.cmd.ready &&
                (io.rocc.cmd.valid || rocc_blocked)

// Replay the instruction in the writeback stage
val replay_wb_rocc = wb_reg_valid && wb_ctrl.rocc &&
                     !io.rocc.cmd.ready
```

Listing 3.2: RoCC-specific stall logic in the Rocket Core.

This logic provides definitive proof of a hardware co-design. The `replay_wb_rocc` signal physically stalls the CPU pipeline if a command is in the writeback stage and Gemmini is not ready (`!io.rocc.cmd.ready`). Furthermore, the `rocc_blocked` register freezes the pipeline at the decode stage if a new RoCC instruction arrives while the accelerator is already busy. The CPU's progression is now physically dependent on the state of the accelerator.

3.4.5.4 The Privileged Response Path

Finally, the RoCC interface provides a privileged, low-latency path for the accelerator to write results back into the CPU's register file. This is arbitrated with other internal units, as shown in Listing 3.3.

```
// Source: rocket-chip/src/main/scala/rocket/RocketCore.scala
when (io.rocc.resp.fire) {
  div.io.resp.ready := false.B // Disable div response
  ll_wdata := io.rocc.resp.bits.data
  ll_waddr := io.rocc.resp.bits.rd
  ll_wen := true.B
}
```

Listing 3.3: RoCC response handling in the Rocket Core.

When the Gemmini accelerator has a result, the hardware routes the response data (`io.rocc.resp.bits.data`) directly to the register file's write port (`ll_wdata`) and asserts the write enable signal (`ll_wen`). This is a privileged write path that completely bypasses the memory system, a feature impossible to achieve with a standard peripheral.

3.4.5.5 The RoCC Memory Interface

Beyond the command interface, the RoCC protocol provides a privileged, high-bandwidth connection directly to the CPU's L1 Data Cache, shown in Figure 3.15.

This is a key advantage over a standard memory-mapped peripheral, which would suffer from the higher latency of the L2 cache and system bus.

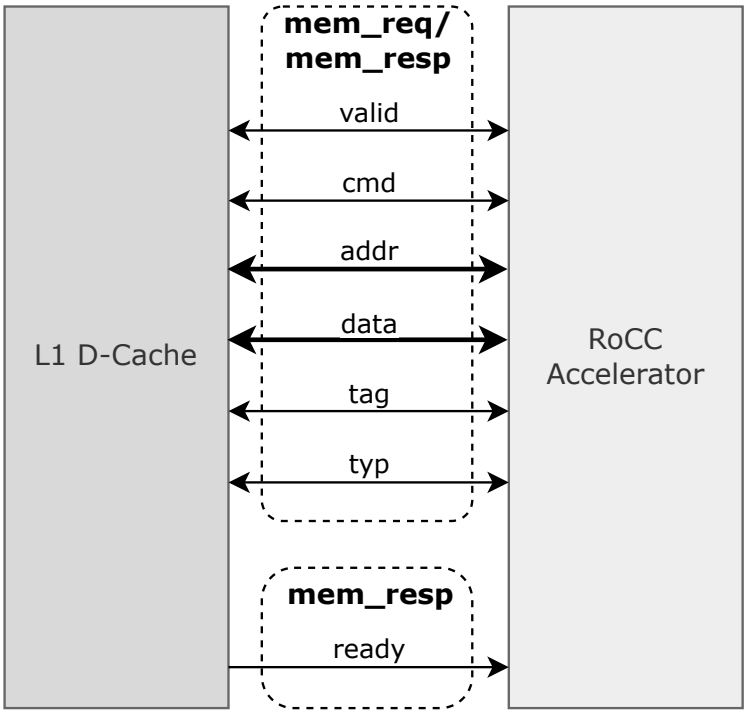


Figure 3.15: The asymmetrical RoCC memory interface. The request channel (`mem_req`) uses a full `ready/valid` handshake, while the response channel (`mem_resp`) is a simpler `valid`-only interface.

Critically, this interface is asymmetrical. The request channel from Gemmini to the L1D$ (`mem_req`) is a full `Decoupled` interface, allowing Gemmini to wait if the cache is busy. However, the response channel from the L1D$ to Gemmini (`mem_resp`) has only a `valid` signal. This is a deliberate design choice: the accelerator cannot apply back-pressure to the L1D$ and must accept memory responses immediately to prevent stalling the main CPU pipeline. This nuance highlights the deeply integrated nature of the memory co-design. As confirmed by the timing analysis in Figures 3.16 and 3.17, this interface exhibits a two-cycle latency and a maximum throughput of 50%, performance characteristics that must be considered when designing and scheduling operations for the accelerator

3.4.5.6 Conclusion

In summary, the RoCC integration is a profound act of hardware co-design, proven by the evidence from the hardware generator. The process modifies the CPU's instruction decoder, injects new stall conditions into the core hazard unit, provides

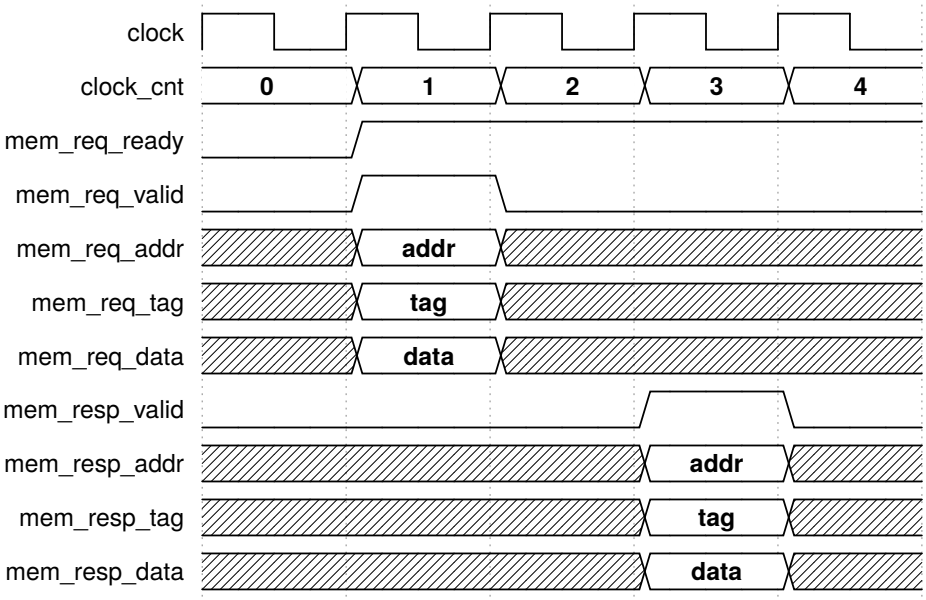


Figure 3.16: Timing diagram for a single RoCC memory request. The diagram demonstrates the two-cycle latency of the memory interface, as the `mem_resp_valid` signal is asserted two clock cycles after the `mem_req_valid` signal. Note the absence of a `mem_resp_ready` signal, confirming the asymmetrical nature of the interface. [21]

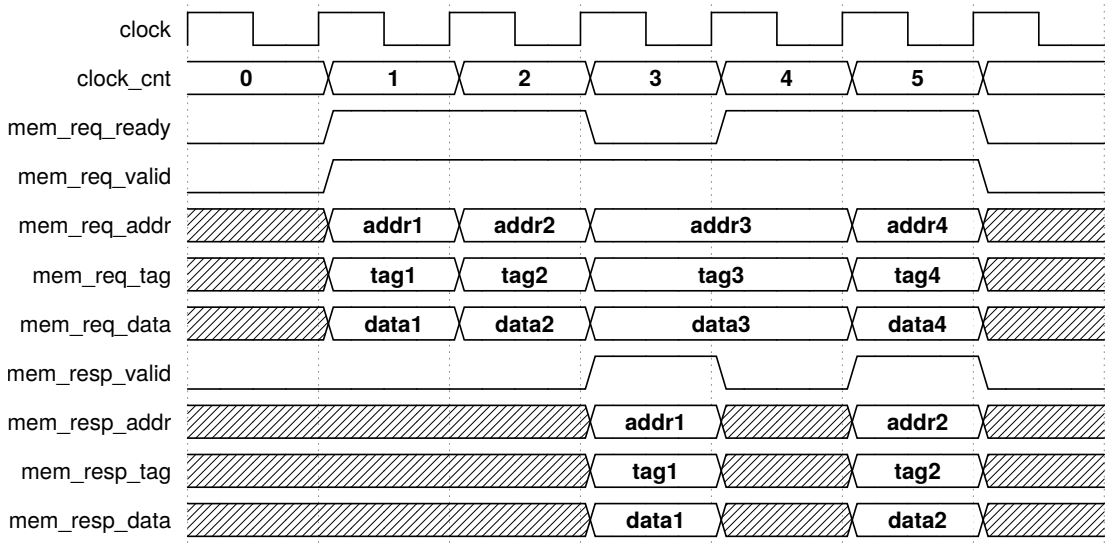


Figure 3.17: Timing diagram for consecutive RoCC memory requests. This illustrates the 50% maximum throughput of the interface. The L1D$ can accept two back-to-back requests but then requires a one-cycle pause before accepting the next two, a critical performance characteristic for data-intensive accelerators. [21]

a privileged response path to the register file, and grants direct, arbitrated access to the L1 cache. These code-level modifications create the tightly-coupled, high-performance link necessary to realize the performance uplift for DNN workloads,

which will be quantified in Chapter 8.

With the hardware co-design methodology now established as the mechanism for integrating our accelerator, the next chapter will explore the architectural foundations of Deep Neural Network (DNN) acceleration to understand why such specialized hardware like Gemmini is required in the first place. The complete, detailed definitions of the RoCC hardware interface bundles are provided in Appendix **??** for reference.

Chapter 4: Architectural Foundations of DNN Acceleration

The efficient execution of Deep Neural Networks (DNNs) presents one of the most significant challenges and opportunities in modern computer architecture. While today's general-purpose CPUs are highly optimized for a wide range of tasks, their fundamental design principles are often mismatched with the unique computational and data movement patterns of large-scale neural networks. This chapter will establish the foundational concepts that motivate the need for specialized hardware. We will begin by exploring the core problem—the immense cost of data movement—and then introduce the classic architectural paradigm designed to solve it: the systolic array. Finally, we will examine how this timeless principle is embodied in state-of-the-art industrial accelerators, providing the necessary context to understand the design of the Gemmini accelerator, which is the central subject of this thesis.

4.1 The Data-Compute Divide: The Memory Wall

The performance of any computing system is ultimately limited by two factors: the speed of its computations and the speed at which it can supply data to its computational units. For decades, Moore's Law drove exponential growth in the number of transistors on a chip, leading to dramatic increases in processing power. However, the speed of off-chip memory systems, such as DRAM, has improved at a much slower pace. This growing disparity between processor speed and memory speed is famously known as the **Memory Wall** [31].

For data-intensive workloads like DNNs, this is the primary performance bottleneck. The problem can be understood with a simple analogy: imagine a master chef (the processor) who can perform culinary tasks with superhuman speed. However, their ingredients (the data) are stored in a large warehouse across the street (DRAM). Even though the chef is exceptionally fast, their overall productivity is dominated by the time spent walking to and from the warehouse to fetch each ingredient.

This bottleneck is not just about latency; it is fundamentally about energy. As quantified by Horowitz [13], the energy required to perform a complex 32-bit floating-point computation is dwarfed by the energy needed to fetch its operands from off-chip DRAM (Figure 4.1). This immense energy cost of data movement means that for DNNs, which perform billions of operations, any viable high-performance hardware solution must be designed with a primary objective: to minimize data movement.

Operation:	Energy (pJ)
8b Add	0.03
16b Add	0.05
32b Add	0.1
16b FP Add	0.4
32b FP Add	0.9
8b Multiply	0.2
32b Multiply	3.1
16b FP Multiply	1.1
32b FP Multiply	3.7
32b SRAM Read (8KB)	5
32b DRAM Read	640

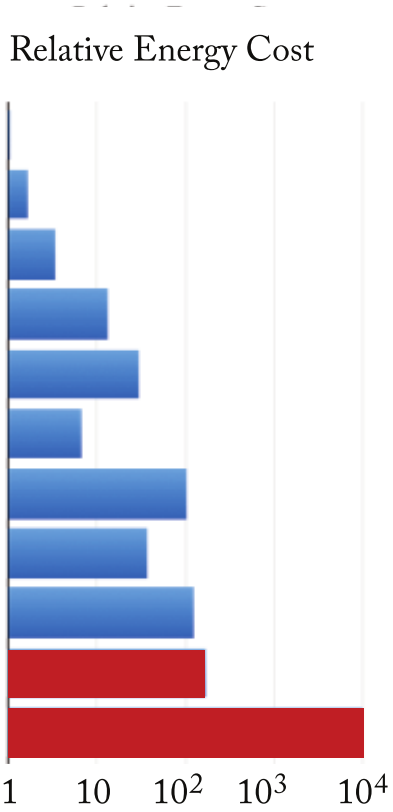


Figure 4.1: The relative energy cost of various arithmetic and memory operations. A 32-bit DRAM read is orders of magnitude more costly than a 32-bit floating-point addition. This disparity is the primary motivation for specialized accelerator architectures. Adapted from Horowitz, as presented in Sze et al. [25, 13].

4.2 A Classic Solution: The Systolic Principle

In his seminal 1982 paper, H.T. Kung proposed a novel architectural paradigm designed specifically to address the memory wall: the **systolic array** [17]. The architecture is named by analogy to the human circulatory system. Just as the heart rhythmically pumps blood to the body's cells, in a systolic system, memory "pumps" data through a regular grid of simple Processing Elements (PEs).

The core principle is to **maximize the number of computations performed for each data item fetched from memory**. Figure 4.2 provides a powerful conceptual comparison. In a conventional architecture, each operand must be fetched from memory for a single operation in the PE, making the system bottlenecked by memory bandwidth. In contrast, the systolic approach creates a pipeline of PEs. A single data element is fetched and then passed from one PE to the next, participating in a new computation at each step. This data reuse allows the systolic architecture to achieve significantly higher throughput with the exact same memory bandwidth.

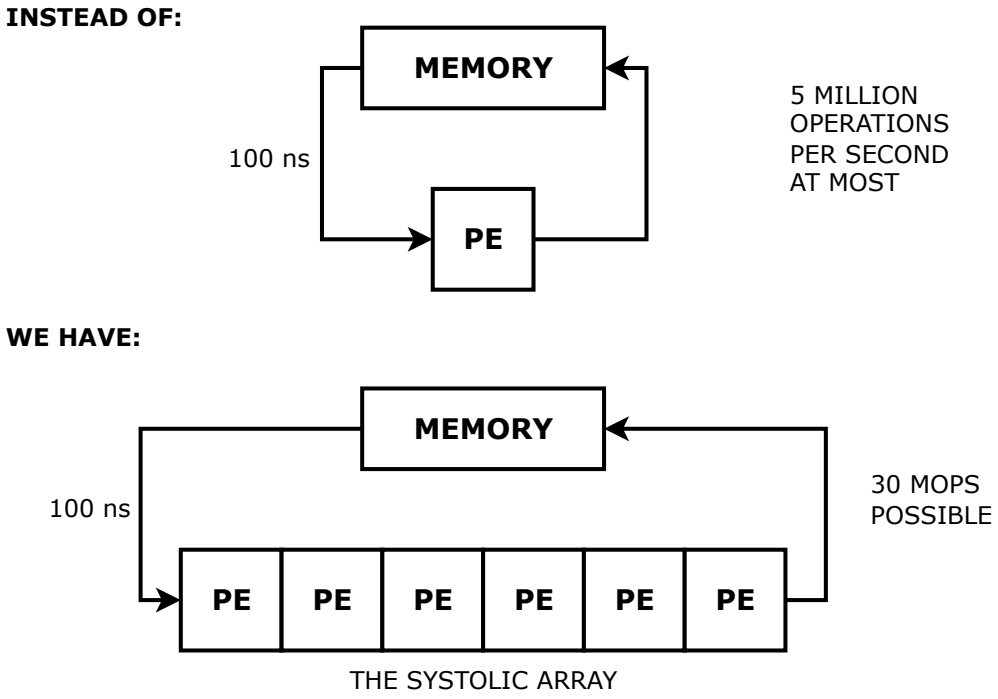


Figure 4.2: A conceptual comparison of a conventional von Neumann architecture and a systolic architecture. The conventional approach (top) is bottlenecked by the memory bus. The systolic approach (bottom) uses an array of PEs to achieve high throughput with the same memory bandwidth by reusing data internally. Adapted from the presentation of Kung's work [17].

While Kung's original comparison work laid the theoretical foundation and illustrated the benefit, the mechanism for achieving this data reuse is elegant in its own right. As shown in Figure 4.3, modern systolic arrays implement this principle for matrix multiplication by feeding input operands from two matrices (A and B) into the array from orthogonal sides. The values from matrix A flow horizontally, while the values from matrix B flow vertically. As these data wavefronts intersect at each clock cycle, every PE in the grid performs a Multiply-Accumulate (MAC) operation in parallel. The result is a massively parallel computation structure that amortizes the high energy cost of a single memory access over dozens or even hundreds of useful arithmetic operations, providing a direct architectural solution to the data movement bottleneck.

4.3 Case Studies in Modern Systolic Design

While the systolic principle is four decades old, it has become the dominant paradigm for modern high-performance DNN accelerators. We will examine two

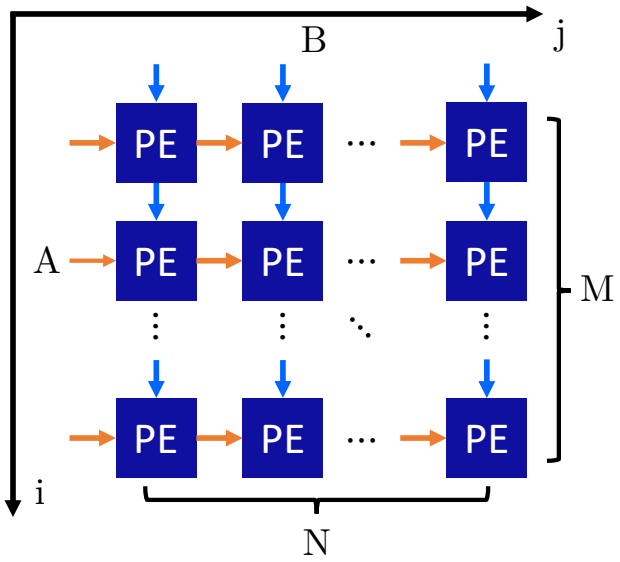


Figure 4.3: The fundamental principle of a systolic array for matrix multiplication. Input matrix A flows horizontally, while input matrix B flows vertically. This rhythmic dataflow allows for massive parallelism and data reuse.

landmark industrial designs that showcase this principle in practice.

4.3.1 Google's TPUv1: A Datacenter-Scale Design

Google's Tensor Processing Unit (TPU) was one of the first large-scale custom ASICs designed specifically for accelerating DNN inference in datacenters [14]. As shown in Figure 4.4, its architecture is a direct and powerful embodiment of the systolic principle. The heart of the TPU is a massive 256x256 Matrix Multiply Unit, a systolic array of 65,536 8-bit multiply-accumulate (MAC) units.

The TPUv1's design philosophy is a masterclass in optimizing for a specific domain: high-throughput datacenter inference. Several key decisions reflect this focus. First, the use of 8-bit integer arithmetic was a critical trade-off, sacrificing the precision of floating-point numbers for immense gains in area and energy efficiency. This was based on the insight that for inference, the resilience of neural networks to quantization allows for acceptable accuracy with lower-precision data. Second, the enormous on-chip Unified Buffer (a 24 MiB software-managed scratchpad) is essential to "keep the beast fed"—it buffers entire layers of weights and activations on-chip to ensure the massive systolic array is never starved for data. This design choice implicitly favors large batch sizes, where the cost of loading the model can be amortized over many simultaneous inference requests, prioritizing throughput over single-request latency.

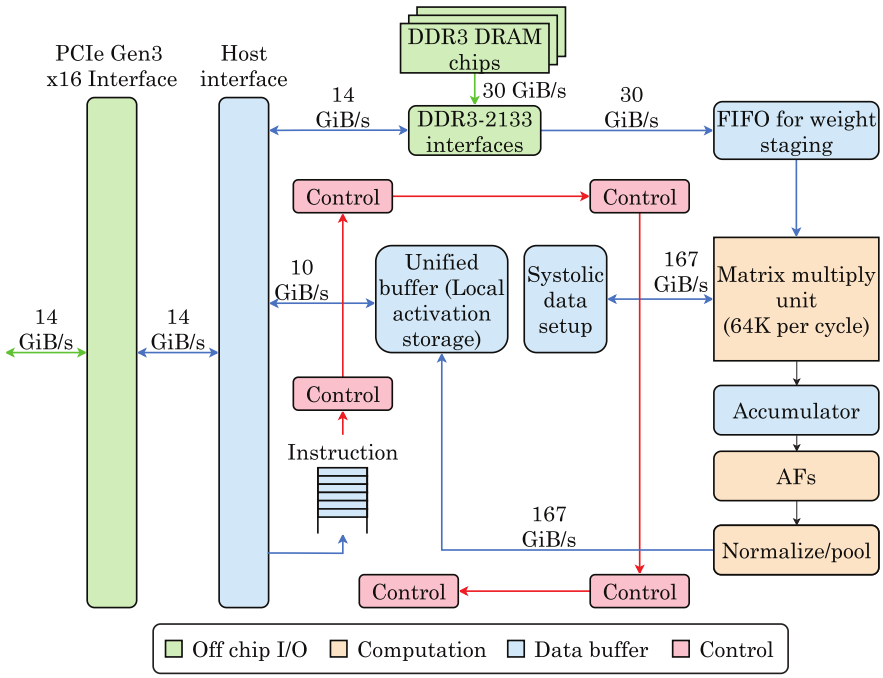


Figure 4.4: A high-level block diagram of the Google TPUv1. The design is dominated by the systolic Matrix Multiply Unit and its large on-chip Unified Buffer. Adapted from Jouppi et al. [14, 20].

4.3.2 NVIDIA NVDLA: An Open-Source Accelerator IP

In contrast to the monolithic, datacenter-focused TPU, the NVIDIA Deep Learning Accelerator (NVDLA) is an open-source, configurable IP core designed for integration into a wide range of Systems-on-Chip (SoCs), particularly for embedded systems [8]. The NVDLA's design philosophy prioritizes **modularity and integration flexibility** over raw scale.

As shown in Figure 4.5, the architecture is partitioned into distinct, specialized hardware units for convolution, post-processing, and memory management. This modularity is a key feature, allowing an SoC designer to integrate only the necessary components. A significant design choice differentiating the NVDLA from the TPU is its reliance on industry-standard bus protocols like AXI. While this may introduce more protocol overhead than a custom interface, it dramatically simplifies the process of integrating the NVDLA into a third-party SoC from a different vendor. Furthermore, its configurability (e.g., in the number of MAC units from the `nv_small` to `nv_large` configurations) makes it a versatile solution that can be tailored to the specific power, performance, and area (PPA) constraints of an embedded device.

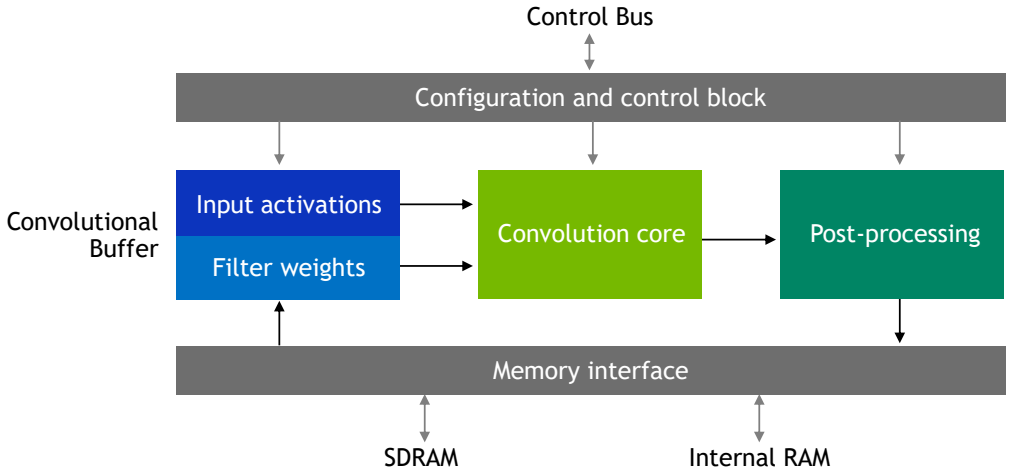


Figure 4.5: The high-level architecture of the NVDLA. The design is partitioned into distinct functional units connected via standard bus interfaces. [8].

4.4 A Formal Framework: Dataflows

The design choices made in architectures like the TPU and NVDLA can be formally described using the concept of a **dataflow**. The dataflow defines the strategy for moving data through the systolic array to maximize reuse. The effectiveness of a given dataflow depends on the structure of the DNN layer being executed. The two most common dataflows, which we will see are directly relevant to Gemmini, are:

- **Weight Stationary (WS):** This dataflow prioritizes weight reuse. Filter weights are pre-loaded into the PEs and remain stationary, while input activations are streamed in. This strategy, used by the Google TPU, is highly efficient for layers with large input feature maps and relatively small filters.
- **Output Stationary (OS):** This dataflow prioritizes minimizing the movement of partial sums. Each PE is responsible for accumulating the final value for a single output activation, which remains stationary in its accumulator. Inputs and weights are streamed to the PE as needed. This can be more efficient for layers where partial sum read/write energy is a dominant cost.

These strategies, visually contrasted in Figure 4.7 and 4.9, represent a fundamental trade-off in accelerator design. The choice of dataflow directly impacts which types of data reuse, shown in Figure 4.6, are most effectively exploited.

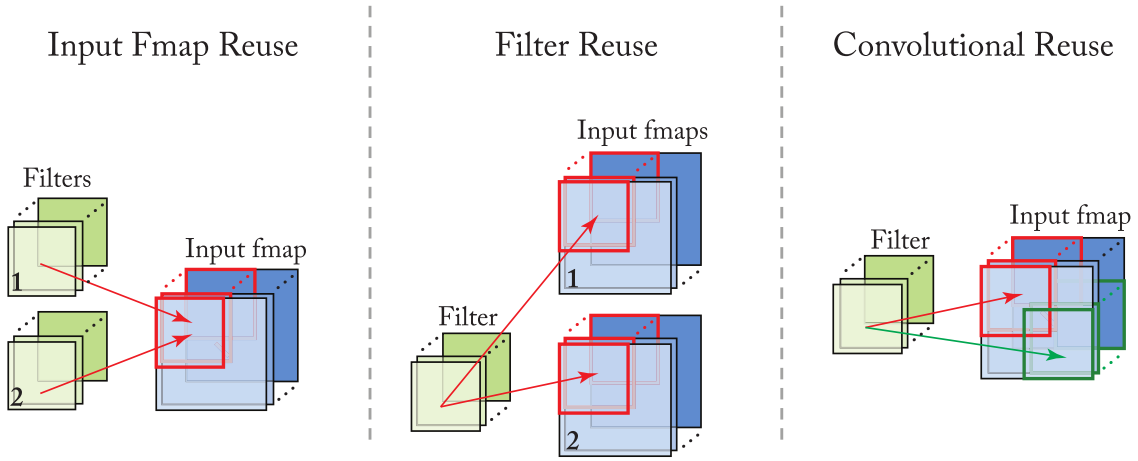


Figure 4.6: The three primary forms of data reuse in convolutional layers: convolutional, filter, and input reuse. [25].

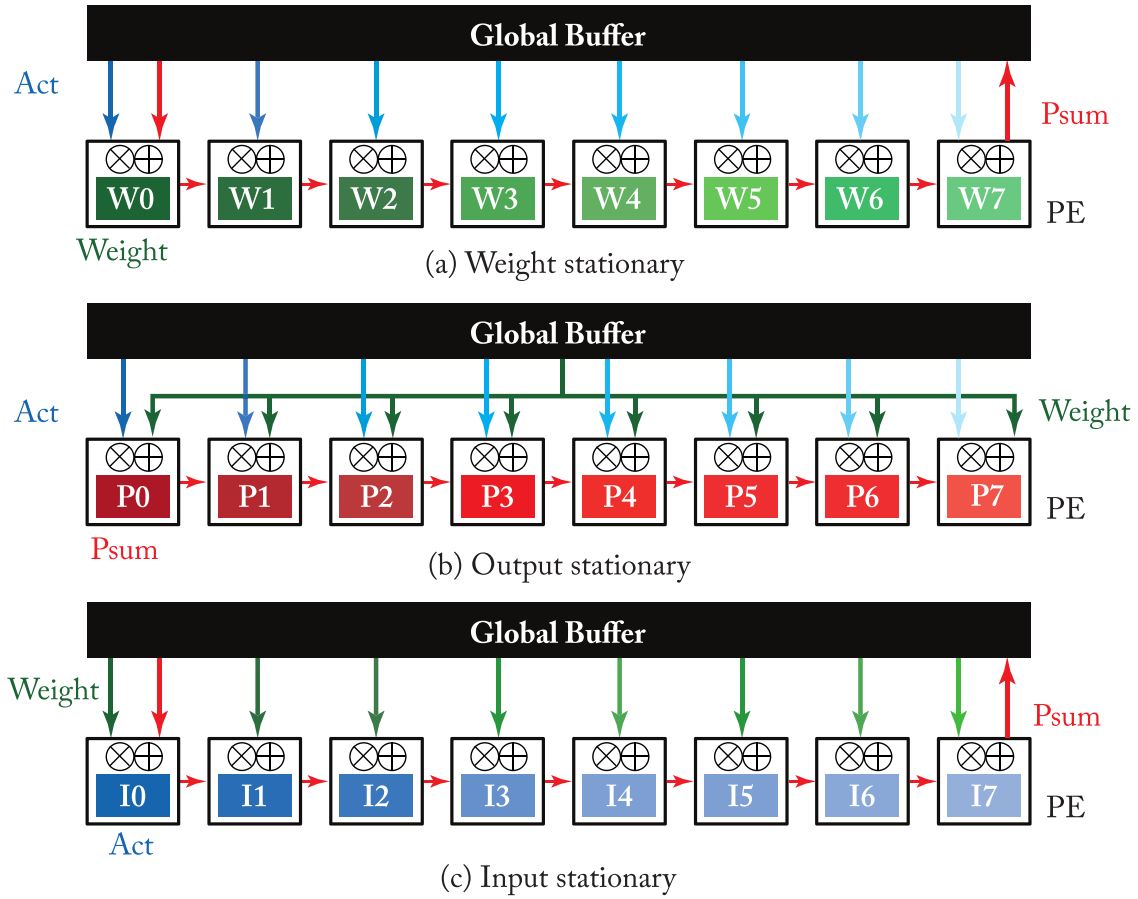


Figure 4.7: A visual comparison of Weight, Output and Input Stationary dataflows. [25].

4.5 The Gemmini Approach: A Full-Stack Research Platform

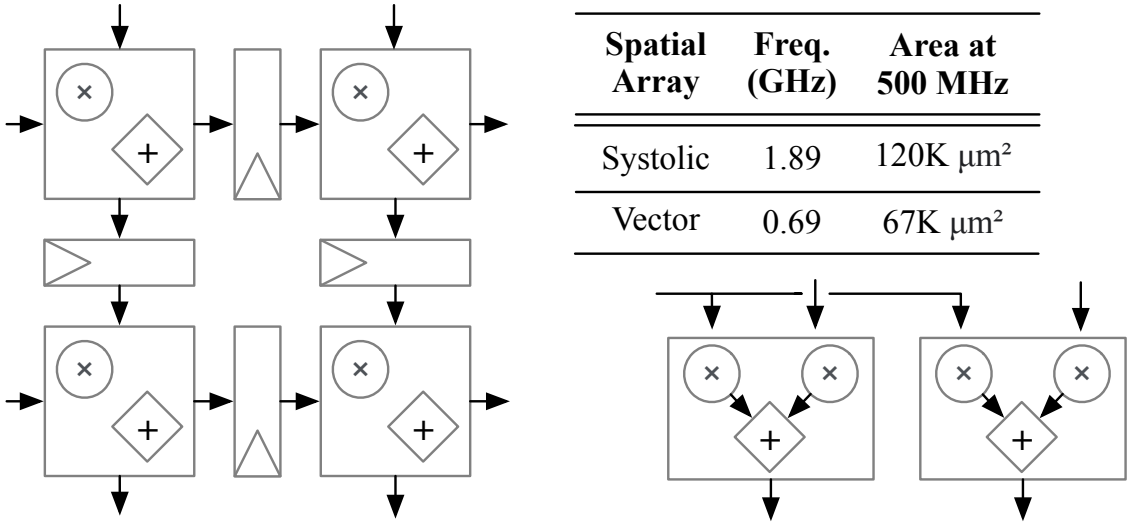
While the TPU demonstrates the immense potential of large-scale systolic arrays and the NVDLA showcases the value of modular, open-source IP, both present challenges for comprehensive, system-level academic research. The TPU is a proprietary, closed-source design, limiting deep architectural inspection. The NVDLA, while open, is often evaluated as an isolated hardware block, which can obscure the real-world performance bottlenecks that arise from interactions with the host CPU, operating system, and shared memory hierarchy [11].

To address these challenges, this thesis utilizes **Gemmini**, an open-source, *full-stack DNN accelerator generator* from the University of California, Berkeley [11]. Gemmini's design is founded on the philosophy that to truly understand an accelerator's performance, it must be evaluated as part of a complete, functioning system. It achieves this by leveraging the Chipyard framework to generate not only the accelerator RTL but also a Linux-capable SoC, enabling the study of critical cross-stack effects such as:

- **SoC Resource Contention:** Competition for shared resources like the L2 cache and the main memory bus between the CPU and the accelerator.
- **Operating System Overhead:** The performance impact of OS services, including context switches and page fault handling.
- **Virtual Memory Interaction:** The latency and performance implications of the accelerator's DMA engine interacting with the system's TLBs and Page Table Walker.

Crucially, as a generator written in Chisel, Gemmini is not a single, fixed hardware design. It is a highly flexible template that can be configured at generation time to produce a wide spectrum of systolic architectures. As illustrated in Figure 4.8, by tuning its parameters, Gemmini can generate designs ranging from highly-pipelined, TPU-like structures to more loosely-coupled, NVDLA-like parallel vector engines. This architectural flexibility makes it an exceptionally powerful platform for academic research and design-space exploration.

Therefore, for the objectives of this thesis—to implement and evaluate a custom accelerator within a realistic SoC environment—Gemmini provides the ideal platform. It combines the computational power of a systolic array with the verifiability of a full-stack, open-source framework, enabling a rigorous and holistic analysis.

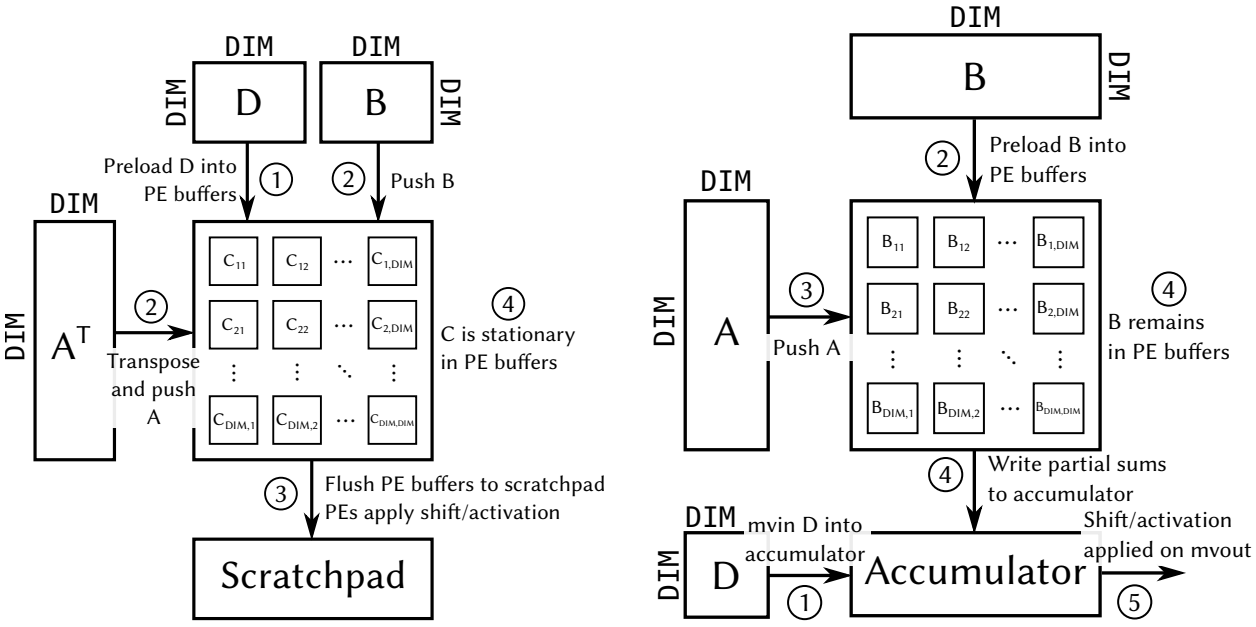


Spatial Array	Freq. (GHz)	Area at 500 MHz
Systolic	1.89	120K μm²
Vector	0.69	67K μm²

Systolic spatial array
(TPU-like)

Parallel vector engines
(NVDLA-like)

Figure 4.8: The design space enabled by the Gemmini generator, allowing it to produce a wide variety of spatial array structures, from systolic (TPU-like) to parallel vector engines (NVDLA-like). Source: [11].



(a) Output Stationary (OS)

(b) Weight Stationary (WS)

Figure 4.9: A visual comparison of the two primary dataflows supported by Gemmini. (a) In the **Output Stationary** dataflow, the result matrix C remains stationary in the PEs to accumulate partial sums. (b) In the **Weight Stationary** dataflow, the weight matrix B is preloaded and remains stationary in the PEs to maximize weight reuse. The choice between these strategies represents a fundamental trade-off in accelerator design. Source: [11].

4.6 Chapter Summary

This chapter has built a foundational argument for the necessity of specialized hardware for DNN acceleration. We began with the core architectural challenge, the Memory Wall, and introduced the systolic array as a classic and enduring solution that minimizes costly data movement by maximizing data reuse. By examining modern industrial accelerators like the Google TPU and NVDLA, we have seen this principle in action. Finally, we formalized these strategies with the concept of dataflows, such as Weight Stationary and Output Stationary. These foundational concepts provide the necessary framework to understand the design and operation of the Gemmini accelerator, which will be the focus of the subsequent chapters.

Chapter 5: The Gemmini DNN Accelerator: An Architectural Overview

As established in the previous chapter, the System-on-Chip (SoC) developed in this thesis utilizes a single Rocket Core augmented with a specialized accelerator for deep learning workloads. This chapter delves into the architecture of this accelerator: Gemmini, a sophisticated component designed to overcome the fundamental limitations of general-purpose processors for Deep Neural Network (DNN) tasks.

Figure 5.1: Gemmini Logo. [11].

5.1 Top-Level Architectural Blueprint

The Gemmini generator can produce a wide range of accelerator instances based on a flexible architectural template. To understand the Gemmini architecture, we first examine the high-level template shown in Figure 5.2. This blueprint illustrates the three primary components of the accelerator: the Controller, the on-chip memory (Scratchpad and Accumulator), and the compute core (Spatial Array), and their interface with the host CPU and system DRAM. The following sections provide a detailed analysis of each of these blocks.

5.2 Analysis of Core Architectural Components

5.2.1 The Systolic Compute Engine

The computational heart of Gemmini is its systolic array, a highly configurable spatial array that functions as a powerful General Matrix-Matrix Multiplication (GEMM) engine. The array's microarchitecture, as detailed in Figure 5.3, features a two-level hierarchy composed of Tiles and individual Processing Elements (PEs). Each PE is capable of performing a Multiply-Accumulate (MAC) operation in a single clock cycle. This hierarchical and configurable design allows Gemmini to efficiently execute the dense matrix multiplications fundamental to modern Deep Neural Networks (DNNs).

The key parameters of the systolic array can be specified at generation time:

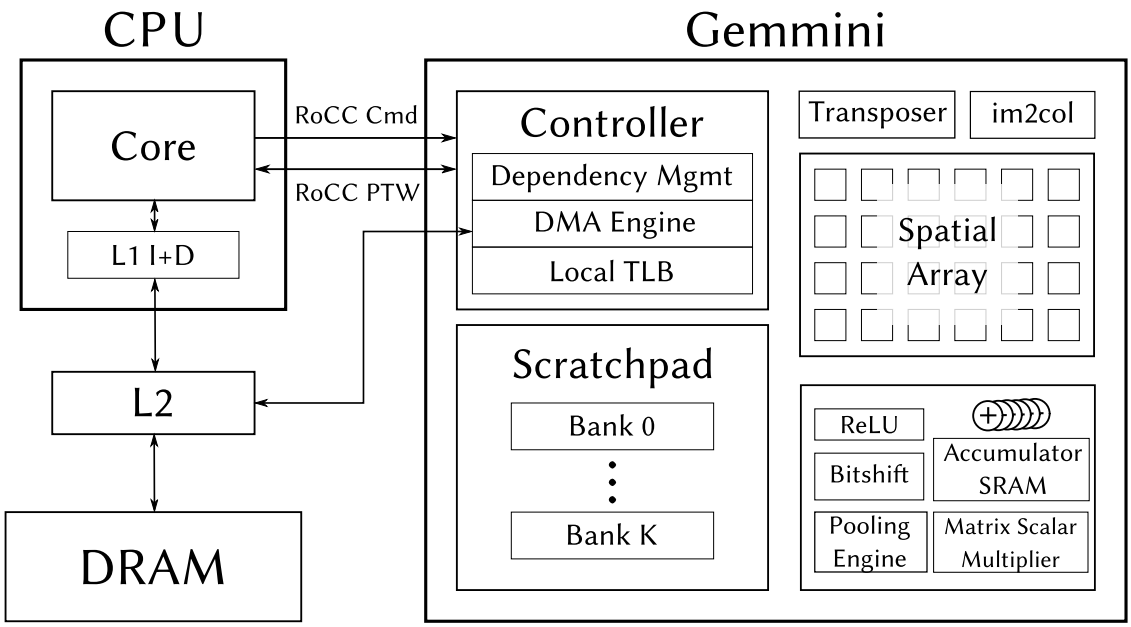


Figure 5.2: High-level overview of the Gemmini accelerator template integrated with a host CPU and system memory hierarchy. [11].

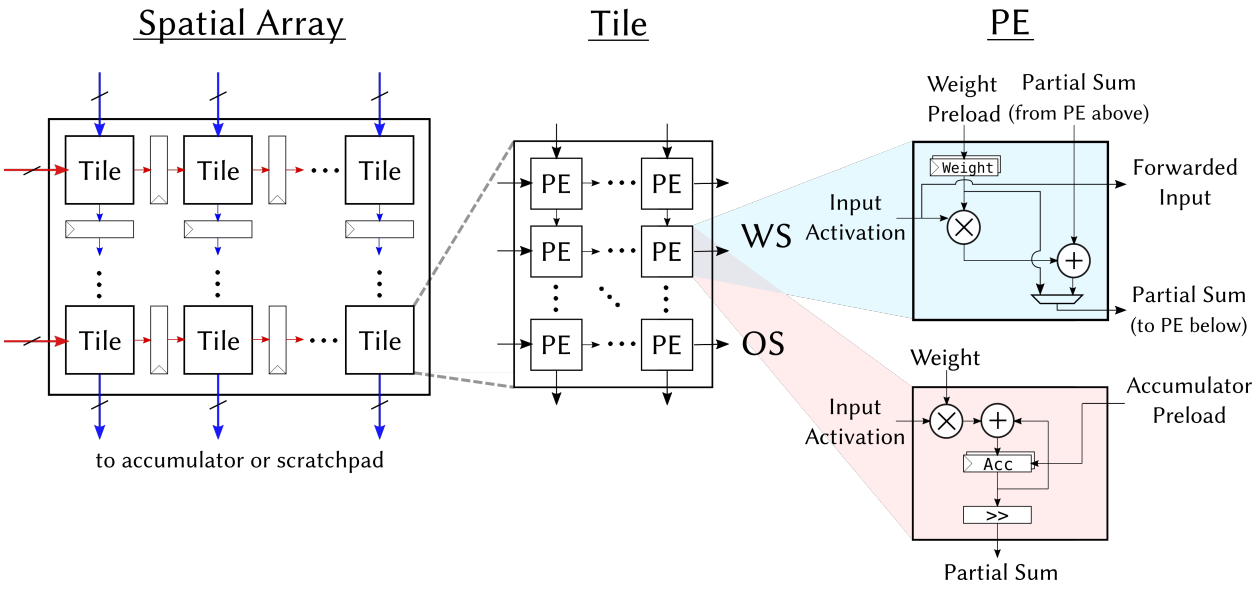


Figure 5.3: The two-level hierarchy of Gemmini's spatial array, composed of tiles and Processing Elements (PEs). [11].

- **Dimensions:** The size of the PE grid, structured into a hierarchy of tiles, can be configured (e.g., a 16x16 array composed of 2x2 tiles, each with 8x8 PEs). This allows for a trade-off between hardware area and parallel processing capability.

- **Dataflow:** Gemmini primarily supports two dataflow strategies: Weight Stationary (WS) and Output Stationary (OS).

  - In **WS dataflow**, the DNN weights are pre-loaded into the PEs and remain stationary, maximizing weight reuse as input activations are streamed through.
  - In **OS dataflow**, partial sums of the output activations remain stationary within the PEs, while both inputs and weights are streamed. This is particularly beneficial for layers with low weight reuse.

  The choice of dataflow has a significant impact on performance and energy efficiency.

- **Pipelining:** The degree of pipelining within the array and its PEs can be configured, affecting the maximum clock frequency and overall throughput.

5.2.1.1 Mapping Convolution to Hardware: The `im2col` Transformation

While systolic arrays are fundamentally designed to accelerate General Matrix-Matrix Multiplication (GEMM), they are adept at accelerating 2D convolutions through a crucial data-restructuring technique known as `im2col` ("image-to-column"). This process unrolls the overlapping input image patches from a convolutional layer into the columns of a new, larger matrix. As shown in Figure 5.4, this new matrix can then be multiplied with a corresponding weight matrix, effectively **converting the convolution into a GEMM operation** that the systolic array can process with maximum efficiency.

Recognizing the computational cost of this pre-processing step, Gemmini can be configured with a dedicated hardware unit to perform the `im2col` transformation on-the-fly. This offloads the data restructuring from the host CPU, freeing it for other tasks and streamlining the overall computation pipeline. The decision to include this unit represents a classic area-versus-performance trade-off in the accelerator design.

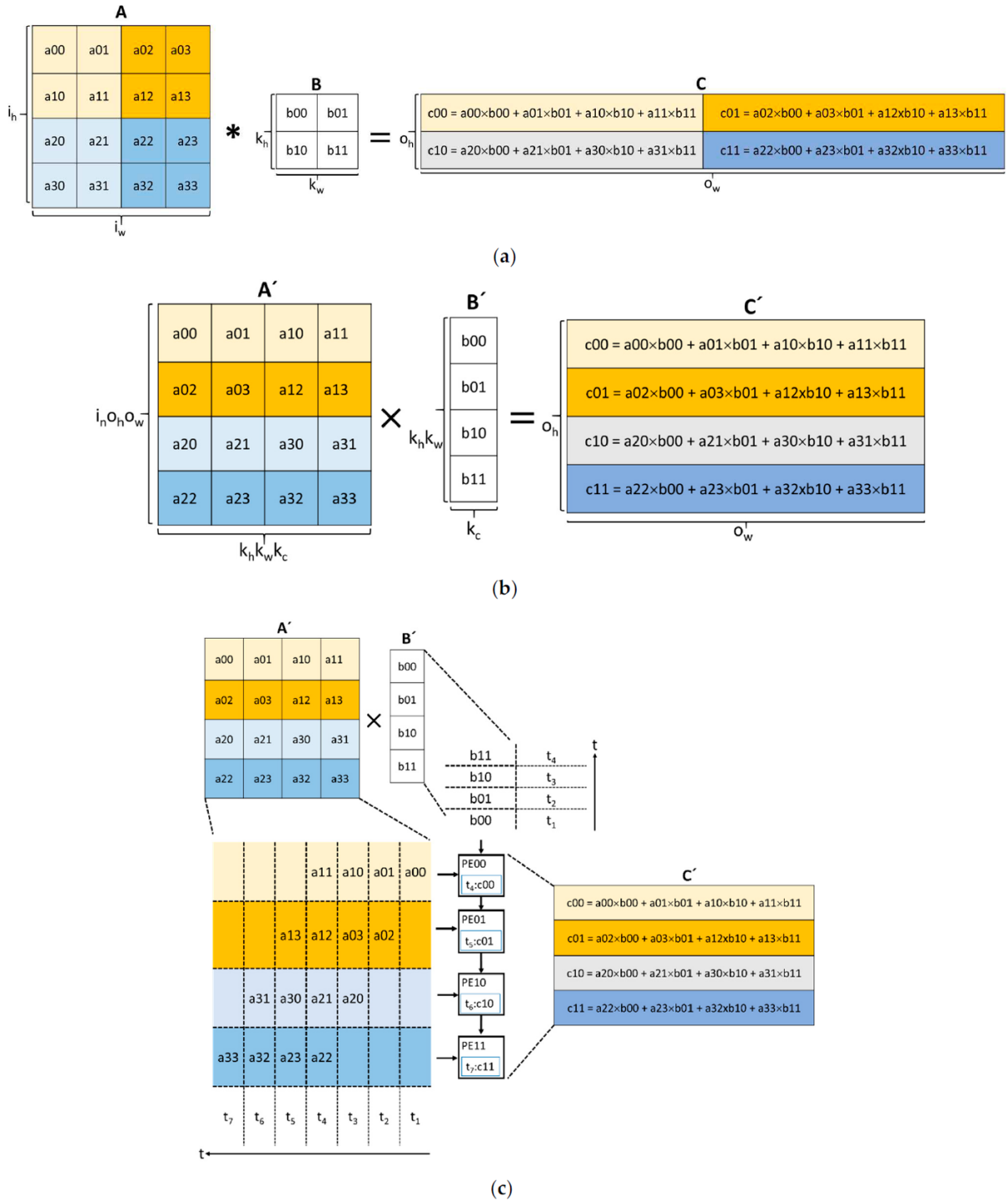


Figure 5.4: From convolution to systolic array computation: (a) direct convolution operation; (c) example of a systolic array operation; (b) A visualization of the `im2col`-based GEMM operation. The 3D input tensor (left) is transformed into a 2D matrix (center) that can be processed by the systolic array. Adapted from Gookyi et al. [12].

5.2.2 The On-Chip Memory Subsystem

To feed its high-throughput systolic array, Gemmini includes a specialized on-chip memory system. This design represents a deliberate trade-off between hardware-

managed caches and software-managed memories.

- **Scratchpad Memory:** A software-managed SRAM used for staging input activations, weights, and intermediate feature maps close to the systolic array. Its purpose is to reduce the latency and energy consumption associated with fetching data from higher levels of the memory hierarchy, such as the shared L2 cache or main memory (DRAM). Its capacity and banking are configurable.

- **Accumulator Memory:** Separate from the main scratchpad, Gemmini features a dedicated memory to store the partial sums generated by the MAC operations. This memory typically uses a higher precision data type (e.g., 32-bit integers) than the input data (e.g., 8-bit integers) to maintain accuracy during accumulation.

5.3 SoC Integration and System-Level Features

5.3.1 The RoCC Coprocessor Interface

Gemmini integrates into the Chipyard SoC via the **Rocket Custom Coprocessor (RoCC)** interface (Section 3.4). As a tightly-coupled coprocessor, it receives custom RISC-V instructions dispatched directly by the host Rocket Core's pipeline. This allows for low-latency command and control.

This integration uses a decoupled access-execute model, as illustrated conceptually in Figure 5.5. The host CPU issues three main types of commands to Gemmini: load (from main memory to scratchpad), execute (perform computation on data within the scratchpad), and store (from scratchpad to main memory). These commands are placed into separate hardware queues, allowing Gemmini's internal controller then processes these commands asynchronously, and Gemmini's DMA engine to handle data movement in parallel with the systolic array's computations. This overlapping of communication and computation is crucial for hiding memory latency and maximizing the utilization of the systolic array.

5.3.2 Virtual Memory Support

To operate correctly under a full-featured operating system like Linux, an accelerator's DMA engine must be able to translate virtual memory addresses into physical addresses. Gemmini is designed with this "full-stack" capability. It can be configured

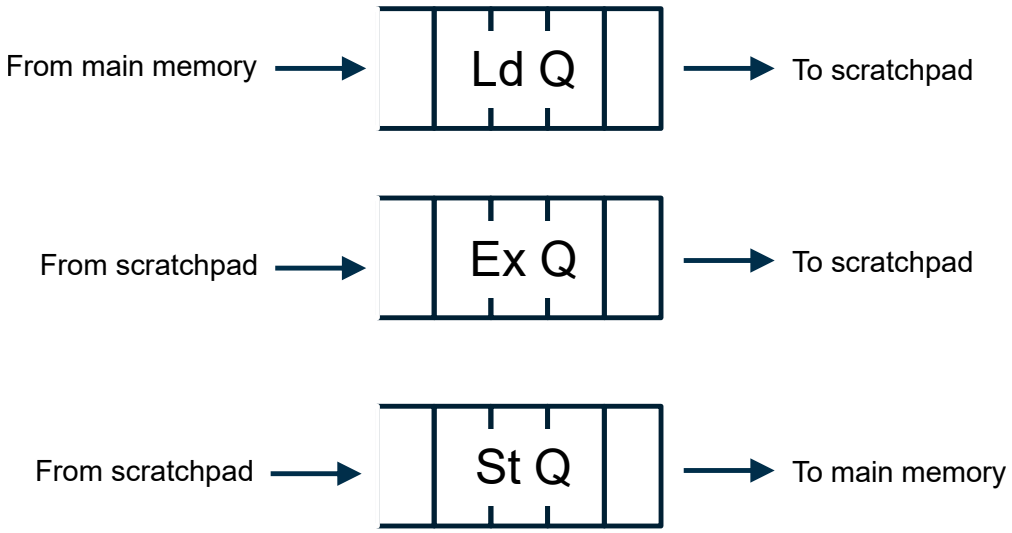


Figure 5.5: Conceptual view of Gemmini's decoupled access-execute pipelines, managed by separate hardware command queues for Load, Execute, and Store operations. [11].

to share the host CPU's Page Table Walker (PTW) to perform its own address translations, and it includes its own local Translation Lookaside Buffer (TLB) to cache these translations and reduce latency.

5.4 The Multi-Level Programming Model

To cater to different user needs, from application developers to hardware architects, Gemmini supports a multi-level programming model. This stack provides layers of abstraction, allowing users to work at a level appropriate for their task, from direct hardware control to high-level framework integration.

5.4.1 Low-Level: Gemmini ISA

At the lowest level, Gemmini is controlled by its custom RoCC instruction set. These instructions are typically wrapped in C preprocessor macros for easier use from C/C++ code. The instruction set includes primitive commands like `mvin` (move data in), `mvout` (move data out), `preload` (load data into the PE array), and `compute`. It also features more "complex" instructions like `loop_matmul` that abstract away the manual tiling and scheduling of primitive commands, offloading the loop control to hardware state machines [10].

5.4.2 Mid-Level: Hand-Tuned C Library

To abstract the complexities of direct ISA programming, Gemmini provides a C library (typically accessed via `gemmini.h`). This library offers functions for common DNN operations, such as `tiled_matmul` and `tiled_conv`. These functions implement optimized heuristics for loop tiling and data movement to maximize scratchpad utilization based on the specific hardware parameters of the generated Gemmini instance.

5.4.3 High-Level: Compiler Support (Conceptual)

The Gemmini ecosystem is designed to support higher-level compilation flows. This involves tools that can take DNN models described in standard frameworks like ONNX (Open Neural Network Exchange) and compile them down to Gemmini instructions, either by calling the mid-level C library or by directly generating low-level ISA calls. This further simplifies the deployment of DNNs on Gemmini-accelerated SoCs.

5.5 Summary

In summary, Gemmini is a sophisticated, highly configurable DNN accelerator generator that embodies the principles of systolic computation. Its key architectural features include a powerful systolic array, a specialized on-chip memory hierarchy, and a tightly-coupled integration with a host processor via the `RoCC` interface, including support for system-level features like virtual memory. This design enables the high-performance execution of DNN workloads within a full-stack, system-level context.

Having established this architectural overview, the following chapter will now perform a deep dive into the specifics of Gemmini's internal command processing and its multi-level programming model.

Chapter 6: Gemmini's Internal Architecture and Programming Model

While Chapter 5 described the conceptual architecture of Gemmini, this chapter explores its internal structure and the software interfaces used to control it. This analysis will follow the journey of a single instruction from the host CPU down to the physical hardware, revealing how Gemmini's sophisticated design translates high-level commands into high-performance computation. A deeper understanding of these implementation-level details is necessary to effectively program the accelerator and analyze its performance.

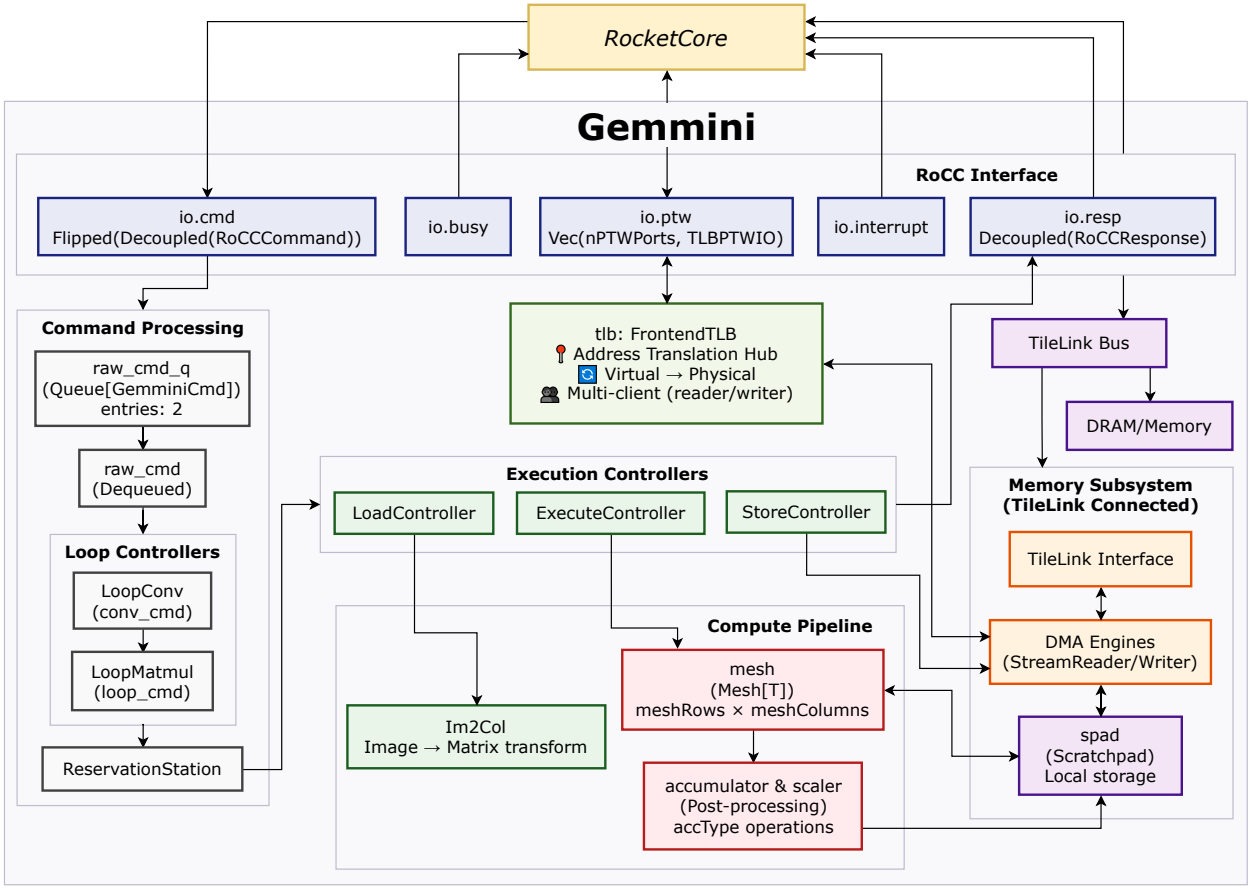


Figure 6.1: Overview of the internal modules in the generated Gemmini hardware architecture, showing the flow of commands from the RoCC interface through various queues and controllers.

Figure 6.1 serves as a high-level architectural blueprint for the entire Gemmini accelerator. The journey of a command begins when a `RoCCCommand` is dispatched from the `RocketCore` and arrives at the **RoCC Interface**. The command first enters the **Command Processing** block, where it is enqueued before being passed to the **Loop Controllers**. Here, the two-level command-unrolling hierarchy (`LoopConv`

and LoopMatmul) autonomously translates the high-level instruction into a stream of simpler, tiled micro-operations. These primitive commands are then staged in the **ReservationStation**, which acts as a central dispatch hub.

From the Reservation Station, commands are issued to the three independent **Execution Controllers**, which manage the core decoupled operations of the accelerator. The LoadController and StoreController orchestrate all data movement with the **Memory Subsystem**, issuing requests through its TileLink Interface to either the on-chip spad (Scratchpad) or off-chip DRAM. In parallel, the ExecuteController drives the **Compute Pipeline**, managing the optional Im2Col unit and issuing commands to the systolic mesh and the post-processing accumulator & scaler. This entire process is supported by system-level services, such as the **FrontendTLB**, which handles virtual memory address translation via the io.ptw port.

This decoupled, multi-stage architecture is the key to Gemmini's performance, allowing long-latency memory operations to be overlapped with high-throughput computation. The subsequent sections will now explore each of these major blocks in detail.

6.1 The Command-Unrolling Hierarchy

A custom RoCC instruction sent from the Rocket core is not a simple command executed directly by the systolic array. Instead, it is a high-level, almost CISC-like directive, such as LOOP_WS or LOOP_CONV_WS, that can encapsulate an entire deep learning layer computation. The fundamental design principle of Gemmini is to offload the immense complexity of interpreting this command from the CPU into a dedicated, multi-level hardware hierarchy.

This hardware-based command unrolling is a defining characteristic of a powerful, tightly-integrated co-processor. It autonomously handles tasks that would otherwise require thousands of CPU cycles—such as loop control, address calculation, and data dependency checking—freeing the host processor for other work. This section details the two primary levels of this unrolling hierarchy: LoopConv and LoopMatmul.

6.1.1 Level 1: Convolution Decomposition (LoopConv)

The first stage in processing a high-level command is the LoopConv module (Figure 6.2), which functions as a Convolution Loop Unroller. Convolution operations involve complex, nested loops over batch dimensions (N), input/output channels (C),

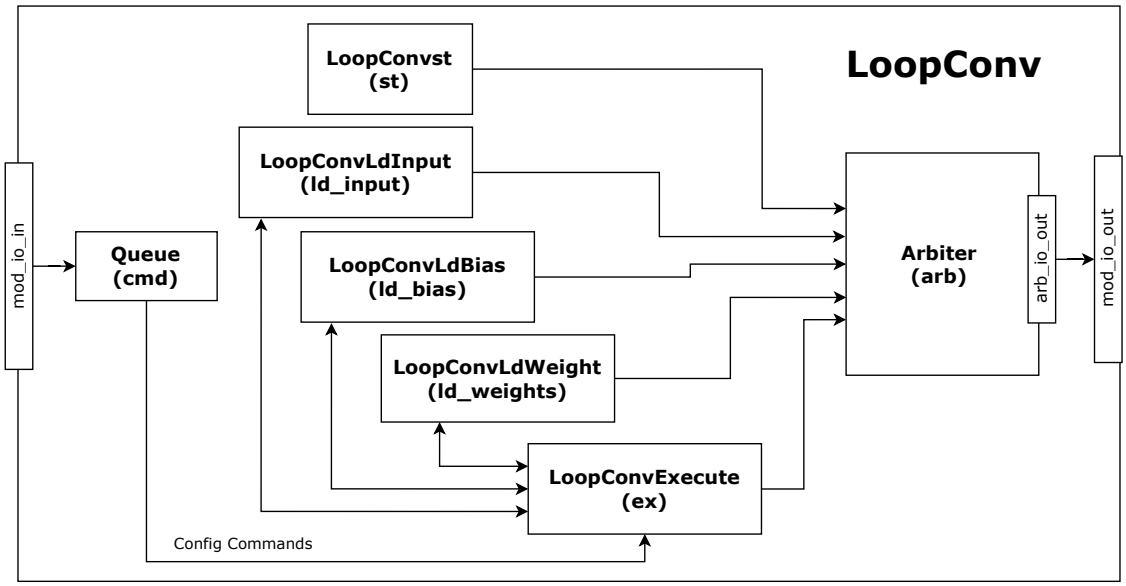


Figure 6.2: The `LoopConv` module. This is the first level of the command-unrolling hierarchy, responsible for decomposing a complex convolution operation into a series of matrix multiplication tasks.

spatial dimensions (H, W), and kernel dimensions (R, S). Without hardware offloading, a programmer would need to manage this 6 to 7-dimensional loop nest, along with data tiling and resource allocation, in software.

The `LoopConv` module automates this entire process. It interprets a single high-level instruction (e.g., `gemmini_loop_conv_ws`) and decomposes the complex convolution into a sequence of simpler matrix multiplication tasks. Its internal architecture is comprised of several parallel hardware units that handle the two distinct phases of the operation: data loading and execution.

First, the data loading phase is handled by three independent, concurrent units:

- `LoopConvLdInput`: Loads input feature maps from memory.
- `LoopConvLdWeight`: Loads convolution kernel weights from memory.
- `LoopConvLdBias`: Loads bias values from memory.

Once the required data has been fetched, the execution phase begins, managed by two final components:

- `LoopConvExecute`: Executes the convolution operation by passing tasks to the `LoopMatmul` module.
- `Arbiter`: A 5-to-1 arbiter that manages access to the memory interface, ensuring that the parallel load and execute units do not create resource contention.

These generated tasks are then passed down to the next level of the command-unrolling hierarchy.

6.1.2 Level 2: Tiled Matrix Multiplication (`LoopMatmul`)

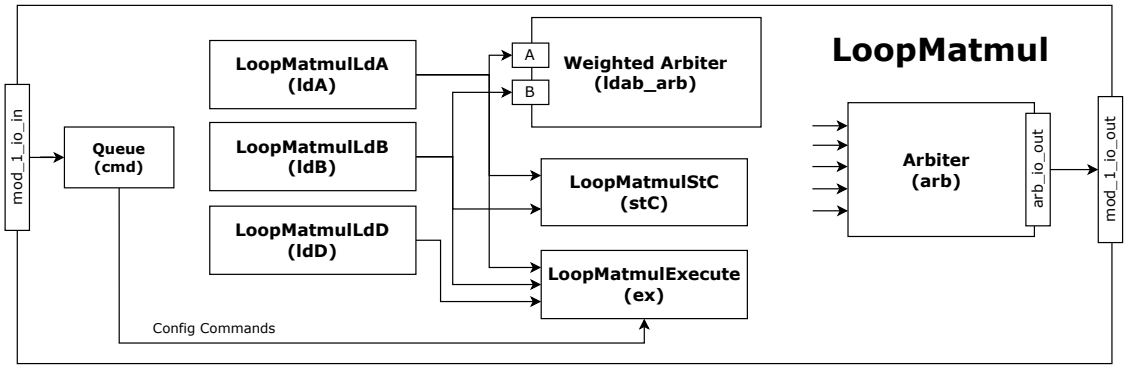


Figure 6.3: The `LoopMatmul` module. This second-level unroller translates matrix multiplication tasks into a stream of smaller, tiled micro-operations for the physical hardware.

The `LoopMatmul` module (Figure 6.3) receives the sequence of matrix multiplication jobs from `LoopConv` and performs the second level of command unrolling. Its primary responsibility is to break these matrix-level operations down into smaller, fixed-size tiles that perfectly match the physical dimensions of the systolic array. This module is effectively a micro-code engine that processes five distinct types of operations—ranging from configuration (`CONFIG_CMD`) and memory transfers (`LOAD3_CMD`, `STORE_CMD`) to the core computational directives (`PRELOAD_CMD`, `COMPUTE_CMD`). This section delves into the internal architecture and policies that allow it to manage this complexity.

6.1.2.1 Internal Architecture and Decoupled Execution

The internal structure of the `LoopMatmul` controller is composed of several independent hardware modules, each responsible for a specific part of the execution pipeline:

- `LoopMatmulLdA`: Loads matrix A from DRAM to scratchpad memory.
- `LoopMatmulLdB`: Loads matrix B (weights) from DRAM to scratchpad memory.
- `LoopMatmulLdD`: Loads bias/accumulator data from DRAM to accumulator memory.

- `LoopMatmulExecute`: Performs the actual matrix multiplication computation.
- `LoopMatmulStC`: Stores results from the accumulator back to DRAM.
- `LoopMatmulStCSpad`: Stores results from the accumulator to scratchpad memory.

While these modules operate independently, they are coordinated through a lightweight handshake mechanism referred to in the design as iterator synchronization. For instance, the `LoopMatmulExecute` unit will not begin computation on a tile of data until it receives a signal that both `LoopMatmulLdA` and `LoopMatmulLdB` have successfully loaded the required operands into the scratchpad. This allows the `Load` modules to work ahead, fetching data for a future computation (`tile i+1`) while the `Execute` module is still busy processing the current one (`tile i`).

This decoupled architecture is a deliberate and critical design choice for overcoming the "memory wall". The high latency of fetching data from off-chip DRAM is a primary performance bottleneck in any accelerator. By overlapping the long-latency memory operations (Load and Store) with the core computation (Execute), the system effectively hides this latency. This ensures that the systolic array—the most valuable computational resource—is starved for data as infrequently as possible, maximizing its utilization and achieving a throughput which would be unattainable with a simple, rigid pipeline.

6.1.2.2 Resource Management and Arbitration Priority

The parallel operation of these internal modules inevitably leads to contention for shared resources. To manage this, the `LoopMatmul` controller employs a hardware arbiter that enforces a fixed-priority scheduling policy. This policy is not arbitrary; it is carefully designed to maintain pipeline flow and prevent deadlocks. The priority scheme is as follows:

1. **Highest Priority - Store to DRAM**: Commands from the `LoopMatmulStC` module receive top priority. This is essential to prevent accumulator buffer overflow and free up internal resources for subsequent computations.
2. **Execute Operations**: The `LoopMatmulExecute` module's preload and compute commands receive second priority to keep the systolic array actively computing.

3. **Load Bias/Accumulator**: The `LoopMatmulLdD` module's commands for loading bias data receive third priority.

4. **Load Input Matrices**: A weighted sub-arbiter manages requests from `LoopMatmulLdA` and `LoopMatmulLdB` modules with fourth priority.

5. **Lowest Priority - Store to Scratchpad**: The `LoopMatmulStCSpad` module receives the lowest priority, as scratchpad stores are internal operations that are less critical to overall pipeline progression than DRAM transfers.

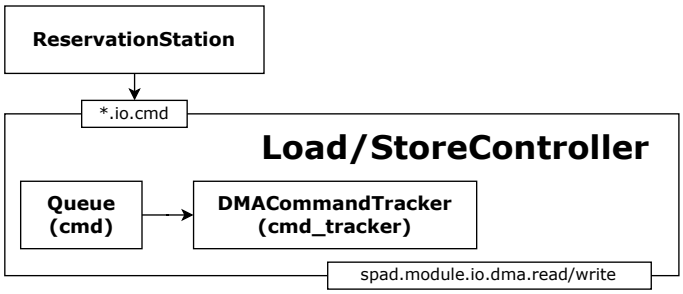
This priority scheme ensures that result data is promptly written to DRAM to free accumulator resources, while maintaining sufficient data throughput to keep the computational units busy. The impressive result of this entire hierarchical process is that the `LoopMatmul` module transforms a single high-level command, like `LOOP_WS(matmul 1024x1024)`, into **thousands** of precisely calculated, optimally ordered, resource-aware hardware operations ready for immediate execution by the back-end controllers.

6.2 Decoupled Execution and Resource Management

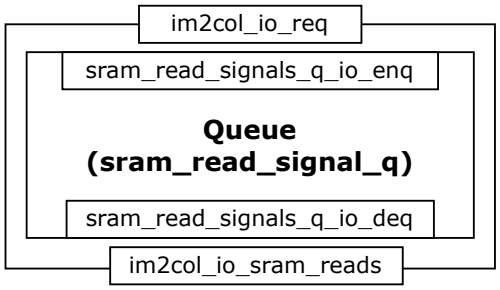
The stream of primitive micro-operations generated by the command-unrolling hierarchy is not fed into a rigid, lock-step pipeline. Instead, Gemmini employs a more sophisticated and powerful paradigm: decoupled execution. This design allows the various stages of a complex operation—loading data, executing computation, and storing results—to proceed concurrently and asynchronously, managed by a central dispatch hub and a set of independent back-end controllers.

6.2.1 The Reservation Station: A Central Dispatch Hub

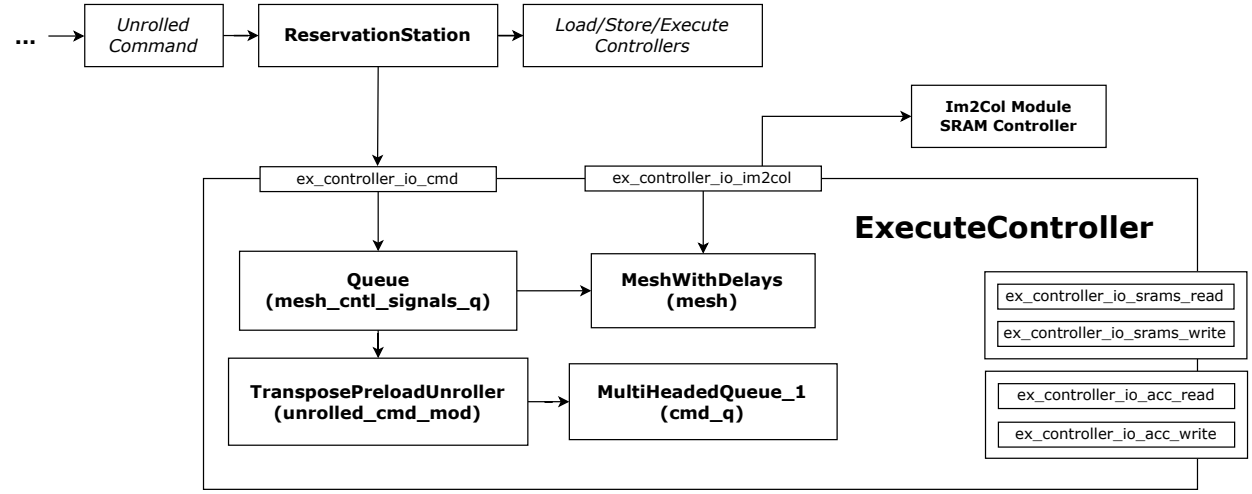
At the heart of the decoupled back-end is the Reservation Station. It receives a stream of commands from the `LoopMatmul` module that are far more than simple instructions; they are complete, resource-aware directives. These directives include optimally sequenced commands, complete address calculations for DRAM and scratchpad, resource-aware flow control to prevent buffer overflows, and attribution metadata for debugging. The Reservation Station's primary role is to act as a central orchestrator, managing data dependencies and dispatching these directives to the appropriate hardware units.



(a) The Load and Store Controllers manage data movement between scratchpad and DRAM via a DMA tracker.



(b) Im2Col module (an SRAM Controller)



(c) The Execute Controller, which issues commands to the systolic array (`Mesh`) and `im2col` units.

Figure 6.4: The primary back-end controllers that receive commands from the Reservation Station.

This enables a form of out-of-order execution, where a command can be dispatched as soon as its required data and hardware resources are available. The primitive commands it dispatches correspond to the fundamental actions of the accelerator pipeline, including loading data (`LOAD_CMD`, `LOAD2_CMD`, `LOAD3_CMD`), preloading weights into the array (`PRELOAD_CMD`), initiating the core computation (`COMPUTE_AND_FLIP_CMD`), and writing results back to memory (`STORE_CMD`).

6.2.2 Back-End Controllers: Load, Store, and Execute

From the Reservation Station, primitive commands are dispatched to one of three specialized, independently operating back-end controllers.

- `LoadController`: Handles all data loading operations, fetching data from DRAM to the scratchpad memory.

- `StoreController`: Manages data storage operations, writing results from the scratchpad back to DRAM.
- `ExecuteController`: Orchestrates the actual computation, issuing commands to the systolic array and managing on-the-fly data transformations.

The `LoadController` and `StoreController` (Figure 6.4a) are focused exclusively on managing the high-latency operations of the memory hierarchy. The `Execute Controller` (Figure 6.4c), however, contains a sophisticated collection of its own internal hardware modules, including the systolic array mesh itself, an `im2col` unit (Figure 6.4b), and various queues for command buffering. (A detailed breakdown of the `ExecuteController`'s internal sub-modules is provided in Appendix B.1).

This clear division of labor, orchestrated by the Reservation Station, is the cornerstone of the accelerator's performance. It allows long-latency memory operations and intense computation to overlap, effectively hiding the "memory wall" and ensuring the valuable systolic array is kept maximally utilized.

6.3 The Physical Hardware: Compute and Memory

The control-flow mechanisms described above orchestrate the work performed by the two foundational hardware components of the accelerator: the compute engine that performs the mathematical operations, and the memory subsystem that feeds it.

6.3.1 The Processing Element (PE) and Distributed Arithmetic

The fundamental building block of Gemmini's compute engine is the **Processing Element (PE)**. Each PE is a self-contained unit capable of performing a Multiply-Accumulate (MAC) operation, which is the core computation of a neural network. As shown in Figure 6.5, the PE is designed with the flexibility to support different dataflow strategies, such as Weight Stationary (WS) and Output Stationary (OS), allowing it to be optimized for different layer types.

A critical design choice in Gemmini is its approach to floating-point arithmetic. Gemmini avoids using a traditional, centralized Floating Point Unit (FPU). Instead, it implements a **distributed floating-point architecture** where arithmetic capabilities are embedded within every single PE. This is achieved through a powerful abstraction layer known as the `Arithmetic.scala` typeclass, which provides a full suite of IEEE 754-compliant operations by leveraging the Berkeley HardFloat library.

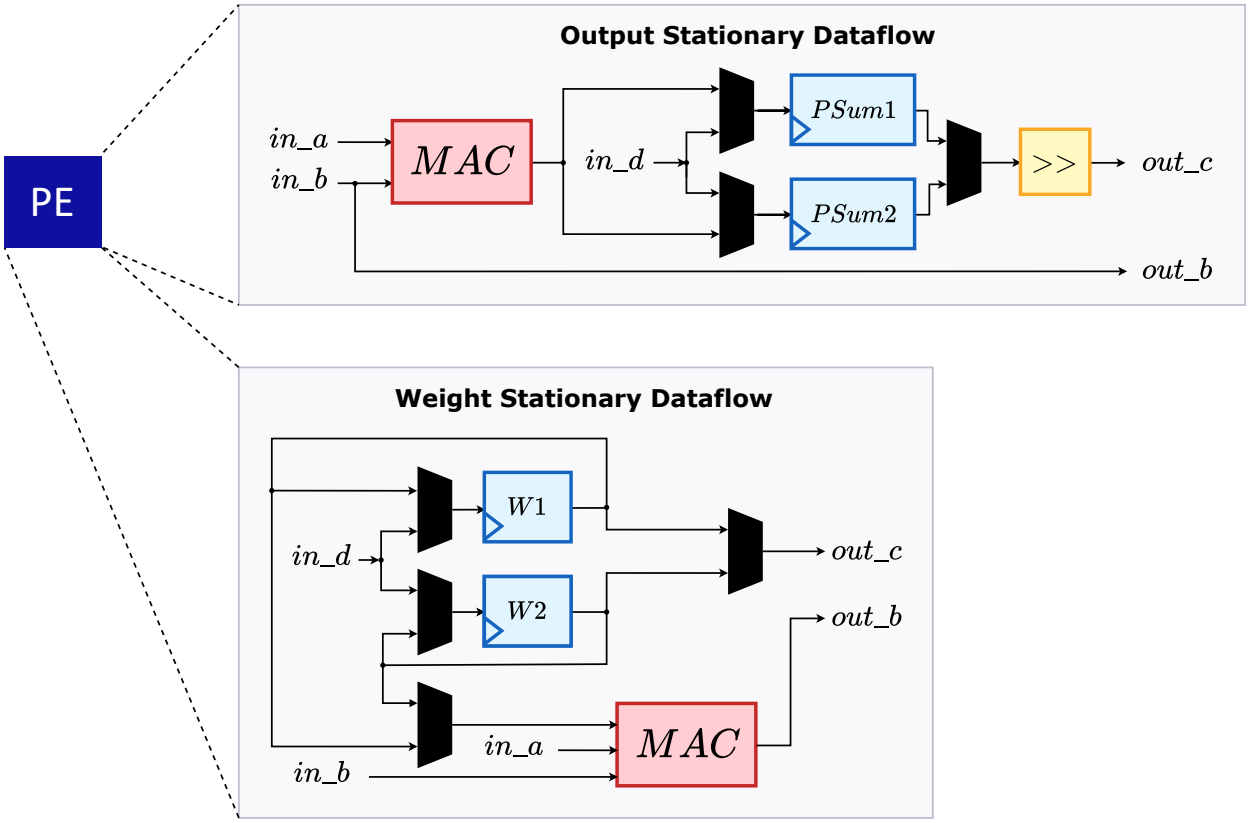


Figure 6.5: A single Processing Element (PE), the fundamental compute unit. It contains a Multiply-Accumulate (MAC) unit and the necessary logic to support multiple dataflows, such as Output Stationary and Weight Stationary.

The MAC unit at the heart of each PE (Figure 6.6) utilizes this typeclass, effectively making every PE a specialized FPU. This design transforms the systolic array into a massively parallel floating-point computation engine, tailor-made for the demands of modern DNNs.

6.3.2 The Tile and The Mesh: A Tale of Two Hierarchies

Individual PEs are not placed directly into a single large array. Instead, they are organized into a two-level hierarchy of **Tiles** and a **Mesh** (Figure 6.7). The design decision of how registers are used at each level is central to the entire accelerator's performance.

6.3.2.1 The Tile: A Combinational Block of PEs

A Tile is a two-dimensional array of PEs. The defining characteristic of a Tile is that it is **purely combinational**, meaning there are no pipeline registers between

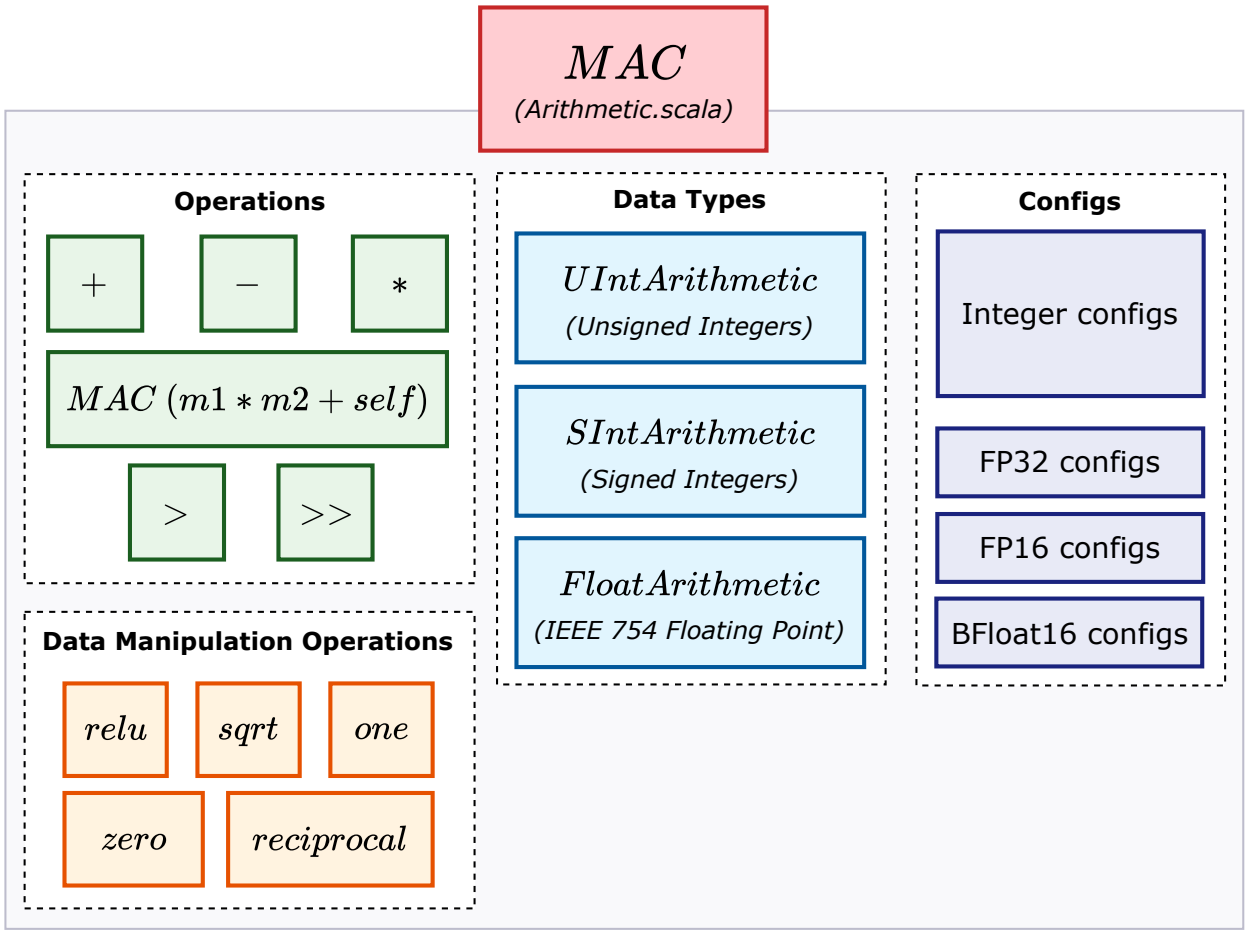


Figure 6.6: The Multiply-Accumulate (MAC) unit architecture. It is implemented using the `Arithmetic.scala` typeclass, which provides a suite of operations and supports multiple data types, including integers and IEEE 754 floating-point numbers.

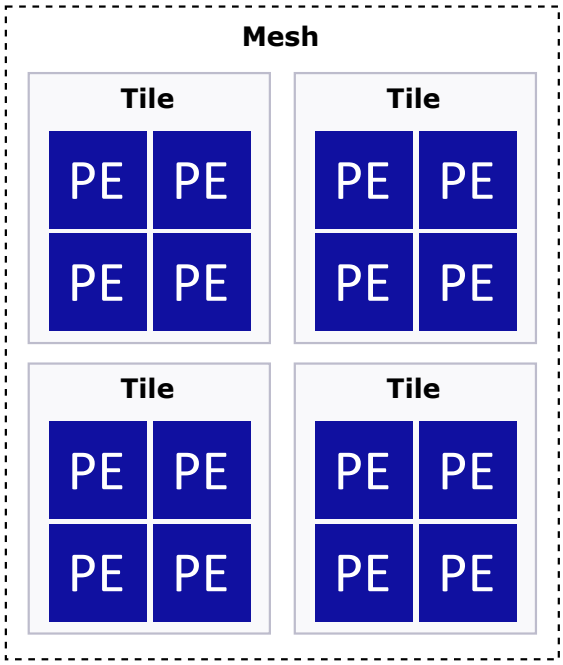
the PEs within a single Tile. This design choice, evident in the hardware generation code where PE inputs are connected directly, is made to minimize local latency and reduce the area and power overhead of registers. Grouping PEs into a combinational Tile allows for efficient broadcasting of input values to multiple PEs, promoting local data reuse.

6.3.2.2 The Mesh: A Pipelined Systolic Array of Tiles

The Mesh is a two-dimensional array of Tiles. Unlike the combinational logic within a Tile, the Mesh is **explicitly pipelined**, with shift registers placed between each Tile. This is the key that creates the systolic behavior of the entire array. These registers create the "systolic heartbeat," allowing waves of data to flow across the array in a controlled, rhythmic fashion. This pipelining between Tiles is what enables high throughput and scalability, as it breaks up long combinational paths and

allows every Tile in the array to be actively computing on a different piece of data simultaneously.

(a) A 4x4 mesh of 1x1-PE tiles. This configuration has 16 tiles and a deep pipeline, favoring a higher clock frequency.



(b) A 2x2 mesh of 2x2-PE tiles. This configuration has only 4 tiles and a shallower pipeline, favoring lower latency at the cost of clock speed.

Figure 6.7: Different Mesh configurations, illustrating the trade-off between the number of tiles and the number of PEs per tile, while maintaining the same total number of 16 PEs.

6.3.3 Architectural Trade-offs in Mesh Configuration

The Gemmini generator allows for configuring the hierarchy of the Mesh, offering a classic architectural trade-off between latency and clock frequency. For a system with a fixed number of PEs (e.g., 256), one can choose between having many small tiles or a few large tiles, as shown in Figure 6.7.

- **Many Small Tiles (e.g., a 16x16 mesh of 1x1-PE tiles):** This configuration maximizes the number of pipeline stages. The combinational logic path within each stage is very short (just a single PE), which allows for a **higher maximum clock frequency**. However, it takes more clock cycles for a sin-

gle piece of data to traverse the entire array, resulting in **higher pipeline latency**. This configuration is pipeline-optimized.

- **Few Large Tiles (e.g., a 4x4 mesh of 4x4-PE tiles):** This configuration has fewer, deeper pipeline stages. The combinational path within each stage is much longer (passing through multiple PEs), which limits the **maximum clock frequency**. However, an operation can pass through the entire array in fewer clock cycles, leading to **lower pipeline latency**. This configuration is throughput-optimized for batch processing.

The default configurations for Gemmini typically favor smaller (1x1) tiles, representing a design sweet spot that prioritizes high-frequency operation, which is often easier to implement and validate in a final ASIC or FPGA design.

6.3.4 The Scratchpad: An Intelligent Memory Subsystem

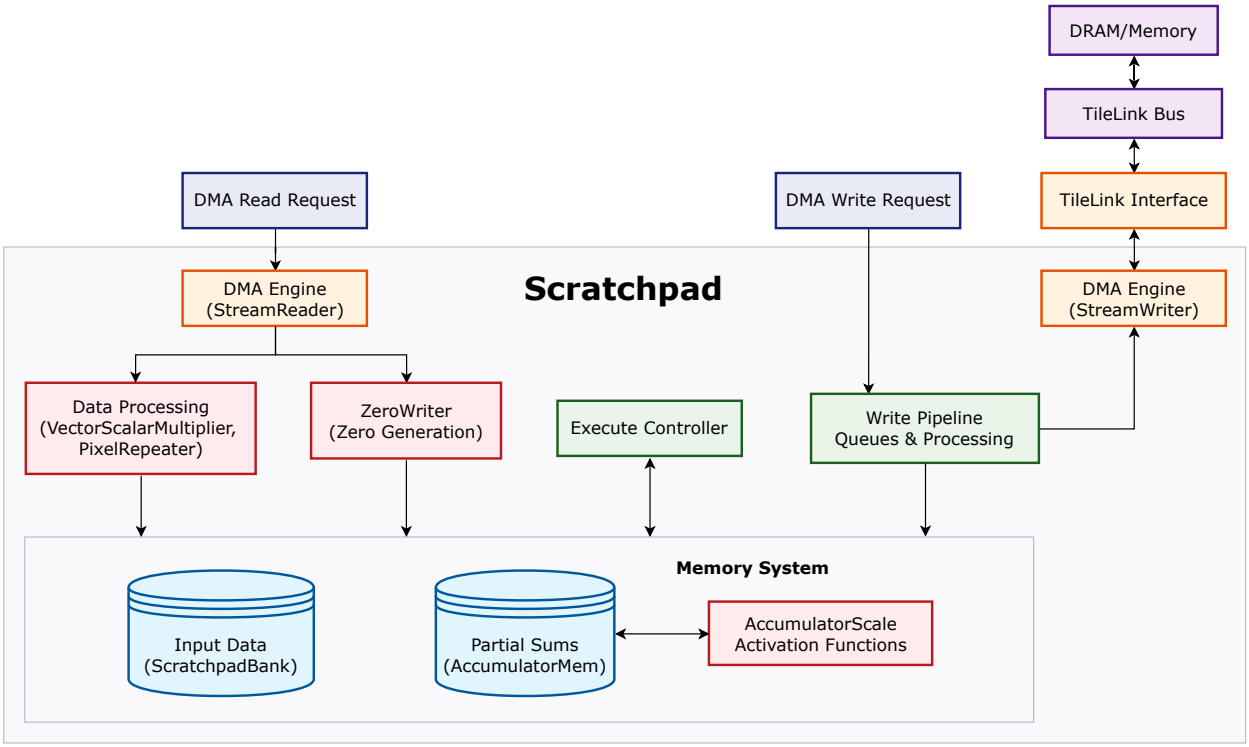


Figure 6.8: The internal structure of the Scratchpad module, an intelligent memory subsystem with integrated DMA engines and data processing units that serves the systolic array.

The Scratchpad (Figure 6.8) implements Gemmini's on-chip memory system, serving as the central data hub that bridges the gap between high-latency external DRAM

and the high-throughput systolic array. Far more than simple storage, the Scratchpad is a sophisticated data movement engine with integrated processing capabilities, dual data paths, and complex arbitration logic, enabling the high-throughput, low-latency data access patterns essential for efficient matrix computation. Its design demonstrates the critical importance of specialized memory subsystems in achieving peak performance in domain-specific accelerators.

The Scratchpad exposes two primary interfaces to the rest of the system: an **External Memory Interface** that connects to DRAM via the TileLink bus and DMA engines, and a **Compute Unit Interface** that provides direct, low-latency access for the systolic array.

Internally, its architecture is founded on a set of specialized **Memory Banks**. Multiple `ScratchpadBank`s store input data like weights and activations, with their banked structure enabling the parallel access patterns necessary for feeding the array. In parallel, dedicated `AccumulatorMem` banks store the higher-precision partial sums generated during computation, featuring integrated hardware to support activation functions and scaling operations.

Data movement between these internal banks and external memory is managed by dedicated **DMA Engines**, the `StreamReader` and `StreamWriter`. Critically, the Scratchpad contains several **Integrated Data Processing Units** that operate on data as it moves, a key design choice that significantly reduces CPU overhead. These units perform a range of on-the-fly transformations, including scaling for quantization, pixel repetition for convolution optimizations, and applying activation functions. (A detailed breakdown of these processing units is provided in **Appendix B.2**).

Ultimately, the sophisticated architecture of the Scratchpad—with its combination of banked memory, integrated data processing, and dedicated DMA engines—is a direct and powerful answer to the "memory wall" problem. It is an active, intelligent subsystem engineered not just to store data, but to transform and deliver it, ensuring the massively parallel Mesh is constantly supplied with operands and enabling the high-throughput computation that is the central goal of this work.

6.4 Chapter Summary

This chapter has provided a deep dive into the internal architecture of the Gemmini accelerator, tracing the path of a command from the RoCC interface to the physical compute units. We began by examining the two-level command-unrolling

hierarchy, where high-level directives are translated into primitive micro-operations by the `LoopConv` and `LoopMatmul` modules. We then analyzed the decoupled execution model, orchestrated by the central Reservation Station, which allows for the concurrent operation of Load, Store, and Execute controllers to hide memory latency. Finally, we explored the physical hardware, including the distributed arithmetic in the Processing Elements and the configurable tile-and-mesh hierarchy of the systolic array.

This detailed analysis of Gemmini's internal architecture completes the theoretical and design-level exploration of our system. The subsequent chapter transitions from this architectural blueprint to practice, documenting the complete implementation process—from SoC configuration and hardware generation to the final system deployment and testing on the target FPGA platform.

Chapter 7: System Implementation and Testing

This chapter details the implementation of the RISC-V System-on-Chip (SoC) featuring a Gemmini accelerator. The preceding chapters have established the theoretical foundations of the RISC-V ISA, the Chipyard framework, and the architectural principles of DNN acceleration. We now transition from theory to practice, documenting the complete process from hardware configuration and generation to final system validation on an FPGA platform. This chapter will cover the specific SoC configuration, the integration of the Gemmini accelerator as a Rocket Custom Coprocessor (RoCC), the software stack required to boot a full Linux operating system, the automated build process, and finally, the verification of the complete system on a Xilinx VC707 FPGA board. This practical account serves as a tangible demonstration of the maturity of the open-source, full-stack design methodology adopted for this research.

7.1 Hardware Implementation: SoC Configuration

The foundation of our hardware design is the Chipyard SoC generation framework, which provides a flexible and powerful environment for constructing complex RISC-V systems. The result of this process is a complete, Linux-capable SoC, whose top-level architecture is illustrated in Figure 7.1. This diagram encapsulates the full-stack integration central to this thesis, depicting the system from the processor core and accelerator down to the specific peripheral IP cores and their connections to the physical hardware on the FPGA board.

At the heart of the system is the `RocketTile` (Figure 7.1), which contains the 64-bit RISC-V `RocketCore` and its private L1 instruction and data caches. Tightly coupled to the processor pipeline, as discussed in Chapter 5, is the `Gemmini RoCC Accelerator`. This integration via the `RoCC` interface allows for low-latency command dispatch from the CPU to the accelerator, a critical feature for high-performance operation.

The `RocketTile` communicates with the rest of the system through the coherent `SystemBus`, which is implemented using the `TileLink` protocol. This bus serves as the main interconnect for all major subsystems. The memory hierarchy extends from the L1 caches to a unified L2 cache and finally to the off-chip DDR3 memory. To bridge the on-chip `TileLink` protocol with the industry-standard `AXI` protocol used by the external memory controller IP, a `TileLink to AXI4 bridge` is automatically

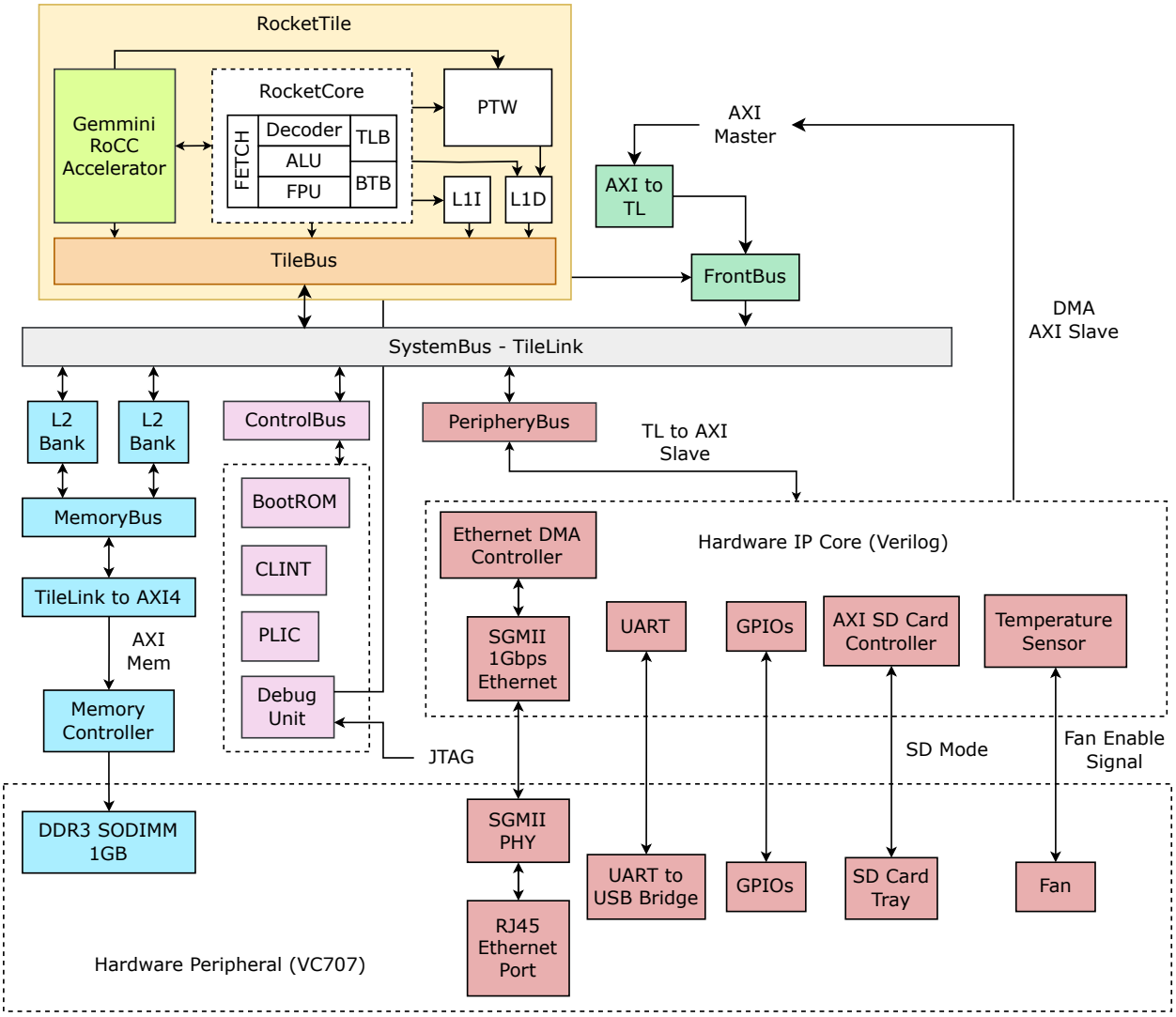


Figure 7.1: The implemented Rocket SoC with Gemmini on the VC707 FPGA board. The Rocket Core is connected to a 16x16 Gemmini systolic array, and various peripherals are integrated via AXI interfaces.

instantiated by the framework. This demonstrates a practical aspect of modern SoC design where components with disparate interface standards are seamlessly integrated.

To create a fully functional, Linux-capable computing platform, a rich set of peripherals is integrated via the `PeripheryBus`. Similar to the memory interface, a `TL to AXI Slave bridge` connects the `TileLink` bus to a set of `AXI`-based peripheral controllers. These include an `AXI SD Card Controller` to enable booting from an SD card, a `UART` controller for console I/O, and a complete Ethernet subsystem for network connectivity. These controller IPs, written in `Verilog`, are then wired to their corresponding physical counterparts on the VC707 board, such as the SD card tray and the RJ45 Ethernet port.

7.1.1 The `Rocket64b1gem16jtag` Configuration

The comprehensive architecture shown in (Figure 7.1) is not designed by hand in `Verilog`, but is programmatically generated from a high-level `Scala` configuration. As outlined in Chapter 3, this generator-based approach allows for rapid architectural exploration and ensures design consistency.

The specific configuration for our SoC is named `Rocket64b1gem16jtag`. This name concisely describes its key features: a 64-bit Rocket Core (`Rocket64`), one "big" core (`b1`), and a 16x16 Gemmini systolic array (`gem16`). This configuration is defined in `Scala` using Chipyard's `Config` classes, which employ a `mixin-based` approach to compose system components. This powerful paradigm allows features to be incrementally added to a base configuration through a series of traits, each beginning with the `With...` prefix. The core of the configuration is shown in Listing 7.1.

```
class Rocket64b1gem16jtag extends Config(
  new WithGemmini(16, 64) ++
  new WithInclusiveCache  ++
  new WithNBreakpoints(8) ++
  new WithNBigCores(1)    ++
  new WithBootROMFile("workspace/bootrom.img") ++
  new WithExtMemSize(0x380000000L) ++
  new WithNExtTopInterrupts(8) ++
  new WithDTS("freechips,rocketchip-vivado", Nil) ++
  new WithDebugSBA        ++
  new WithJtagDTM         ++
  new WithEdgeDataBits(64) ++
  new WithCoherentBusTopology ++
  new WithoutTLMonitors ++
  new BaseConfig)
```

Listing 7.1: The `Rocket64b1gem16jtag` configuration class.

The `WithGemmini(16, 64)` trait is the most critical for this work. It instructs the generator to instantiate the Gemmini accelerator with a specific set of hardware parameters derived from the defaultConfig within the Gemmini generator source code. The generated accelerator is a sophisticated system in its own right, comprising several key hardware blocks:

- `WithNBigCores(1)`: Instantiates a single, high-performance 64-bit Rocket Core. The Rocket Core was selected for its maturity and stability within the open-

source ecosystem, providing a reliable foundation for our system.

- `WithGemmini(16, 64)`: This is the crucial trait that integrates the Gemmini DNN accelerator, specifying a 16x16 systolic array mesh and a 64-bit wide DMA bus. This trait instructs the generator to instantiate a complete accelerator system based on its internal `defaultConfig`. The resulting hardware is a sophisticated system-within-a-system, comprising:
  - A Systolic Array Core: A 16x16 grid of Processing Elements (PEs), each capable of an 8-bit integer multiply and a 32-bit integer accumulate per cycle. The array supports both Weight Stationary (WS) and Output Stationary (OS) dataflows.
  - An On-Chip Memory System: A 256KB, 4-banked scratchpad memory for staging data and a separate 64KB, dual-ported accumulator memory for partial sums.
  - Control and Execution Logic: A Reservation Station to enable out-of-order execution of primitive commands, managed by decoupled Load, Store, and Execute controllers.
  - Specialized Hardware Units: A hardware Im2Col unit to accelerate convolution preprocessing and a 4-entry Translation Lookaside Buffer (TLB) to support virtual memory.
- `WithInclusiveCache`: Adds a last-level inclusive L2 cache to the memory hierarchy. This component is vital for performance, serving as a large, shared buffer that reduces the frequency of costly off-chip memory accesses, thereby helping to mitigate the "memory wall" problem.
- `WithExtMemSize(0x380000000L)`: Configures the size of the external DRAM to 14 GiB. A large main memory is essential for running a full operating system like Debian Linux, which requires significant space for the kernel, user-space applications, and runtime data.
- `WithBootROMFile(...)`: Specifies the path to the boot ROM image. This ROM contains the first-stage bootloader code that the processor will execute on reset, initiating the system boot sequence.

This Scala configuration is then consumed by the Chipyard generator-based framework to produce the concrete hardware implementation. The resulting top-level ar-

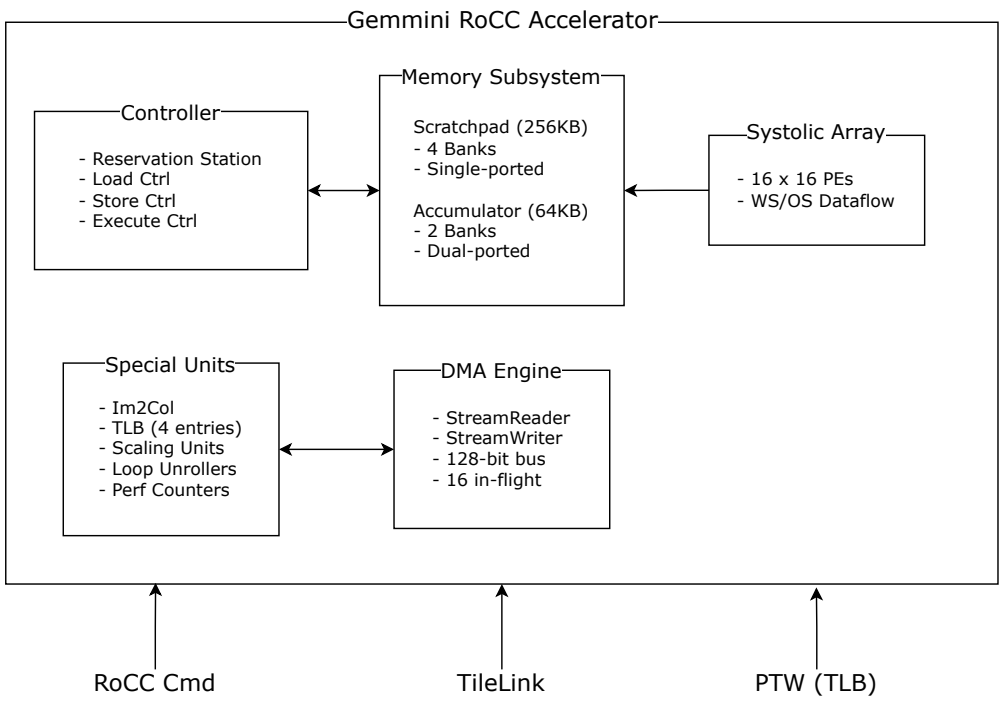


Figure 7.2: Gemmini Configuration on `Rocket64b1gem16jtag` SoC.

chitecture of the SoC, as deployed on the Xilinx VC707 FPGA, is illustrated in Figure 7.1.

7.2 Gemmini Integration via RoCC

While the RoCC interface provides a clean, standardized gateway for the CPU, it abstracts away a sophisticated internal command and control hierarchy within Gemmini itself. This internal micro-architecture functions as a hardware-based microcode engine; it is responsible for autonomously unrolling high-level, CISC-like commands received from the CPU into a stream of primitive micro-operations, thereby orchestrating the intricate dataflows and resource management essential for high-performance systolic computation. This section delves into this multi-layered system, detailing the journey of a command from the initial RoCC dispatch to its fully decoupled execution within the accelerator's asynchronous hardware modules.

7.2.1 The RoCC Interface

The primary conduit for communication between the Rocket Core and the Gemmini accelerator is the RoCC interface (Figure 3.12). This is a standardized feature of the Rocket Chip generator that allows for the tight coupling of co-processors. As part

of this standard, the RISC-V ISA reserves four instruction opcode spaces for custom extensions. The Gemmini accelerator is assigned the `custom3` opcode space, which corresponds to the binary value `0b1111011`. When the Rocket Core's instruction decoder encounters this specific opcode, it forgoes standard execution and instead initiates a dispatch to the attached RoCC unit.

```
class WithGemmini(mesh_size: Int, bus_bits: Int) extends Config((site,
    here, up) => {
  case BuildRoCC => up(BuildRoCC) ++ Seq(
    (p: Parameters) => {
      implicit val q = p
      implicit val v = implicitly[ValName]
      LazyModule(new gemmini.Gemmini(gemmini.GemminiConfigs.
    defaultConfig.copy(
        meshRows = mesh_size, meshColumns = mesh_size, dma_buswidth =
    bus_bits)))
    }
  )
  case SystemBusKey => up(SystemBusKey).copy(beatBytes = bus_bits/8)
})
```

Listing 7.2: The `WithGemmini` configuration trait, which instantiates the accelerator as a RoCC device.

The consequence of this configuration is that the Gemmini accelerator is assigned a reserved block of custom instruction opcodes. As defined in its internal configuration, Gemmini uses `OpcodeSet.custom3`. During program execution, when the Rocket Core's instruction decoder encounters an instruction with an opcode from this set, it recognizes it as a RoCC instruction. Rather than attempting to execute it, the core dispatches the entire instruction—including operands from the processor's register file (`rs1`, `rs2`) and the funct field—directly to the Gemmini unit over the dedicated RoCC command channel. The RocketTile hardware then automatically wires the command and response channels between the core and the Gemmini module.

This direct connection demonstrates a successful accelerator integration. It is what distinguishes the RoCC methodology from a standard memory-mapped peripheral, providing the low-latency communication that is a cornerstone of this system's design for high-performance DNN acceleration. The complete RoCC interface architecture, showing the connection pathways between the Rocket Core and the accelerator, is illustrated in Figure 3.12.

7.3 Peripheral Integration

To create a fully functional, Linux-capable computing platform, the core SoC must be integrated with a rich set of I/O peripherals for storage, networking, and user interaction. This section details the architecture and integration of these essential components. Unlike the Chisel-generated processor core which uses the `TileLink` protocol, most peripherals are integrated as pre-existing, open-source Verilog IP blocks that use the industry-standard `AXI4` protocol.

This creates a heterogeneous bus environment, which is managed by the Chipyard framework's diplomacy-generated protocol bridges. As shown in the overall system diagram (Figure 7.1), peripherals are not attached to the main coherent `SystemBus`. Instead, they connect to a dedicated, AXI4-based `PeripheryBus`. A specialized **TileLink to AXI4 Bridge** is automatically instantiated by the framework to handle all communication between the CPU and this peripheral bus, managing protocol translation, address decoding, and data width conversion.

Each peripheral is managed by a dedicated controller IP that acts as a middleman, translating high-level AXI bus transactions from the Rocket Core into the specific low-level protocols required by the hardware (e.g., SD command sequences or Ethernet MAC framing). For the system software to utilize these components, a corresponding driver must be present in the operating system. This driver interacts with the controller's memory-mapped registers to initiate data transfers and handle interrupts. The following subsections will briefly describe the implementation of the key peripherals integrated into this design: the SD Card controller, the UART controller, and the Ethernet subsystem, among others.

7.3.1 DDR Memory: High-Bandwidth Main Memory

The SoC's main memory system is built around a high-performance DDR3 SDRAM interface, which is essential for providing the capacity and bandwidth required to run a full Linux operating system and handle the large data sets of DNN workloads. The implementation does not use a custom controller but instead leverages the industry-standard **Xilinx Memory Interface Generator (MIG) 7-Series IP core**. This is a common and robust design practice that ensures reliable, high-frequency memory operation.

A key architectural feature of this design is its scalability. While the processor and system bus are designed with a **34-bit address width, capable of accessing**

up to 16 GiB of memory, the current physical implementation on the VC707 board uses a 1 GiB SODIMM. The AXI SmartConnect bridge, instantiated by Chipyard, automatically handles the address translation between the large logical address space and the smaller physical memory map. This demonstrates a forward-looking design, allowing for future memory expansion with minimal changes to the core SoC architecture.

7.3.2 SD Card: AXI SD Card Controller and SD Mode

The SoC includes an AXI SD Card controller, which allows the system to boot from an SD card. This controller is integrated into the system using the Chipyard framework's diplomacy mechanism, which automatically wires up the necessary AXI interfaces.

The SD card is accessed in SD mode, which is a common mode for embedded systems. The controller supports standard SD commands and allows the system to read and write data blocks, making it suitable for booting the operating system and storing user data.

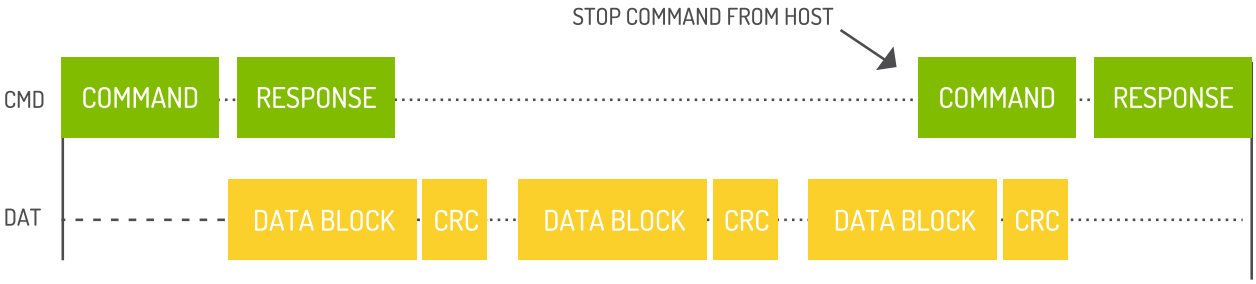


Figure 7.3: SD Mode: Multi-Block Read Operation. [3]

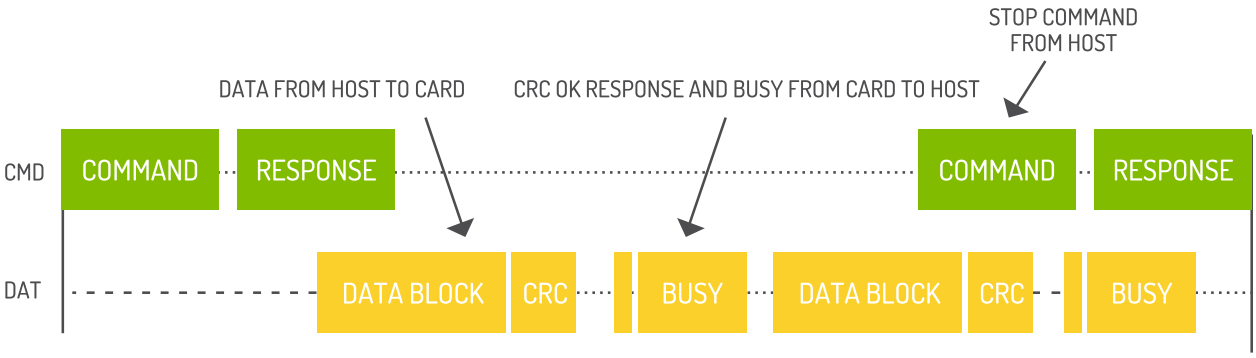


Figure 7.4: SD Mode: Multi-Block Write Operation. [3]

For comparion, here is how the same operation looks in SPI mode, which is another common mode for SD cards. The SPI mode is often used for compatibility

with older systems or when a simpler interface is required. The SPI Mode offers lower speed compared to SD mode, but it is simpler to implement and can be used in systems that do not require high-speed data transfer.

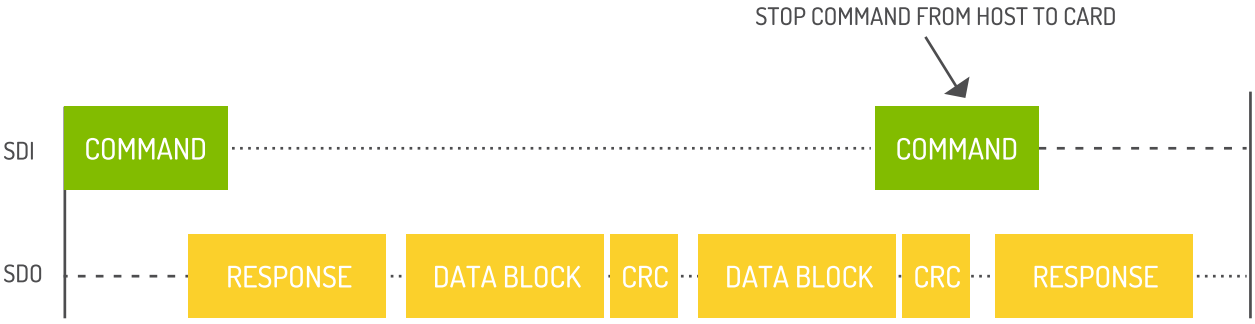


Figure 7.5: SPI Mode: Multi-Block Read Operation. [3]

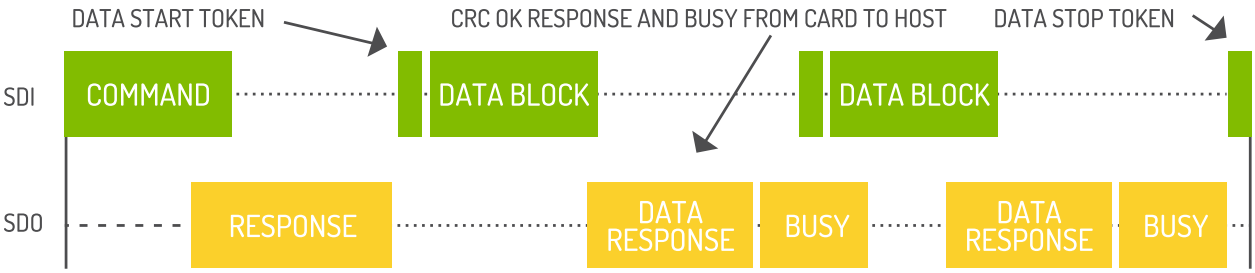


Figure 7.6: SPI Mode: Multi-Block Write Operation. [3]

7.3.3 UART: AXI UART Controller

To provide essential console I/O for debugging and user interaction with the operating system, the SoC integrates a robust UART controller. This component is not a generic library IP but a custom-designed Verilog module tailored for this system, demonstrating the ability to integrate bespoke hardware into the Chipyard flow.

The controller is connected to the `PeripheryBus` via an AXI4-Lite interface, making it a memory-mapped peripheral accessible by the Rocket Core. Architecturally, it is designed for high-performance, reliable communication and includes several key hardware features:

- **FIFO Buffering:** The controller features independent 16-entry transmit (TX) and receive (RX) FIFOs. These hardware buffers are crucial for preventing data loss at high baud rates, as they decouple the CPU from the byte-by-byte management of the serial link by allowing it to read and write data in bursts.

- **Hardware Flow Control:** Full RTS/CTS hardware flow control is implemented, allowing the UART to automatically signal its readiness to receive data and to pause transmission if the remote device is not ready. This is essential for preventing buffer overruns in high-throughput scenarios.

- **Interrupt Generation:** The controller can generate interrupts to the CPU based on configurable events, such as the receive FIFO reaching a certain threshold. This enables efficient, interrupt-driven I/O handling in the Linux kernel, rather than relying on inefficient polling.

7.3.4 Ethernet: AXI Ethernet Controller

The Ethernet subsystem provides high-speed network connectivity for the RISC-V RocketChip SoC, enabling communication with external systems and networks. The implementation consists of a four-layer architecture that bridges the processor's AXI4 bus interface to the physical Ethernet connection on the VC707 FPGA board.

The Ethernet subsystem is implemented as a hierarchical design with four distinct functional blocks, as illustrated in Figure 7.1.

1. **Ethernet DMA Controller**: serves as the primary interface between the RocketChip SoC and the Ethernet MAC layer. Provide functionality for managing data transfers between the SoC and the Ethernet MAC, including packet buffering, flow control, and interrupt handling.

2. **SGMII 1Gbps Ethernet MAC**: The MAC (Media Access Control) layer is implemented using the open-source `eth_mac_1g_fifo` module from the verilog-ethernet project [9]. This component provides: IEEE 802.3 Compliance, integrated frame-level FIFO Buffering, Frame Processing (Automatic frame padding, FCS generation and checking, and frame validation), AXI4-Stream Interface, Standard GMII Interface (Gigabit Media Independent Interface).

3. **SGMII PHY Interface**: implemented using Xilinx's integrated SGMII IP core within the VC707 FPGA. This layer provides: SGMII Protocol (Serial Gigabit Media Independent Interface), Auto-negotiation, Clock Generation (125 MHz clock), Signal Conditioning.

4. **Physical RJ45 Interface**: The physical layer terminates at the RJ45 connector on the VC707 board.

7.3.5 System Monitoring: XADC and Fan Control

To ensure operational stability and manage thermal output, the SoC integrates a dedicated system monitoring and fan control subsystem. This demonstrates a hybrid design approach, combining vendor IP with custom control logic for a robust, real-world solution. The subsystem consists of two primary components:

- **Xilinx XADC IP Core:** The on-chip Xilinx Analog-to-Digital Converter (XADC) is used for real-time monitoring of the FPGA's internal temperature and critical supply voltages (VCCINT, VCCAUX). This IP is configured to operate in a continuous sampling mode and is connected to the system via an AXI4-Lite interface.

- **Custom Fan Controller:** A custom-designed Verilog module provides PWM-based control for the chassis fan. Crucially, this module is directly connected to the XADC's hardware thermal alarm pin. This creates an autonomous, hardware-based safety mechanism: if the die temperature exceeds a predefined threshold (e.g., 60°C), the alarm triggers an emergency override, forcing the fan to 100% duty cycle, independent of any software control.

7.4 Development Environment

The project requires a specific development environment to ensure that all tools, compilers, and dependencies are correctly configured. The primary environment is an Ubuntu 20.04 LTS workstation.

The complete project is included with all submodules for the RISC-V ecosystem components (Chipyard, U-Boot, Linux, OpenSBI) which are required for building the SoC and running the operating system.

7.5 Software Stack and Build Process

To run a complete operating system and user applications, a comprehensive software stack is required. This project utilizes a standard boot flow for embedded Linux systems and automates the entire build process with a `Makefile`.

7.5.1 Boot Flow: OpenSBI, U-Boot, and Linux

The system boots in several stages:

1. **Boot ROM**: On power-up, the Rocket Core begins execution from an on-chip Boot ROM. This initial code is responsible for basic hardware initialization and loading the next boot stage from an external device, in this case, an SD card.

2. **OpenSBI**: The first piece of software loaded from the SD card is the RISC-V Open Source Supervisor Binary Interface (OpenSBI). OpenSBI provides a crucial abstraction layer, handling low-level hardware control (like inter-processor interrupts and timers) so that the operating system kernel doesn't need to be machine-specific.

3. **U-Boot**: OpenSBI then loads and jumps to the Das U-Boot bootloader. U-Boot is a more powerful bootloader responsible for further hardware initialization (e.g., DDR memory) and loading the Linux kernel into RAM.

4. **Linux Kernel**: Finally, U-Boot passes control to the Linux kernel. The kernel boots, mounts the root filesystem from the SD card, and starts the user-space services, providing a full Debian environment.

7.5.2 SoC Hardware Generation

The first stage of the build process is to generate the Verilog HDL for the SoC.

1. **Initial Verilog and Device Tree Generation**: The build system invokes the Scala Build Tool (SBT) to compile the Chisel sources using the `Rocket 64b1gem16jtag` configuration. This generates the initial Verilog HDL and a base Device Tree Source (`.dts`) file, which describes the core SoC components as inferred by Chipyard's diplomacy framework, as described in Chapter 6.

2. **Peripheral Integration**: This generated core is then integrated with several pre-existing, open-source peripheral controllers written in Verilog. These include the AXI UART controller, the AXI SD Card controller, and the AXI Ethernet controller.

3. **Device Tree Customization**: A crucial step is the creation of the final device tree. The base `system.dts` is combined with a board-specific overlay, `board/vc707/bootrom.dts`, which defines the VC707's unique peripherals like the UART, MMC, and Ethernet controllers at their specific memory-mapped addresses.

4. **Boot ROM Generation**: The customized device tree is then passed to the boot ROM's own build system, which compiles the C code for the first-stage bootloader and embeds the compiled device tree binary within the final 'bootrom.img' file.

5. **Final HDL Generation**: With the finalized 'bootrom.img' created, SBT is invoked a second time. This run uses the new boot ROM image to produce the final, correct Verilog HDL for the SoC, ensuring the processor's memory map includes the fully configured boot ROM.

7.6 Gemmini Testing

We are going to use the gemmini-rocc-tests (https://github.com/ucb-bar/gemmini-rocc-tests) to test the Gemmini accelerator. The tests are designed to verify the functionality of the Gemmini RoCC interface and the correctness of the accelerator's operations. The tests cover a wide range of scenarios, including basic arithmetic operations, matrix multiplication, convolution, and more complex DNN workloads.

With the hardware and software stack fully implemented and the testing methodology defined, the system is now ready for validation and evaluation. The following chapter presents the culminating results of this work, analyzing the hardware resource utilization of the final design, validating the full-stack software deployment, and, most critically, quantifying the performance impact of the Gemmini accelerator.

Chapter 8: Implementation Results and Discussion

This chapter presents the culminating results of the thesis, transitioning from theoretical design to tangible implementation and empirical validation. The System-on-Chip (SoC), designed programmatically in Chisel, was compiled through the FIRRTL intermediate representation and subsequently implemented using the Xilinx Vivado toolchain for the VC707 FPGA target. This chapter is organized into three main sections. First, it details the hardware implementation results, including the final system architecture and an analysis of on-chip resource utilization and power consumption. Second, it documents the successful deployment of the full software stack, demonstrating a boot-to-Linux environment. Finally, and most critically, it evaluates the performance of the integrated Gemmini DNN accelerator, providing quantitative evidence of its effectiveness.

8.1 Hardware Implementation Analysis

The hardware generation process produces a set of Verilog files that are synthesized, placed, and routed by the Vivado Design Suite. The following sections analyze the final hardware implementation from architectural, resource, and power perspectives.

8.1.1 System Block Design and Address Map

The generated system is a complex integration of the RISC-V core, the Gemmini accelerator, memory subsystems, and peripherals. The block diagrams in Figure 8.1, Figure 8.3, and Figure 8.2 provide a visual representation of the final hardware architecture as implemented in Vivado. These diagrams confirm the successful integration of all system components, from the top-level SoC (Figure 8.1) down to the specific connections for DDR memory (Figure 8.3) and I/O peripherals like UART and Ethernet (Figure 8.2).

The system's memory-mapped architecture is confirmed by the Address Editor view in Figure 8.4, which shows the memory ranges assigned to the DDR memory controller, the Gemmini DMA interface, and various peripheral buses. This confirms that the diplomacy framework within Chipyard correctly generated the necessary hardware and address decoding logic.

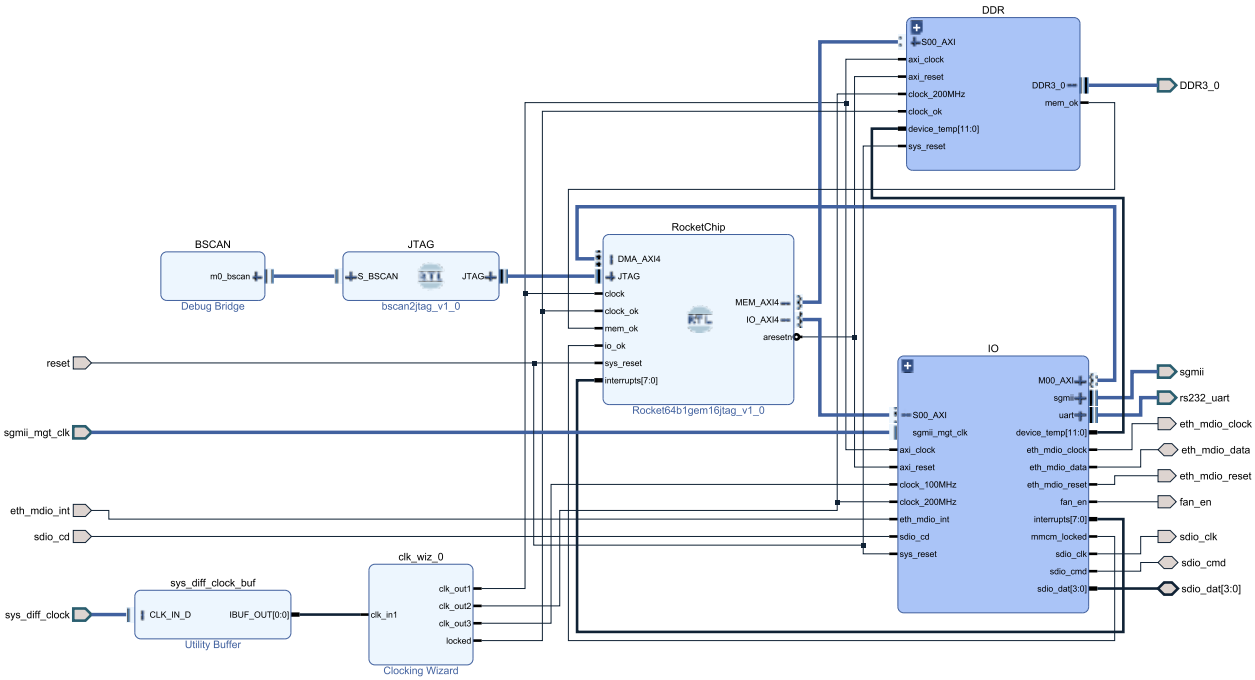


Figure 8.1: Block Design of the System, showing the top-level integration of the RocketChip core, DDR, and IO subsystems.

8.1.2 Vivado Implemented Design Results

8.1.2.1 Device Utilization Analysis

Resource utilization is a critical metric for evaluating the cost and feasibility of an SoC implementation on a given FPGA. The post-implementation utilization report from Vivado provides a detailed accounting of the hardware resources consumed by the design.

As shown in Figure 8.5, the complete, Linux-capable SoC is a substantial design, consuming **80.92%** of the available Look-Up Tables (LUTs), **28.59%** of Block RAM (BRAM), and **8.96%** of DSP slices on the Xilinx VC707's Virtex-7 FPGA. The high LUT utilization is expected given the complexity of a 64-bit processor core with an L2 cache and numerous peripherals. The significant BRAM usage is primarily attributable to the memory-intensive components of the design, namely the L2 cache and the Gemmini accelerator's scratchpad and accumulator memories.

To isolate the resource cost of the Gemmini accelerator itself, a hierarchical utilization analysis was performed. Figure 8.6 shows that the Gemmini module ('riscv_tile/riscv_tile/Queue_Gemmini') consumes 180,142 LUTs. This represents approximately 73.5% of the logic within the 'riscv_tile' and underscores the fact that the specialized accelerator is, by far, the largest single component in the processor

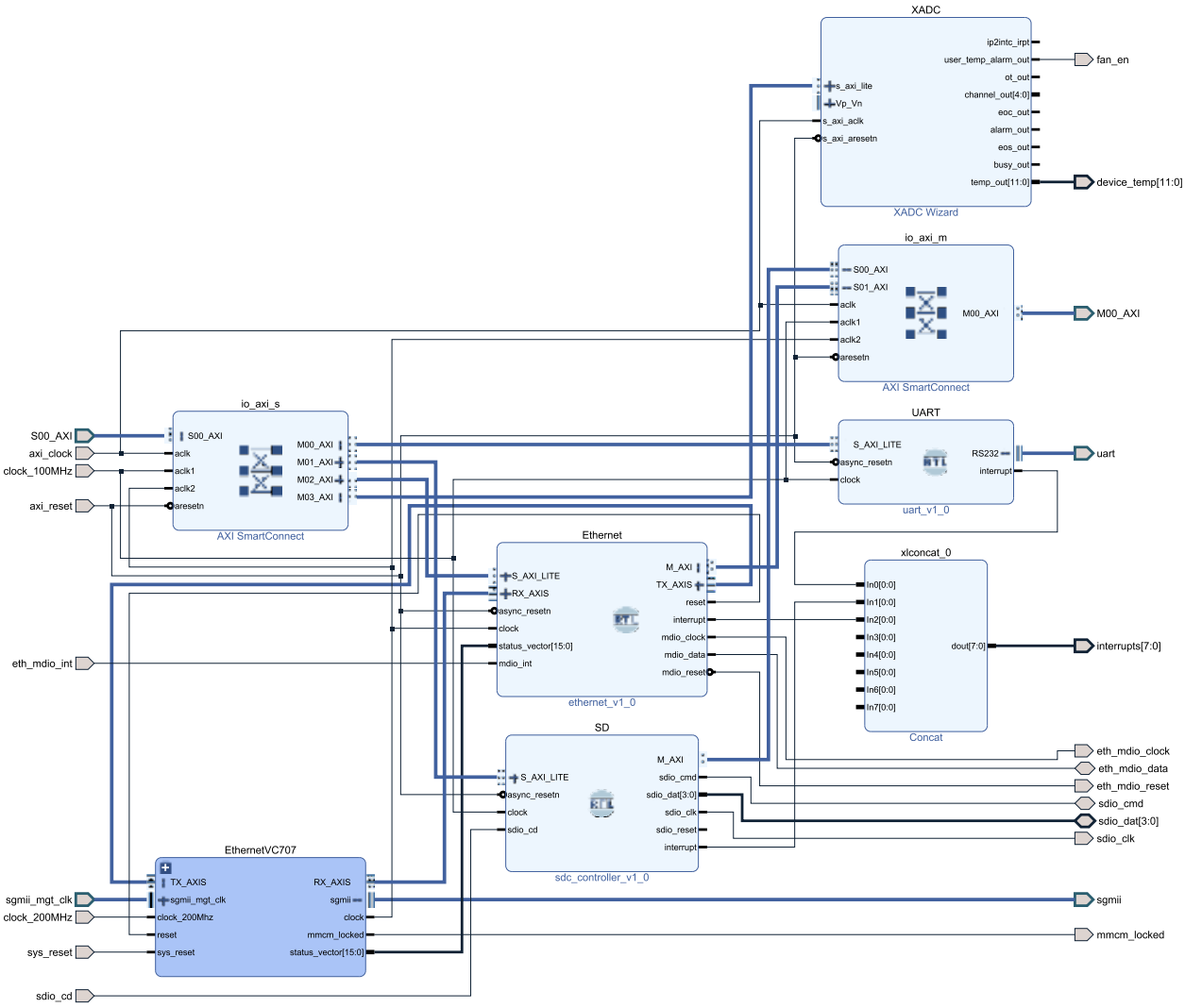


Figure 8.2: Block Design of the IO Subsystem, illustrating the AXI interconnect for peripherals such as Ethernet, SD Card, and UART.

tile, dwarfing the RISC-V core itself. This trade-off of silicon area for specialized performance is the central premise of hardware acceleration.

8.1.2.2 Power Consumption Analysis

Power analysis, performed by Vivado based on the placed-and-routed design and simulation-derived activity factors, is crucial for understanding the thermal and power-delivery requirements of the system.

The power summary in Figure 8.7 reports a **Total On-Chip Power of 3.856W**. A key observation is the breakdown between static and dynamic power. Dynamic power, consumed by switching logic, accounts for 3.309W (86% of the total), while static power is only 0.237W (6%). This is characteristic of a high-performance digi-

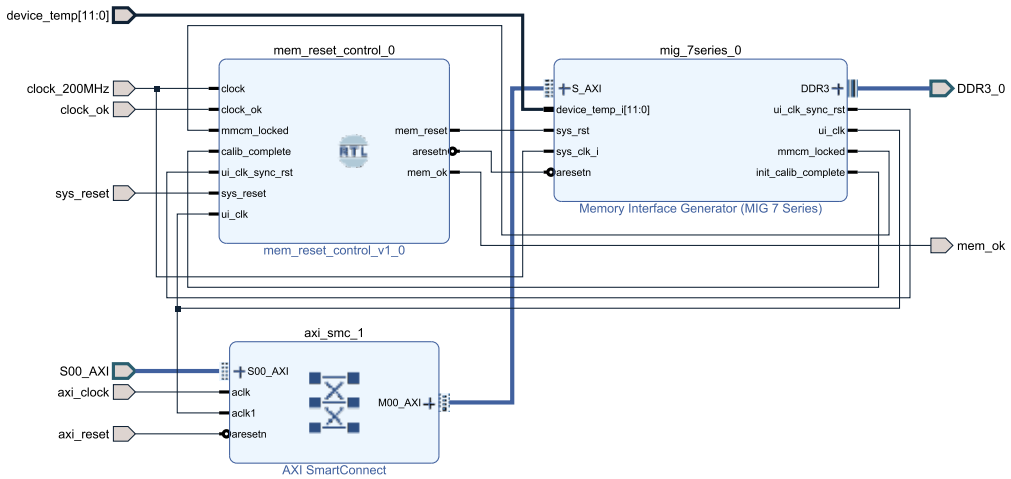


Figure 8.3: Block Design of the DDR Subsystem, detailing the Memory Interface Generator (MIG) and its AXI connection.

Diagram | Address Editor | Address Map

Assigned (7) Unassigned (0) Excluded (0) Hide All

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/IO/Ethernet					
/IO/Ethernet/M_AXI (34 address bits : 16G)					
/RocketChip/DMA_AXI4	DMA_AXI4	reg0	0x0	16G	0x3_FFFF_FFFF
/IO/SD					
/IO/SD/M_AXI (34 address bits : 16G)					
/RocketChip/DMA_AXI4	DMA_AXI4	reg0	0x0	16G	0x3_FFFF_FFFF
Network 1					
/RocketChip					
/RocketChip/IO_AXI4 (31 address bits : 2G)					
/IO/Ethernet/S_AXI_LITE	S_AXI_LITE	reg0	0x6002_0000	64K	0x6002_FFFF
/IO/SD/S_AXI_LITE	S_AXI_LITE	reg0	0x6000_0000	64K	0x6000_FFFF
/IO/UART/S_AXI_LITE	S_AXI_LITE	reg0	0x6001_0000	64K	0x6001_FFFF
/IO/XADC/s_axi_lite	s_axi_lite	Reg	0x6003_0000	64K	0x6003_FFFF
Network 2					
/RocketChip					
/RocketChip/MEM_AXI4 (34 address bits : 16G)					
/DDR/mig_7series_0/memmap	S_AXI	memaddr	0x0	16G	0x3_FFFF_FFFF

Figure 8.4: Address Editor view, confirming the memory map for the integrated peripherals.

tal design where power consumption is dominated by computational activity rather than leakage. The report identifies that the most significant contributors to dynamic power are Clocks, Signals, and I/O, which is expected for a large, high-frequency SoC with multiple external interfaces. The power hierarchical report (Figure 8.8) further shows that the Gemmini module contributes significantly to the overall power budget, consistent with its large area and high level of computational activity.

Summary

Resource	Utilization	Available	Utilization %
LUT	245673	303600	80.92
LUTRAM	9365	130800	7.16
FF	106058	607200	17.47
BRAM	294.50	1030	28.59
DSP	251	2800	8.96
IO	133	700	19.00
GT	1	28	3.57
MMCM	4	14	28.57
PLL	1	14	7.14

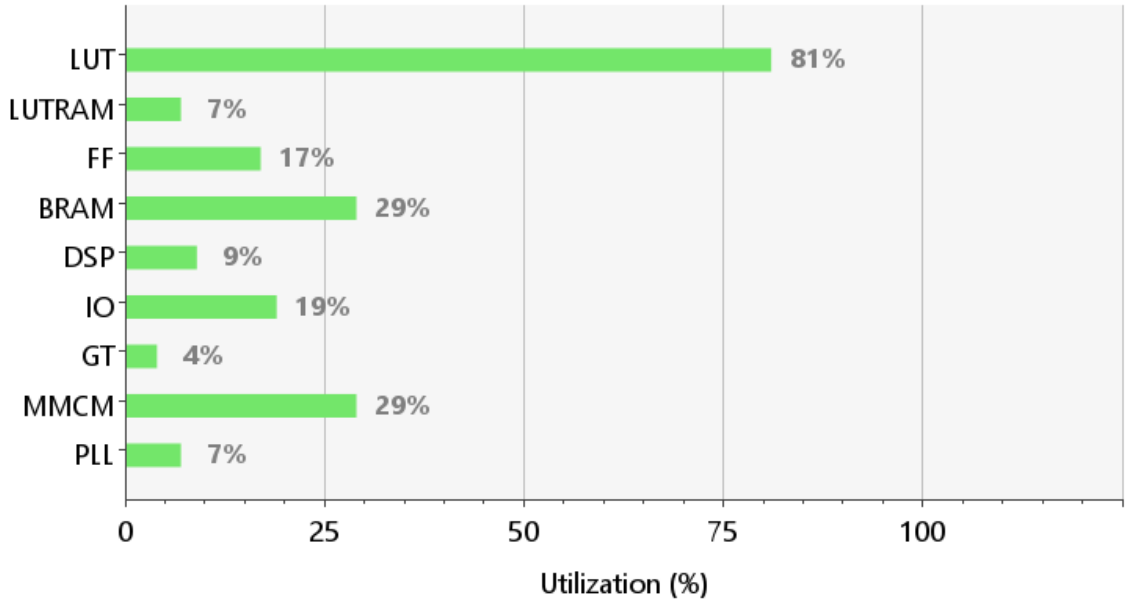


Figure 8.5: Detailed Utilization Summary for the entire SoC on the VC707 FPGA.

Name	Slice LUTs	Slice Registers (607200)	F7 Muxes (151800)	F8 Muxes (75900)	Slice (75900)	LUT as Logic (303600)	LUT as Memory (130800)	Block RAM Tile (1030)	DSPs (2800)
riscv_wrapper	245673	106058	3692	791	71517	236308	9365	294.5	251
riscv_i (riscv)	245673	106058	3692	791	71517	236308	9365	294.5	251
RocketChip (riscv_RocketChip_0)	224082	83768	3299	777	64577	218896	5186	288	251
inst (riscv_RocketChip_0_Rocket64b1gem16jtag	224082	83750	3299	777	64549	218896	5186	288	251
rocket_system (riscv_RocketChip_0_RocketSy	224075	83716	3299	777	64547	218889	5186	288	251
tile_prci_domain (riscv_RocketChip_0_TileP	206701	77251	1867	222	60131	204046	2655	152	251
tile_reset_domain_tile (riscv_RocketChip	206173	77229	1867	222	59988	203913	2260	152	251
v (riscv_RocketChip_0_Gemmini)	180142	64040	678	110	52531	178136	2006	128	236

Figure 8.6: Hierarchical Utilization of the Gemmini accelerator module.

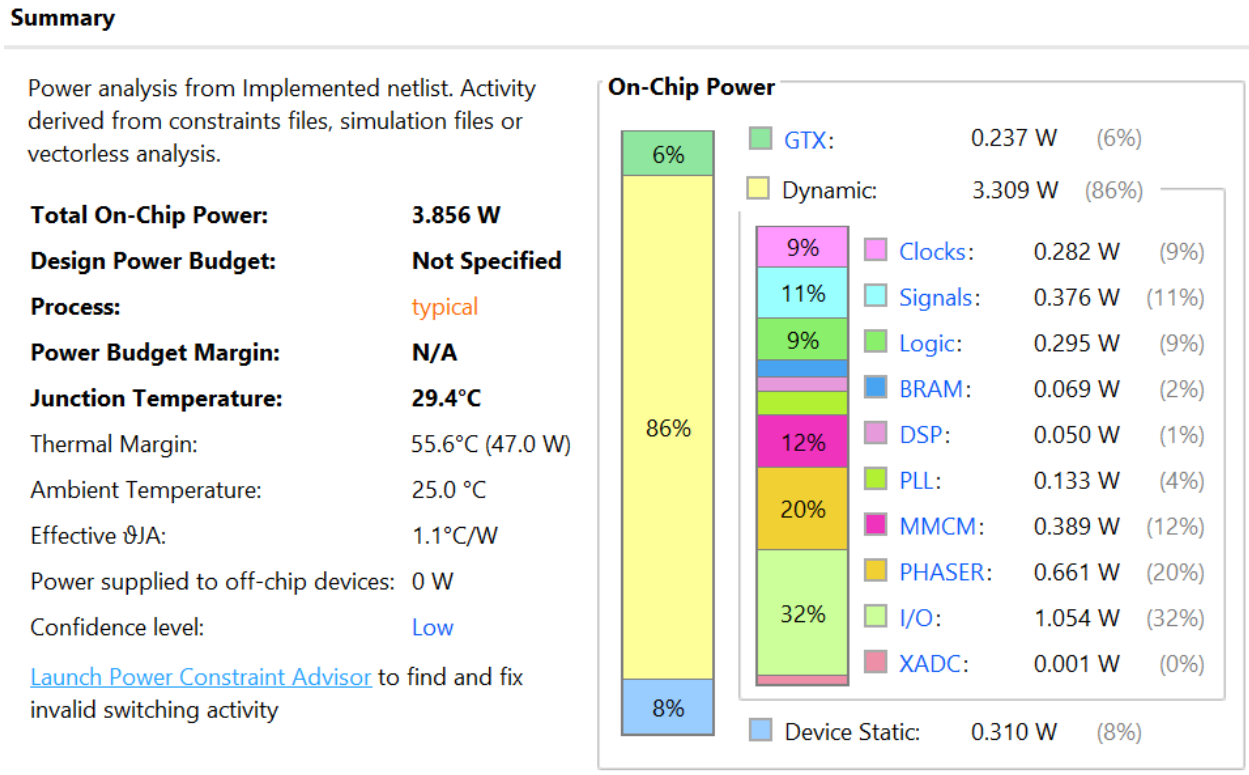


Figure 8.7: Post-implementation on-chip power summary.

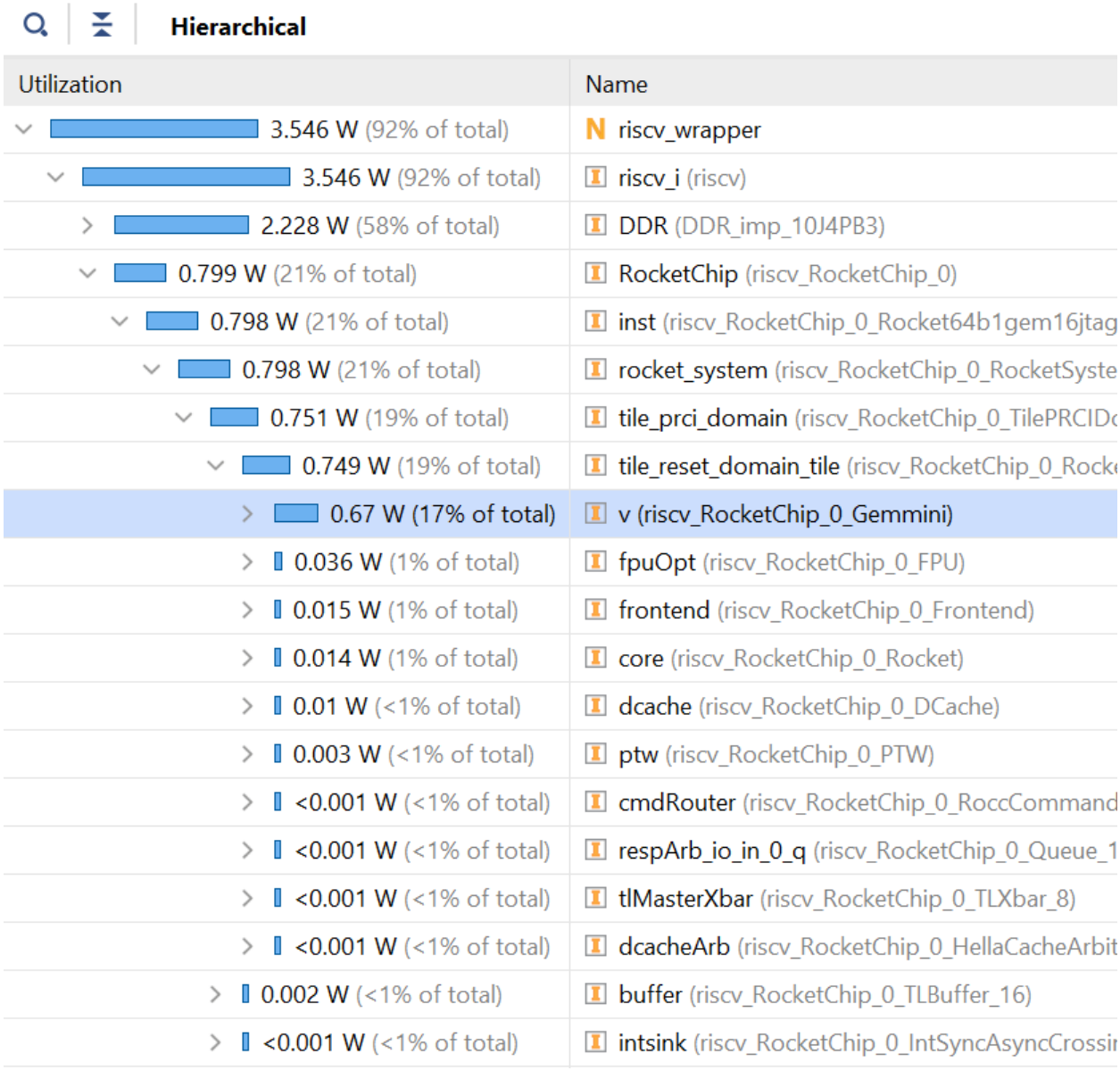


Figure 8.8: Hierarchical power report for the Gemmini accelerator.

8.2 Software Stack Validation

A primary objective of this thesis was to implement a full-stack, Linux-capable system. This section provides evidence of this successful implementation by documenting the complete boot process, from the initial on-chip Boot ROM to a fully functional Debian Linux user environment.

8.2.1 System Boot Sequence

The system boot process involves a multi-stage software chain, starting with the first-stage bootloader in ROM, which then loads subsequent stages from the SD card. The console output in Listing 8.1 captures this entire sequence.

The process begins with the ‘RISC-V 64, Boot ROM‘ identifying the hardware and proceeding to load the next stage, OpenSBI, which is the ‘boot.elf‘ file on the SD card. OpenSBI (Open Source Supervisor Binary Interface) successfully initializes, reporting platform details such as the HART (Hardware Thread) count and timer frequency. It then loads and jumps to the U-Boot bootloader. U-Boot initializes the DRAM and other system hardware before loading the Linux kernel (‘image-v6.11.10‘) and the Flattened Device Tree (FDT) blob into memory. Finally, the message ‘Starting kernel ...‘ marks the successful handoff to the Linux operating system, which proceeds to boot, initialize its drivers, and present a login prompt.

```
RISC-V 64, Boot ROM V3.8

OpenSBI v1.6
   ____                   _____ ____ _____
  / __ \                 / ____|  _ \_   _|
 | |  | |_ __   ___ _ __ | (___ | |_) || |
 | |  | | '_ \ / _ \ '_ \ \___ \|  _ < | |
 | |__| | |_) |  __/ | | |____) | |_) || |_
  \____/| .__/ \___|_| |_|_____/|____/_____|
        | |
        |_|

Platform Name               : freechips,rocketchip-vivado
...
(Abridged OpenSBI and U-Boot output for brevity)
...
```

```
## Flattened Device Tree blob at 00010080
   Booting using the fdt blob at 0x010080
   Loading Device Tree to 0000000084ffc000, end
   0000000084fffe3f ... OK

Starting kernel ...

[    0.000000] Linux version 6.11.10-dirty (btdat@btdat-hpZ4)
   ...
[    0.000000] Machine model: freechips,rocketchip-vivado
[    0.000000] SBI specification v2.0 detected
```

Listing 8.1: Console output capturing the full boot sequence from Boot ROM to the Linux kernel.

8.2.2 Debian Linux Environment

After booting, the system provides a standard Debian GNU/Linux environment. Listing 8.2 confirms this by showing a successful login and the output of standard Linux commands. The ‘uname -a‘ command verifies the kernel version and the ‘riscv64‘ architecture. Critically, the output of ‘cat /proc/cpuinfo‘ confirms the processor’s identity as ‘sifive,rocket0‘ and details its ISA as ‘rv64imafdc_...‘, matching the design specification. This successful boot to a full-featured OS validates the entire hardware and software co-design process.

```
Debian GNU/Linux trixie/sid debian ttyAU0

debian login: debian
Password:
Linux debian 6.11.10-dirty #1 SMP Thu Jan  9 16:51:21 +07 2025
   riscv64
...
debian@debian:~$ uname -a
Linux debian 6.11.10-dirty #1 SMP Thu Jan  9 16:51:21 +07 2025
   riscv64 GNU/Linux
debian@debian:~$ cat /proc/cpuinfo
processor       : 0
hart            : 0
isa             : rv64imafdc_zicntr_zicsr_zifencei_zihpm_zca_zcd
```

```
mmu             : sv39
uarch           : sifive,rocket0
mvendorid       : 0x0
marchid         : 0x1
mimpid          : 0x20181004
hart isa        : rv64imafdc_zicntr_zicsr_zifencei_zihpm_zca_zcd
```

Listing 8.2: Demonstration of a functional Debian Linux environment on the implemented SoC.

8.3 Gemmini Accelerator Performance Evaluation

With the SoC's functional correctness and stability validated, the final and most critical evaluation is to quantify the performance benefits of the Gemmini DNN accelerator. The evaluation involved running a comprehensive suite of benchmarks from the `gemmini-rocc-tests` repository, comparing the cycle counts of software-only implementations on the RISC-V Rocket Core against the hardware-accelerated versions. The results provide a definitive measure of the speedup achieved across various workloads.

8.3.1 Functional Correctness

As a prerequisite for performance evaluation, the functional correctness of the accelerator was verified. All 25 test cases within the benchmark suite, spanning matrix multiplications, convolutions, and full network layers, passed successfully. A representative test, `tiled_matmul_ws`, which multiplies two 512×512 matrices, confirmed that the output matrix from the Gemmini accelerator was bit-perfect identical to the one produced by the pure software implementation. This confirms the correctness of the entire hardware/software co-design, from RoCC instruction dispatch to DMA transfers and the systolic array computation.

8.3.2 Performance Speedup and Analysis

The primary motivation for this work was to demonstrate the dramatic performance uplift provided by a specialized hardware accelerator for DNN workloads. The results, summarized in Table 8.1, unequivocally validate this thesis.

The performance gains are profound. While standard convolutions already show a remarkable ∼541× average speedup, the most impressive results come from workloads

Table 8.1: Summary of Gemmini performance speedup across key operational categories. Detailed results for each test case are provided in Appendix E.

Operation Category	**Execution Time Range (ns)**		**Speedup Range**	**Average Speedup**
	CPU	**Gemmini**		
Matrix Multiplication	510M - 4.3B	12M - 39M	41× - 353×	∼**180**×
Standard Convolution	44.7B - 45.4B	81M - 84M	532× - 554×	∼**541**×
Transformed Convolution	132B - 169B	129M - 132M	1,007× - 1,309×	∼**1,110**×
Depthwise Convolution	1.5B	103M	15×	**15**×

optimized for the hardware's dataflow. The `conv_trans_weight_0132` test, which leverages weight tensor transformations to maximize data reuse within the systolic array, achieved a staggering **1,308-fold speedup** over the software-only implementation.

$$\text{Speedup} = \frac{\text{CPU Time}}{\text{Gemmini Time}} = \frac{169,097,846,400}{129,206,400} \approx 1308.7\times \tag{8.1}$$

This result is a powerful demonstration of the principles discussed in Chapter 4. By architecting the hardware and software to work in concert, we can effectively overcome the Memory Wall bottleneck that cripples general-purpose CPUs on these workloads.

8.3.3 Real-World Workload Analysis: MobileNetV1

To evaluate performance on a realistic application, the full MobileNetV1 neural network was executed on the system. The accelerator completed the inference task in **3.81 billion nanoseconds (3.81 seconds)**. A breakdown of the execution time, shown in Table 8.2, reveals critical insights into the performance characteristics of modern CNNs.

Notably, the relatively low-arithmetic-intensity **Depthwise Convolution** layers dominate the runtime, consuming 87.5% of the total execution time. This is despite having a lower speedup (∼15×) compared to standard convolutions (∼541×). This analysis highlights that optimizing for the most common layer types is critical for

Table 8.2: Cycle breakdown for a single inference pass of MobileNetV1 on the Gemmini accelerator.

MobileNetV1 Component	Execution Time (ns)	Percentage of Total
Depthwise Convolution	3,336,960,000	87.5%
Matrix Multiplication	310,441,600	8.1%
Standard Convolution	74,860,800	2.0%
Other (ResAdd, etc.)	92,643,200	2.4%
Total	**3,814,905,600**	**100%**

overall application performance.

8.3.4 Discussion of Results

The empirical data reveals several key characteristics of the implemented system:

- **Dataflow is King:** The highest speedups (>1000×) were achieved in tests (`conv_trans_*`) where the data was explicitly arranged to match the hardware's preferred weight-stationary dataflow. This confirms that performance is not just about raw compute power, but about the synergy between algorithm and architecture.
- **Computational Intensity Matters:** The comparatively low 15× speedup on depthwise convolutions is expected. These layers have a lower ratio of computation to memory access, making them more memory-bound and limiting the benefits of the systolic array.
- **Hardware Utilization:** A performance analysis test (`tiled_matmul_ws_perf`) revealed that even with high speedup, the peak utilization of the systolic array was only **2%**. This suggests that the system is often limited by data movement (memory bandwidth) rather than by computation. This is a common challenge in accelerator design and presents a clear avenue for future research, such as implementing more advanced data prefetching mechanisms or exploring different scratchpad memory architectures.

These quantitative results serve as the ultimate validation of this thesis. They prove that a tightly-coupled, specialized hardware accelerator, implemented through a full-stack, open-source methodology, provides a dramatic and essential performance uplift for modern computational workloads.

Chapter 9: Conclusion and Future Work

This thesis successfully evaluated the Chipyard framework, demonstrating its superiority over traditional Quartus-based flows for our laboratory's objectives due to its integrated VLSI toolchain, support for industry-standard AXI4 interconnects, and its open-source nature.

LIST OF PUBLICATIONS

[P1] N. H. Nguyen, T. P. Dang, T. K. Tran, T. D. Bui, T. T. Hoang, and H. T. Huynh. "A Configurable 2D-Integer DCT Hardware Accelerator Compatible with H.266 Standard based on RISC-V Architecture". In: *2024 7th International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*. Yogyakarta, Indonesia, 2024, pp. 894–899. DOI: 10.1109/ISRITI64779.2024.10963564.

[P2] N. H. Nguyen, T. P. Dang, T. D. Bui, T. T. Hoang, C. K. Pham, and H. T. Huynh. "Designing and Implementing a 2D Integer DCT Hardware Accelerator Fully Compatible with Versatile Video Coding". In: *Computational Science and Its Applications - ICCSA 2024 Workshops*. Ed. by Osvaldo Gervasi, Beniamino Murgante, Chiara Garau, David Taniar, Ana Maria A. C. Rocha, and M. N. Faginas Lago. Vol. 14815. Lecture Notes in Computer Science. Springer, Cham, 2024. DOI: 10.1007/978-3-031-65154-0_7.

[P3] T. M. T. Huynh, T. K. Tran, T. D. Bui, C. K. Pham, and H. T. Huynh. "An Efficient Algorithm Compatible with Low-performance Hardware for AI Edge Devices in Arrhythmia Prediction". In: *5th International Conference on Intelligent Systems & Networks (ICISN 2025)*. 2025.

[P4] D. H. Phan, T. M. T. Huynh, T. K. Tran, T. D. Bui, C. K. Phan, and H. T. Huynh. "Efficient AI Model and Hardware Architecture Based on CNN for Arrhythmia Prediction". In: *2025 10th IEEE International Conference on Integrated Circuits, Design, and Verification (ICDV 2025)*. 2025.

BIBLIOGRAPHY

[1] Alon Amid et al. "Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs". In: *IEEE Micro* 40.4 (2020), pp. 10–21. ISSN: 1937-4143. DOI: 10.1109/MM.2020.2996616.

[2] Krste Asanović et al. *The Rocket Chip Generator*. Tech. rep. UCB/EECS-2016-17. Electrical Engineering and Computer Sciences, University of California, Berkeley, 2016. URL: http://www.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html.

[3] Cactus Technologies. *An Introduction To SD Card Interface*. White Paper CTWP009. Suite C, 15/F, Capital Trade Center, 62 Tsun Yip Street, Kwun Tong, Kowloon, Hong Kong: Cactus Technologies Limited. URL: https://www.cactus-tech.com/wp-content/uploads/2019/03/An-Introduction-To-SD-Card-Interface.pdf.

[4] C. Celio, D. Patterson, and K. Asanovic. *The Berkeley Out-of-Order Machine (BOOM) Design Specification*. University of California, Berkeley. 2016. URL: https://github.com/ccelio/riscv-boom-doc/raw/gh-pages/boom-spec.pdf.

[5] C. Celio, D. A. Patterson, and K. Asanovic. *The Berkeley out-of-order machine (BOOM): An industry-competitive, synthesizable, parameterized RISC-V processor*. Tech. Rep. UCB/EECS-2015-167. EECS Department, University of California, Berkeley, 2015. URL: https://www.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-167.html.

[6] Christopher Celio. *A Highly Productive Implementation of an Out-of-Order Processor Generator*. Tech. rep. UCB/EECS-2018-151. Electrical Engineering and Computer Sciences, University of California at Berkeley, 2018. URL: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2018/EECS-2018-151.html.

[7] Enjoy-Digital. *LiteX: Build your hardware, easily!* GitHub Repository. Accessed: July 10, 2025. 2025. URL: https://github.com/enjoy-digital/litex.

[8] Farzad Farshchi, Qijing Huang, and Heechul Yun. "Integrating NVIDIA Deep Learning Accelerator (NVDLA) with RISC-V SoC on FireSim". In: *2019 2nd Workshop on Energy Efficient Machine Learning and Cognitive Computing for*

Embedded Applications (EMC2). 2019, pp. 21–25. DOI: 10.1109/EMC249363.2019.00012.

[9] Alex Forencich. *Verilog Ethernet Components*. GitHub Repository. Accessed: July 2025. 2023. URL: https://github.com/alexforencich/verilog-ethernet.

[10] Hasan Genc et al. *Gemmini @ MLSys 2022*. Presentation Slides, MLSys 2022 Tutorial. Accessed: [Your Access Date]. 2022. URL: https://mlsys.org/virtual/2022/tutorial/18433.

[11] Hasan Genc et al. "Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration". In: *Proceedings of the 58th Annual ACM/IEEE Design Automation Conference*. IEEE Press, 2022, pp. 769–774. ISBN: 9781665432740. URL: https://doi.org/10.1109/DAC18074.2021.9586216.

[12] Dennis Agyemanh Nana Gookyi et al. "Deep Learning Accelerators' Configuration Space Exploration Effect on Performance and Resource Utilization: A Gemmini Case Study". In: *Sensors* 23.5 (2023). ISSN: 1424-8220. DOI: 10.3390/s23052380. URL: https://www.mdpi.com/1424-8220/23/5/2380.

[13] Mark Horowitz. "1.1 Computing's energy problem (and what we can do about it)". In: *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 2014, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323.

[14] Norman P. Jouppi et al. "In-Datacenter Performance Analysis of a Tensor Processing Unit". In: *SIGARCH Comput. Archit. News* 45.2 (June 2017), pp. 1–12. ISSN: 0163-5964. DOI: 10.1145/3140659.3080246. URL: https://doi.org/10.1145/3140659.3080246.

[15] Florent Kermarrec et al. "LiteX: an open-source SoC builder and library based on Migen Python DSL". In: *CoRR* abs/2005.02506 (2020). arXiv: 2005.02506. URL: https://arxiv.org/abs/2005.02506.

[16] Binh Kieu-do-Nguyen et al. "Hardware Software Co-Design for Multi-Threaded Computation on RISC-V-Based Multicore System". In: *IEEE Access* 12 (2024), pp. 177312–177326. DOI: 10.1109/ACCESS.2024.3505940.

[17] Kung. "Why systolic architectures?" In: *Computer* 15.1 (1982), pp. 37–46. DOI: 10.1109/MC.1982.1653825.

[18] LiteX-Hub. *LiteX-Boards: LiteX boards files*. GitHub Repository. Accessed: July 10, 2025. 2025. URL: https://github.com/litex-hub/litex-boards.

[19] lowRISC. *OpenTitan: Open source silicon root of trust*. GitHub Repository. Accessed: July 10, 2025. 2025. URL: https://github.com/lowRISC/opentitan.

[20] Sparsh Mittal and Sumanth Umesh. "A survey On hardware accelerators and optimization techniques for RNNs". In: *Journal of Systems Architecture* 112 (2021), p. 101839. ISSN: 1383-7621. DOI: https://doi.org/10.1016/j.sysarc.2020.101839. URL: https://www.sciencedirect.com/science/article/pii/S1383762120301314.

[21] Davide Pala. "Design and programming of a coprocessor for a RISC-V architecture: Guidelines for embedding computing cores as RISC-V coprocessors". Supervisor: Prof. Massimo Poncino. Master's Thesis. Torino, Italy: Politecnico di Torino, 2017. URL: http://webthesis.biblio.polito.it/id/eprint/6589.

[22] RISC-V International. *Members*. Accessed: Jun. 16, 2025. 2025. URL: https://riscv.org/members/.

[23] SiFive. *A Winning Processor Portfolio*. Accessed: Dec. 30, 2024. 2024. URL: https://www.sifive.com/risc-v-core-ip.

[24] SiFive Inc. *SiFive TileLink Specification*. Version 1.7.1. Accessed: Jun. 6, 2025. Dec. 2018. URL: https://sifive.cdn.prismic.io/sifive%2F57f93ecf-2c42-46f7-9818-bcdd7d39400a_tilelink-spec-1.7.1.pdf.

[25] Vivienne Sze et al. *Efficient processing of deep neural networks*. Jan. 2020. DOI: 10.1007/978-3-031-01766-7. URL: https://doi.org/10.1007/978-3-031-01766-7.

[26] The OpenTitan Project. *OpenTitan: Open source silicon root of trust (RoT)*. Website. Accessed: July 10, 2025. 2025. URL: https://opentitan.org/.

[27] The PULP Platform Team. *PULP Implementation*. Website. Accessed: July 10, 2025. 2025. URL: https://pulp-platform.org/implementation.html.

[28] The PULP Platform Team. *PULP Platform*. Website. Accessed: July 10, 2025. 2025. URL: https://pulp-platform.org/.

[29] A. Waterman et al. *The RISC-V instruction set manual volume II: Privileged architecture version 1.7*. Tech. Rep. UCB/EECS-2015-49. EECS Department, University of California, Berkeley, 2015.

[30] Andrew Waterman and Krste Asanović, eds. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. Document Version 20240411. RISC-V Foundation. Apr. 2024. URL: https://github.com/riscv/riscv-isa-manual/releases/download/20240411/unpriv-isa-asciidoc.pdf.

[31] Wm. A. Wulf and Sally A. McKee. "Hitting the memory wall: implications of the obvious". In: *SIGARCH Comput. Archit. News* 23.1 (Mar. 1995), pp. 20–24. ISSN: 0163-5964. DOI: 10.1145/216585.216588. URL: https://doi.org/10.1145/216585.216588.

[32] J. Zhao. *Chipyard Intro and Fundamentals*. Presentation Slides, ISCA 2021 Tutorial. Accessed: Jun. 6, 2025. 2021. URL: https://fires.im/isca21-slides-pdf/02_chipyard_basics.pdf.

[33] Jerry Zhao et al. "SonicBOOM: The 3rd Generation Berkeley Out-of-Order Machine". In: *2020 Workshop on Computer Architecture Research with RISC-V (CARRV)*. IEEE, 2020. DOI: 10.1109/CARRV51705.2020.9408173.

Appendix A: MÃ NGUỒN CỦA THIẾT KẾ

Appendix B: Detailed Processing Units

B.1 ExecuteController Sub-Modules

The `ExecuteController` is a complex module that contains several specialized hardware units to manage the core computation and data transformation pipeline. The primary sub-modules include:

- **Im2Col Module**: A specialized SRAM controller for convolution operations that converts image data into column format for efficient convolution. It manages sliding window operations over input data, handles address generation for accessing input feature maps, and provides specialized data access patterns for CNN operations.
- **Queue (sram_read_signals_q)**: A pipeline synchronization buffer that buffers control signals for SRAM read operations, provides timing alignment between address generation and data availability, and handles pipeline delays in the Im2Col data path.
- **Queue (mesh_cntl_signals_q)**: A mesh control signal buffer that queues control signals fed into the computational mesh. It synchronizes computation control with data arrival timing and provides pipeline buffering for mesh operations.
- **MultiHeadedQueue**: A multi-port queue for command buffering that allows multiple commands to be read simultaneously. This provides flexible command scheduling and issue capabilities, and is essential for supporting out-of-order execution patterns.
- **TransposePreloadUnroller**: A command transformation unit that unrolls complex operations into simpler primitive operations, handles matrix transpose operations for the weight-stationary dataflow, and manages preload command sequences for optimal data placement.
- **MeshWithDelays (Computational Mesh)**: The main systolic array for matrix computations that performs the actual matrix multiplication operations. It implements the systolic array dataflow (input/weight/output stationary), handles multiple simultaneous matrix multiplications, and manages timing and data flow through the PE array.

B.2 Scratchpad Integrated Processing Units

The Gemmini Scratchpad contains several hardware units that process data on-the-fly as it is moved to or from memory, offloading these tasks from the host CPU. The primary units include:

- `VectorScalarMultiplier (vsm/vsm_1)`: Applies scaling operations to data during movement, with separate scalers for scratchpad and accumulator data paths. This is essential for handling quantized data types.
- `PixelRepeater`: Handles pixel repetition optimizations for first-layer convolution operations, reducing memory bandwidth for specific layer types.
- `AccumulatorScale unit (acc_scale_unit)`: A critical component that applies scaling and activation functions (e.g., ReLU) to accumulator data before it is written back to memory.
- `Normalizer unit (acc_norm_unit)`: Performs normalization operations on accumulator data.
- `ZeroWriter`: Generates zero data when the `all_zeros` flag is set, avoiding unnecessary memory reads and saving power.

Appendix C: RoCC Interface and Gemmini Integration Details

This appendix provides a detailed technical reference for the Rocket Custom Coprocessor (RoCC) interface and its specific implementation for the Gemmini accelerator. The Chisel/Scala code listings below are extracted from the `rocket-chip` and `gemmini` generator source files and serve as the ground-truth definition of the hardware interface and its integration with the CPU pipeline.

C.1 SoC Configuration and Accelerator Instantiation

The integration begins at the SoC configuration level. The Gemmini accelerator is instantiated as a `LazyRoCC` module and is assigned a specific opcode space for its custom instructions.

```
// From generators/gemmini/src/main/scala/gemmini/Controller.scala
class Gemmini[T <: Data : Arithmetic, U <: Data, V <: Data](...)
  extends LazyRoCC (
    opcodes = OpcodeSet.custom3, // Assigns Gemmini to the custom3 space
    nPTWPorts = if (config.use_shared_tlb) 1 else 2
  ) {
  override lazy val module = new GemminiModule(this)
  // ... TileLink node connections
}
```

Listing C.1: Gemmini's extension of the `LazyRoCC` base class, which registers it as a RoCC accelerator within the Chipyard framework.

```
// From rocket-chip/src/main/scala/tile/LazyRoCC.scala
object OpcodeSet {
  def custom0 = new OpcodeSet(Seq("b0001011".U))
  def custom1 = new OpcodeSet(Seq("b0101011".U))
  def custom2 = new OpcodeSet(Seq("b1011011".U))
  def custom3 = new OpcodeSet(Seq("b1111011".U)) // Used by Gemmini
}
```

Listing C.2: Definition of the RISC-V custom opcode spaces available in Rocket Chip.

C.2 RoCC Hardware Interface Bundles

The following Chisel `Bundle` classes define the physical wires that constitute the complete RoCC interface between the CPU and the accelerator.

```
// Source: rocket-chip/src/main/scala/tile/LazyRoCC.scala
class RoCCIO(val nPTWPorts: Int)(implicit p: Parameters)
  extends Bundle {
  // Command channel from CPU to Accelerator
  val cmd = Flipped(Decoupled(new RoCCCommand))

  // Response channel from Accelerator to CPU
  val resp = Decoupled(new RoCCResponse)

  // Direct memory interface to the L1 Data Cache
  val mem = new HellaCacheIO

  // Page Table Walker interface for virtual memory
  val ptw = Vec(nPTWPorts, new TLBPTWIO)

  // Status and Control Signals
  val busy = Output(Bool())
  val interrupt = Output(Bool())
  val exception = Input(Bool()) // Signals an exception from the CPU
}
```

Listing C.3: The top-level hardware bundle for the complete RoCC interface.

```
// Source: rocket-chip/src/main/scala/tile/LazyRoCC.scala
class RoCCCommand(implicit p: Parameters) extends CoreBundle()(p) {
  val inst = new RoCCInstruction     // The decoded instruction
  val rs1 = Bits(xLen.W)             // Data from source register 1
  val rs2 = Bits(xLen.W)             // Data from source register 2
  val status = new MStatus           // Current processor status flags
}
```

Listing C.4: The `RoCCCommand` bundle, defining the payload dispatched from the CPU to the accelerator.

```
// Source: rocket-chip/src/main/scala/tile/LazyRoCC.scala
class RoCCInstruction extends Bundle {
  val funct = Bits(7.W)    // Function code (specifies Gemmini operation)
  val rs2 = Bits(5.W)      // Source register 2 address
  val rs1 = Bits(5.W)      // Source register 1 address
  val xd = Bool()          // Destination register valid bit
  val xs1 = Bool()         // Source register 1 valid bit
  val xs2 = Bool()         // Source register 2 valid bit
  val rd = Bits(5.W)       // Destination register address
```

```
  val opcode = Bits(7.W)  // Opcode (custom3 for Gemmini)
}
```

Listing C.5: The `RoCCInstruction` bundle, defining the raw instruction format.

C.3 Gemmini Custom Instruction Set

The `funct7` field of a RoCC instruction is passed to the accelerator to specify an operation. Listing C.6 shows the definition of the primary commands used by Gemmini. These are high-level directives that can trigger complex sequences within the accelerator.

```
// Source: generators/gemmini/src/main/scala/gemmini/GemminiISA.scala
object GemminiISA {
  // Basic Operations
  val CONFIG_CMD = 0.U
  val LOAD_CMD = 2.U
  val STORE_CMD = 3.U
  val COMPUTE_AND_STAY_CMD = 5.U
  val PRELOAD_CMD = 6.U
  val FLUSH_CMD = 7.U

  // Complex, CISC-like loop commands
  val LOOP_WS = 8.U
  val LOOP_CONV_WS = 15.U

  // Configuration sub-commands
  val CONFIG_EX = 0.U
  val CONFIG_LOAD = 1.U
  val CONFIG_STORE = 2.U
}
```

Listing C.6: The `GemminiISA` object, defining the function codes for Gemmini's instructions.

C.4 CPU-Side Pipeline Integration Details

The following code snippets from the `rocket-chip` source demonstrate the deep integration of the RoCC interface within the Rocket Core's five-stage pipeline.

C.4.1 Command Processing Flow

The CPU pipeline identifies, routes, and dispatches RoCC commands.

```
// From rocket-chip/src/main/scala/rocket/IDecode.scala
// The decoder table maps the CUSTOM3 opcode to a control signal profile
// that uses the register file but not the main ALU.
CUSTOM3 -> List(Y,N,Y,N,N,N,N,N,A2_ZERO,A1_RS1,IMM_X,DW_XPR,...),
```

Listing C.7: Instruction Decode stage logic for recognizing RoCC opcodes.

```
// From rocket-chip/src/main/scala/tile/RocketTile.scala
// The Rocket Tile routes the command from the core's RoCC port
// to the command router, which directs it to the correct accelerator.
if (outer.roccs.size > 0) {
  cmdRouter.get.io.in <> core.io.rocc.cmd
  core.io.rocc.resp <> respArb.get.io.out
  // ... busy and interrupt signal aggregation
}
```

Listing C.8: Command routing from the CPU core to the RoCC accelerator.

C.4.2 Pipeline Stall and Hazard Management

The CPU's hazard unit is modified to stall the pipeline based on the accelerator's status, providing physical evidence of the hardware co-design.

```
// From rocket-chip/src/main/scala/rocket/RocketCore.scala

// Stall the Decode stage if RoCC is busy or not ready
val rocc_blocked = Reg(Bool())
rocc_blocked := !wb_xcpt && !io.rocc.cmd.ready &&
                (io.rocc.cmd.valid || rocc_blocked)
val ctrl_stalld = id_ctrl.rocc && rocc_blocked || ...

// Replay the instruction in the Writeback stage if the
// accelerator is not ready to accept the command.
val replay_wb_rocc = wb_reg_valid && wb_ctrl.rocc &&
                     !io.rocc.cmd.ready
```

Listing C.9: RoCC-specific stall and replay logic from the CPU's hazard unit.

C.4.3 Response and Register File Write

The RoCC interface has a privileged path to write results directly into the CPU's register file, bypassing the main memory hierarchy.

```
// From rocket-chip/src/main/scala/rocket/RocketCore.scala
io.rocc.resp.ready := !wb_wxd // CPU is ready for a response if not writing a GPR

when (io.rocc.resp.fire) {
  // RoCC response takes priority over other units like the divider
  div.io.resp.ready := false.B

  // Route RoCC data directly to the register file write port
  ll_wdata := io.rocc.resp.bits.data
  ll_waddr := io.rocc.resp.bits.rd
  ll_wen := true.B
}
```

Listing C.10: Response arbitration in the Writeback stage.

C.4.4 Status and Interrupt Handling

The CPU and RoCC accelerator coordinate on status and interrupts.

```
// From rocket-chip/src/main/scala/rocket/RocketCore.scala
io.rocc.cmd.bits.status := csr.io.status // Pass current CPU status to RoCC
io.rocc.exception := wb_xcpt             // Signal exceptions to RoCC

// From rocket-chip/src/main/scala/rocket/CSR.scala
// RoCC interrupt is wired into the Machine Interrupt Pending (MIP) register
mip.rocc := io.rocc_interrupt
```

Listing C.11: Status and interrupt coordination logic.

Appendix D: Time Measuring Methodology

Due to Linux does not support reading CPU cycles directly using the rdcycle instruction, the `clock_gettime` function (CLOCK_MONOTONIC) is used to measure elapsed time in nanoseconds. The following C code snippet demonstrates how to read the CPU cycles using this method:

```
// From generators/gemmini/software/gemmini-rocc-tests/include/
   gemmini_testutils.h
static uint64_t read_cycles() {
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
        perror("clock_gettime failed");
        return 0;
    }
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}
```

Exclusive tests were performed to measure the overhead of this method, which was found to be negligible compared to the overall execution time of the tests.

The result shows that the overhead of using `clock_gettime(CLOCK_MONOTONIC)` is the lowest, around 7.4 s, while `gettimeofday` has a similar overhead of around 7.7 s. The `clock_gettime(CLOCK_PROCESS_CPUTIME_ID)` has a higher overhead of around 26.4 s, which is expected as it provides more detailed CPU time information.

```
/*
 * timing_test.c - Test program to compare clock_gettime() vs
   gettimeofday()
 *
 * This program tests the timing functions available on Linux and
   compares
 * their resolution, accuracy, and overhead.
 */

#define _POSIX_C_SOURCE 200809L  // Enable POSIX.1-2008 features
#define _DEFAULT_SOURCE          // Enable additional glibc features

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/time.h>
#include <unistd.h>
```

```c
#include <stdint.h>

// Test how many times we can call each timing function
#define NUM_ITERATIONS 10000

// Function to get time using clock_gettime(CLOCK_MONOTONIC)
uint64_t get_monotonic_time_ns() {
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
        perror("clock_gettime failed");
        return 0;
    }
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}

// Function to get time using gettimeofday()
uint64_t get_timeofday_us() {
    struct timeval tv;
    if (gettimeofday(&tv, NULL) != 0) {
        perror("gettimeofday failed");
        return 0;
    }
    return (uint64_t)tv.tv_sec * 1000000ULL + tv.tv_usec;
}

// Function to get process CPU time
uint64_t get_process_time_ns() {
    struct timespec ts;
    if (clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &ts) != 0) {
        perror("clock_gettime(CLOCK_PROCESS_CPUTIME_ID) failed");
        return 0;
    }
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}

void test_timing_resolution() {
    printf("=== Testing Timing Resolution ===\n\n");

    // Test clock_gettime resolution
    struct timespec res;
    if (clock_getres(CLOCK_MONOTONIC, &res) == 0) {
        printf("CLOCK_MONOTONIC resolution: %ld.%09ld seconds\n",
```

```
58                 res.tv_sec, res.tv_nsec);
59         printf("  = %.1f nanoseconds\n", res.tv_sec * 1e9 + res.tv_nsec)
   ;
60     } else {
61         printf("Failed to get CLOCK_MONOTONIC resolution\n");
62     }
63 
64     if (clock_getres(CLOCK_PROCESS_CPUTIME_ID, &res) == 0) {
65         printf("CLOCK_PROCESS_CPUTIME_ID resolution: %ld.%09ld seconds\n
   ",
66                 res.tv_sec, res.tv_nsec);
67         printf("  = %.1f nanoseconds\n", res.tv_sec * 1e9 + res.tv_nsec)
   ;
68     } else {
69         printf("Failed to get CLOCK_PROCESS_CPUTIME_ID resolution\n");
70     }
71 
72     printf("gettimeofday() resolution: ~1 microsecond (1000 nanoseconds)
   \n");
73     printf("\n");
74 }
75 
76 void test_timing_accuracy() {
77     printf("=== Testing Timing Accuracy (Minimal Operation) ===\n\n");
78 
79     uint64_t start, end, elapsed;
80     volatile int dummy = 0;
81 
82     // Test clock_gettime(CLOCK_MONOTONIC)
83     printf("Testing clock_gettime(CLOCK_MONOTONIC):\n");
84     uint64_t min_mono = UINT64_MAX, max_mono = 0, total_mono = 0;
85 
86     for (int i = 0; i < 1000; i++) {
87         start = get_monotonic_time_ns();
88         dummy = dummy + 1;  // Minimal operation
89         end = get_monotonic_time_ns();
90         elapsed = end - start;
91 
92         if (elapsed < min_mono) min_mono = elapsed;
93         if (elapsed > max_mono) max_mono = elapsed;
94         total_mono += elapsed;
95     }
```

```
96
97      printf("  Min time: %lu ns\n", min_mono);
98      printf("  Max time: %lu ns\n", max_mono);
99      printf("  Avg time: %.1f ns\n", (double)total_mono / 1000.0);
100     printf("  Effective resolution: ~%lu ns\n", min_mono);
101
102     // Test gettimeofday()
103     printf("\nTesting gettimeofday():\n");
104     uint64_t min_gtod = UINT64_MAX, max_gtod = 0, total_gtod = 0;
105
106     for (int i = 0; i < 1000; i++) {
107         start = get_timeofday_us();
108         dummy = dummy + 1;  // Minimal operation
109         end = get_timeofday_us();
110         elapsed = (end - start) * 1000;  // Convert to nanoseconds
111
112         if (elapsed < min_gtod) min_gtod = elapsed;
113         if (elapsed > max_gtod) max_gtod = elapsed;
114         total_gtod += elapsed;
115     }
116
117     printf("  Min time: %lu ns\n", min_gtod);
118     printf("  Max time: %lu ns\n", max_gtod);
119     printf("  Avg time: %.1f ns\n", (double)total_gtod / 1000.0);
120     printf("  Effective resolution: ~%lu ns\n", min_gtod);
121
122     printf("\n");
123 }
124
125 void test_timing_overhead() {
126     printf("=== Testing Timing Function Overhead ===\n\n");
127
128     uint64_t start, end;
129     uint64_t overhead_mono = 0, overhead_gtod = 0, overhead_proc = 0;
130
131     // Test clock_gettime(CLOCK_MONOTONIC) overhead
132     start = get_monotonic_time_ns();
133     for (int i = 0; i < NUM_ITERATIONS; i++) {
134         get_monotonic_time_ns();
135     }
136     end = get_monotonic_time_ns();
137     overhead_mono = (end - start) / NUM_ITERATIONS;
```

```
138
139     // Test gettimeofday() overhead
140     start = get_monotonic_time_ns();
141     for (int i = 0; i < NUM_ITERATIONS; i++) {
142         get_timeofday_us();
143     }
144     end = get_monotonic_time_ns();
145     overhead_gtod = (end - start) / NUM_ITERATIONS;
146
147     // Test clock_gettime(CLOCK_PROCESS_CPUTIME_ID) overhead
148     start = get_monotonic_time_ns();
149     for (int i = 0; i < NUM_ITERATIONS; i++) {
150         get_process_time_ns();
151     }
152     end = get_monotonic_time_ns();
153     overhead_proc = (end - start) / NUM_ITERATIONS;
154
155     printf("Function call overhead (average over %d calls):\n",
    NUM_ITERATIONS);
156     printf("  clock_gettime(CLOCK_MONOTONIC):     %lu ns\n",
    overhead_mono);
157     printf("  gettimeofday():                     %lu ns\n",
    overhead_gtod);
158     printf("  clock_gettime(CLOCK_PROCESS_CPUTIME): %lu ns\n",
    overhead_proc);
159     printf("\n");
160 }
161
162 void test_timing_demo() {
163     printf("=== Practical Timing Example ===\n\n");
164
165     uint64_t start_mono, end_mono;
166     uint64_t start_gtod, end_gtod;
167     uint64_t start_proc, end_proc;
168
169     printf("Timing a simple computation (1000 additions):\n");
170
171     // Time with clock_gettime(CLOCK_MONOTONIC)
172     start_mono = get_monotonic_time_ns();
173     volatile long sum = 0;
174     for (int i = 0; i < 1000; i++) {
175         sum += i;
```

```
176     }
177     end_mono = get_monotonic_time_ns();
178 
179     // Time with gettimeofday()
180     start_gtod = get_timeofday_us();
181     sum = 0;
182     for (int i = 0; i < 1000; i++) {
183         sum += i;
184     }
185     end_gtod = get_timeofday_us();
186 
187     // Time with process CPU time
188     start_proc = get_process_time_ns();
189     sum = 0;
190     for (int i = 0; i < 1000; i++) {
191         sum += i;
192     }
193     end_proc = get_process_time_ns();
194 
195     printf("Results:\n");
196     printf("  clock_gettime(CLOCK_MONOTONIC):     %lu ns (%.2f  s )\n",
197            end_mono - start_mono, (end_mono - start_mono) / 1000.0);
198     printf("  gettimeofday():                     %lu  s  (%.2f  s )\n",
199            end_gtod - start_gtod, (double)(end_gtod - start_gtod));
200     printf("  clock_gettime(CLOCK_PROCESS_CPUTIME): %lu ns (%.2f  s )\n"
    ,
201            end_proc - start_proc, (end_proc - start_proc) / 1000.0);
202     printf("  Sum result: %ld\n", sum);
203     printf("\n");
204 }
205 
206 void show_current_time() {
207     printf("=== Current Time Values ===\n\n");
208 
209     struct timespec ts;
210     struct timeval tv;
211 
212     // Show CLOCK_MONOTONIC
213     if (clock_gettime(CLOCK_MONOTONIC, &ts) == 0) {
214         printf("CLOCK_MONOTONIC: %ld.%09ld seconds\n", ts.tv_sec, ts.
    tv_nsec);
215     }
```

```
216
217     // Show gettimeofday
218     if (gettimeofday(&tv, NULL) == 0) {
219         printf("gettimeofday():  %ld.%06ld seconds\n", tv.tv_sec, tv.
    tv_usec);
220     }
221
222     // Show CLOCK_PROCESS_CPUTIME_ID
223     if (clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &ts) == 0) {
224         printf("PROCESS_CPUTIME: %ld.%09ld seconds\n", ts.tv_sec, ts.
    tv_nsec);
225     }
226
227     printf("\n");
228 }
229
230 int main() {
231     printf("Linux Timing Functions Test Program\n");
232     printf("==================================\n\n");
233
234     printf("This program tests the timing functions used in the Gemmini\
    n");
235     printf("Linux port to replace the rdcycle instruction.\n\n");
236
237     show_current_time();
238     test_timing_resolution();
239     test_timing_accuracy();
240     test_timing_overhead();
241     test_timing_demo();
242
243     return 0;
244 }
```

Listing D.1: C Code for Timing Test

```
1 debian@debian:~$ ./timing_test
2 Linux Timing Functions Test Program
3 ==================================
4
5 This program tests the timing functions used in the Gemmini
6 Linux port to replace the rdcycle instruction.
7
8 === Current Time Values ===
```

```
9
10 CLOCK_MONOTONIC: 288.119331000 seconds
11 gettimeofday():  1710323247.363538 seconds
12 PROCESS_CPUTIME: 0.039620000 seconds
13
14 === Testing Timing Resolution ===
15
16 CLOCK_MONOTONIC resolution: 0.000000001 seconds
17   = 1.0 nanoseconds
18 CLOCK_PROCESS_CPUTIME_ID resolution: 0.000000001 seconds
19   = 1.0 nanoseconds
20 gettimeofday() resolution: ~1 microsecond (1000 nanoseconds)
21
22 === Testing Timing Accuracy (Minimal Operation) ===
23
24 Testing clock_gettime(CLOCK_MONOTONIC):
25   Min time: 6000 ns
26   Max time: 133000 ns
27   Avg time: 7505.0 ns
28   Effective resolution: ~6000 ns
29
30 Testing gettimeofday():
31   Min time: 7000 ns
32   Max time: 133000 ns
33   Avg time: 8165.0 ns
34   Effective resolution: ~7000 ns
35
36 === Testing Timing Function Overhead ===
37
38 Function call overhead (average over 10000 calls):
39   clock_gettime(CLOCK_MONOTONIC):      7393 ns
40   gettimeofday():                      7666 ns
41   clock_gettime(CLOCK_PROCESS_CPUTIME): 26380 ns
42
43 === Practical Timing Example ===
44
45 Timing a simple computation (1000 additions):
46 Results:
47   clock_gettime(CLOCK_MONOTONIC):      69000 ns (69.00  s )
48   gettimeofday():                      69  s  (69.00  s )
49   clock_gettime(CLOCK_PROCESS_CPUTIME): 128000 ns (128.00  s )
```

```
50 Sum result: 499500
```

Listing D.2: Output of Timing Test

Appendix E: Detailed Performance Evaluation Results

This appendix provides the comprehensive, unabridged results of the performance evaluation conducted on the implemented SoC. The data was generated using the `gemmini-rocc-tests` benchmark suite on the Xilinx VC707 FPGA platform.

E.1 Performance Benchmark Results

Table E.1: Detailed Matrix Multiplication Performance

Test	Matrix Size	CPU Cycles	Gemmini Cycles	Speedup
tiled_matmul_os	-	2,036,450,432	38,780,800	52.5×
tiled_matmul_ws	512×512×512	2,488,827,520	20,438,400	121.8×
tiled_matmul_ws_At	-	4,113,003,392	12,444,800	330.5×
tiled_matmul_ws_Bt	-	510,343,424	12,361,600	41.3×
tiled_matmul_ws_full_C	-	4,064,579,328	22,064,000	184.2×
tiled_matmul_ws_low_D	-	4,264,697,856	12,096,000	352.6×

E.1.1 Matrix Multiplication Performance Analysis

Table E.2: Matrix Multiplication Performance Analysis

Test	Matrix Config	Total MACs	Cycles	Ideal Cycles	Util
tiled_matmul_ws_perf	128×256×512	16,777,216	2,425,600	65,536	

Table E.3: Detailed Convolution Performance

Test	Input Size	Output Size	CPU Cycles	
conv	224×224	112×112	44,931,244,800	
conv_first_layer	224×224	112×112	44,727,939,200	
conv_perf	224×224 (4 batch, 3→32 ch)	112×112	-	
conv_dw	112×112	56×56	1,536,294,400	
conv_dw_perf	112×112 (3 batch, 17 ch)	56×56	-	
conv_rect	224×224	112×112	44,761,961,600	
conv_stride	224×224	-	45,357,881,600	
conv_trans_output_1203	-	-	131,830,361,600	
conv_trans_weight_0132	-	-	169,097,846,400	
conv_trans_weight_1203	-	-	134,060,240,000	
conv_with_rot180	-	-	93,918,979,200	

Table E.4: Detailed Convolution with Pooling Performance

Test	Input Size	Output Size	CPU Conv	CPU Pool	CPU To
conv_with_pool	224×224	112×112→56×56	44,810,374,400	4,024,304,000	48,834,678,4
conv_rect_pool	224×224	112×112→56×56	44,755,398,400	4,014,457,600	48,769,856,0

Table E.5: Detailed Residual Addition Performance

Test	Matrix Size	Cycles
resadd	128×512	1,382,400
resadd_stride	-	217,641,600
resadd_stride	-	726,400

E.2 Test Program Documentation and Configurations

This section documents the specific configuration of each test program used in the evaluation, ensuring full reproducibility of the results.

E.2.1 Matrix Multiplication Tests (bareMetalC/)

E.2.1.1 Basic Matrix Multiplication Tests

- `tiled_matmul_os`: Output-stationary matrix multiplication
  - Uses Gemmini's output-stationary dataflow
  - Matrix dimensions: 512×512×512 (non-baremetal)
- `tiled_matmul_ws`: Weight-stationary matrix multiplication
  - Uses Gemmini's weight-stationary dataflow (most efficient)
  - Matrix dimensions: 512×512×512 (I×J×K)

E.2.1.2 Matrix Multiplication Variants

- `tiled_matmul_ws_At`: Weight-stationary with A-transpose

Table E.6: Detailed MLP Model Performance

Model	Layer Distribution	Total Cycles
mlp1_32	6 layers (22.4M, 77.1M, 46.1M, 22.5M, 6.2M, 0.5M)	174,870,400
mlp1	6 layers (22.4M, 77.3M, 45.7M, 18.7M, 6.2M, 0.5M)	170,636,800
mlp4	2 layers (215.6M, 217.2M)	432,838,400

  - Same as `tiled_matmul_ws` but with matrix A transposed
  - Matrix dimensions: 500×300×412 (I×J×K)

- `tiled_matmul_ws_Bt`: Weight-stationary with B-transpose
  - Same as `tiled_matmul_ws` but with matrix B transposed

- `tiled_matmul_ws_full_C`: Weight-stationary with full accumulator width
  - Uses full-width accumulator for output matrix C

- `tiled_matmul_ws_low_D`: Weight-stationary with bias matrix
  - Includes bias matrix D in computation (C = A×B + D)

E.2.1.3 Performance Analysis Tests

- `tiled_matmul_ws_perf`: Weight-stationary performance analysis
  - Matrix dimensions: 128×256×512 (I×J×K)
  - **Verified metrics**:
    * Total MACs: 16,777,216
    * Cycles: 2,425,600 | Ideal: 65,536 | **Utilization: 2%**
    * Memory traffic: 2.16MB RDMA reads, 32KB WDMA writes

E.2.2 Convolution Tests (bareMetalC/)

E.2.2.1 Standard Convolution Tests

- `conv`: Basic 2D convolution operation
  - Input: 224×224×3 (batch=4), Output: 112×112×32
  - Kernel: 3×3, Stride: 2, Padding: 1

- `conv_first_layer`: First layer convolution (typical CNN entry)
  - Same configuration as `conv` but optimized for first layer
  - Input: 224×224×3 (batch=4), Output: 112×112×32

- `conv_rect`: Rectangular convolution
  - Tests non-square convolution operations

- Input: 224×224×3 (batch=4), Output: 112×112×32

- `conv_stride`: Strided convolution
  - Tests convolution with stride=2
  - Input: 224×224×3 (batch=4), Output: 112×112×32

E.2.2.2 Depthwise Convolution Tests

- `conv_dw`: Depthwise convolution
  - Input: 112×112×17 (batch=3), Output: 56×56×17
  - Lower computational intensity than standard convolution

E.2.2.3 Transformed Convolution Tests

- `conv_trans_output_1203`: Output transformation (1,2,0,3)
  - Tests convolution with output tensor transformation
  - Input: 224×224×17 (batch=4), Output: 112×112×32

- `conv_trans_weight_0132`: Weight transformation (0,1,3,2)
  - Tests convolution with weight tensor transformation
  - Input: 224×224×17 (batch=4), Output: 112×112×32

- `conv_trans_weight_1203`: Weight transformation (1,2,0,3)
  - Alternative weight tensor transformation
  - Input: 224×224×17 (batch=4), Output: 112×112×32

E.2.2.4 Convolution with Pooling Tests

- `conv_with_pool`: Convolution followed by pooling
  - Input: 224×224×3 (batch=4), Conv output: 112×112×32, Pool output: 56×56×32

- `conv_rect_pool`: Rectangular convolution with pooling
  - Same configuration as `conv_with_pool`

E.2.2.5 Special Convolution Tests

- `conv_with_rot180`: Convolution with 180° rotation
  - Tests convolution with rotated input
  - Input: 224×224×3 (batch=4), Output: 224×224×17 (stride=1, no size reduction)

E.2.3 Neural Network Models (imagenet/)

E.2.3.1 MobileNet v1 Implementation

- `mobilenet`: Complete MobileNet v1 inference
  - Architecture: Standard convolution + 13 depthwise separable convolution blocks
  - Input: 224×224×3 ImageNet images

E.2.4 Multi-Layer Perceptron Models (mlps/)

E.2.4.1 MLP1 Family

- `mlp1`: 6-layer MLP with RELU activation
  - Architecture: 832→2560→2048→1536→1024→512→64
  - Original dimensions: 784→2500→2000→1500→1000→500→10 (zero-padded)
  - Batch size: 64
  - **Total cycles**: 170,636,800
  - **Layer breakdown**: 22.4M, 77.3M, 45.7M, 18.7M, 6.2M, 0.5M cycles
- `mlp1_32`: 32-bit version of MLP1
  - Same architecture as `mlp1` but with 32-bit precision
  - **Total cycles**: 174,870,400
  - **Layer breakdown**: 22.4M, 77.1M, 46.1M, 22.5M, 6.2M, 0.5M cycles

E.2.4.2 MLP4 Family

- `mlp4`: 2-layer MLP with RELU activation
  - Architecture: 3072→4608→3072
  - Original dimensions: 3036→4554→3036 (zero-padded)
  - Batch size: 64
  - **Total cycles**: 432,838,400
  - **Layer breakdown**: 215.6M, 217.2M cycles

E.2.5 Additional Tests

- `resadd`: Residual addition operations
  - Tests element-wise addition for residual connections
  - **Cycles**: 1,382,400 (128×512 matrix)
- `gemmini_counter`: Hardware counter validation
  - Tests Gemmini's performance counter functionality
  - Used for performance monitoring and debugging